Diskret matematikk 2

Fakultet for teknologi og realfag

Universitetet i Agder

Grimstad, desember 2023

Contents

List of Figures

1 Introduction and Concepts

Symbol	Description
$\exists$	meaning "there exists".
$\forall$	meaning "for all".
$\emptyset$ or $\varnothing$	empty set - a set with no elements.
$\mathbb{N}$	The set of natural numbers (0, 1, 2, 3).
$\mathbb{Z}$	The set of integers (-2, -1, 0, 1, 2).
$\mathbb{Q}$	The set of rational numbers. Tall som kan uttrykkes som brøk $\frac{a}{b}$ hvor $b! = 0$
$\mathbb{R}$	The set of real numbers. Alle mulige tall
$\{\}$	Denotes a set.
$x \in S$	"x is an element of set S".
$A \subseteq B$	"A is a subset of B", meaning all elements of A are in B.
$A \subset B$	"A is a proper subset of B", meaning A is a subset of B but not equal to B. This means that set B contains
$p \wedge q$	Logical "and", true if both p and q are true.
$p \vee q$	Logical "or", true if either p or q (or both) are true.
$\neg p$	Logical negation, true if p is not true.
$\implies$	Used to denote logical implication. The statement "P $\implies$ Q" means "If P is true, then Q is also true".
$\iff$	Represents logical equivalence. The statement $P \iff Q$ means, P is true if and only if Q is true.

1.1 Aksiomer

Et aksiom er en *statement* eller regel som er akseptert som sann uten behov for bevis. De kan sees på som grunnleggende sannheter som er fundamentet for hele systemer. De er de grunnleggende blokkene som brukes for å lage teoremer, og bevise andre statements i systemet.
Eksempel:
I Euclidisk gemeotri er ett av aksiomene at det gjennom to punkter går en rett linje. Dette brukes som et grunnlag for videre geometrisk argumentasjon.
Eksempel 2:
Innen logikk kan et eksempel på et aksiom være: "Hvis A er sann, og A innebærer (implies) B, så er B også sann". Grunnleggende innen logisk deduksjon.

1.2 Statement

1.3 Definisjon

- En definisjon er et sett med statements (formler), som utvetydig kvalifiserer hva et matematisk uttrykk er, og hva det ikke er.
- En definisjon kan definere et matematisk objekt (for eksempel en gruppe) eller en egenskap (positivitet) til et matematisk objekt.
- Definisjoner må være gyldige (veldefinerte). Dette innebærer at de er komplette, og ikke introduserer motsetninger.

1.4 Theorem and proof

- Et teorem er en formel som er sann.
- Vi forventer argumentasjon som viser hvordan sannheten til teoremet er derivert fra tidligere kunnskap.
- Denne argumentasjonen kalles *proof*

1.5 Characterizations

- En karakterisering er et sett med formler som definerer et matematisk objekt på en unik måte. Dette er logisk likt, men distinkt fra, definisjonen til et objekt.
- Hver karakterisering trenger bevis (korrekthet)

•

1.6 Propositional logic (Syntax)

The propositional calculus is a system handling propositions (or sentences) and their relations.

1. A propositional symbol standing alone is a well-formed proposition
2. If any proposition A (such as $(P \vee Q)$) is well-formed, then so is its negation $\neg A$; $(\neq(P \vee Q))$.
3. If A and B are well-formed propositions, then so are (A∧B), (A∨B), and (A→B)

Another angle:
Propositional logic, also known as propositional calculus, is the **study of statements** that **can either be true or false, but not both simultaneously**. These **statements** are known as **propositions**. For example, "it is raining" is a proposition, because it can clearly be true or false.

The power of propositional logic comes from combining these simple propositions using logical operators to form more complex expressions. The most common operators include

- **AND (Conjunction)**: It implies that two statements must be true together. For example, "it is raining AND it is cold".
- **OR (Disjunction)**: Implies that at least one of the statements is true. For example, "it is raining OR it is sunny".
- **NOT (Negation)**: Reverses the truth of a statement. For example, "it is NOT raining".

1.6.1 Conditional

The **logical conditional** is the equivalent of the expression **"If A then B"**. The result is true if it is consistent with that statement. The only inconsistent situation is if B is false when A is true. This contradicts the conditional statement.

P	**Q**	**P ⟹ Q**
0	0	1
0	1	1
1	0	0
1	1	1

1.7 Predicate logic (Syntax)

Predicate logic extends propositional logic with quantifiers, allowing variables of non-logical objects

1. A variable is a well-formed **term**
2. A function application $f(t_1, t_2, ...t_n)$ is a **term** if all t_i are terms (also nullary functions = constants)
3. A predicate application $P(t_1, t_2, ..., t_n)$ is a **formula** if all t_i are terms
4. An equality $t_1 = t_2$ is a **formula** if t_1 and t_2 are terms.
5. If F is a formula, then so is its negation $\neg F$. If A and B are formulas, then so are (A∧B), (A∨B), and (A→B).
6. If F is a formula and x is a variable, then $\forall xF$ and $\exists xF$ are formulas.

Another angle:
Predicate logic, or first-order logic, extends the concepts of propositional logic by introducing quantifiers and variables. While propositional logic deals with whole statements, predicate logic breaks them down into more granular components, like subjects and predicates, and examines their internal structure.

The key elements of predicate logic are:

- **Predicates:** These are functions that return true or false. They are like propositions but can include variables. For example, "it is raining in (Place)" where (Place) is a variable

- **Quantifiers:** These define the scope of the variables. The two main quantifiers are:
  - **Universal Quantifier ∀:** It means "for all" or "every". For example, "For EVERY place, it is raining" (∀ places, it is raining).
  - **Existential Quantifier (∃:** It means "there exists" or "some". For example, "There exists a place where it is raining" (∃ a place, it is raining).

1.8 Connectives

Connective	Symbol	Description
Negation	$\neg$	NOT
Conjunction	$\wedge$	AND
Disjunction	$\vee$	OR
Conditional	$\Longrightarrow$	If ... then
Biconditional	$\Longleftrightarrow$	If and only if

Table 1: Logical Connectives and Symbols

1.8.1 Truth table

In order to clarify the meaning of a proposition or a connective, a **truth table** is used. Truth tables help visualizing the truth values of propositions. A value of *true* is represented by a "1" and a value of $false$ is represented by a "0".

1.8.2 Conjunction

Take the followinng proposition:
Marty wears green boots, and Marty has a dog.

- If Marty doesn't wear green boots and doesn't have a dog, then the proposition is **false**.
- If Marty doesn't wear green boots but has a dog, then the proposition is **false**.
- If Marty wears green boots, but doesn't have a dog, then the proposition is **false**.
- If Marty wears green boots and has a dog, then the proposition is **true**

We can represent this in a truth table:

P	Q	$P \wedge Q$
0	0	0
0	1	0
1	0	0
1	1	1

1.8.3 Disjunction

This follows **OR**.
Example: *The elephants are green, OR George wears red booth (or both)*

P	Q	$P \vee Q$
0	0	0
0	1	1
1	0	1
1	1	1

1.8.4 Conditional

The **logical conditional** is the equivalent of the expression "If **A** then **B**". The only inconsistent situation is if B is false when A is true.

P	Q	$P \to Q$
0	0	1
0	1	1
1	0	0
1	1	1

1.8.5 Biconditional

A biconditional is a connective that represents the condition "if and only if". It checks for whether both of the propositions evaluate to the same truth value. Can also be though of as

$$(A \implies B) \wedge (B \implies A$$

P	Q	$P \leftrightarrow Q$
0	0	1
0	1	0
1	0	0
1	1	1

1.8.6 Negation

Read as "not" P.

P	$\neg P$
1	0
0	1

1.8.7 Summary so far

P	Q	$\neg$ P	P $\wedge$ Q	P $\vee$ Q	P $\to$ Q	P $\leftrightarrow$ Q
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Figure 1: Summary of everything

1.8.8 Well formed formula

We can construct more complex propositions by combining the above simple propositions to construct an infinite number of combinations such as

$$\neg[(A \wedge B) \vee C]$$

A proposition is called a "well formed formula" if it is constructed with the following set of rules:

1. Any atomic proposition is a well formed formula.
2. If A is a well formed formula, then $\neg A$ is also a well formed formula.
3. If A and B are well formed formulae, then $(A \wedge B)$ is also a well formed formula.
4. If A and B are well formed formulae, then $(A \vee B)$ is also a well formed formula.
5. If A and B are well formed formulae, then $(A \implies B)$ is also a well formed formula.
6. If A and B are well formed formulae, then $(A \iff B)$ is also a well formed formula.
7. Unless constructed using only 1-6 above, then a proposition isn't a well formed formula.

The use of '( )' and '[ ]' are for grouping subexpressions. This is similar to the parantheses used in arithmetic. Square brackets often serve the same purpose as parantheses, and are used for additional levels of grouping, particularly when the expression already contains multiple sets of parantheses, to improve readability. Somethimes, different types of brackets are used to denote different levels of priority. For example '$[p \wedge (Q \vee R)] \implies S$

1.8.9 Tautology

A proposition is a **tautology** if it is true for all possible truth assignments of the atomic propositions involved.

Example: Show that this proposition is a **tautology:**

$$((A \wedge B) \implies C) \iff (A \implies (B \implies C))$$

A	B	C	$A \wedge B$	$((A \wedge B) \to C)$	$B \to C$	$(A \to (B \to C))$
0	0	0	0	1	1	1
0	0	1	0	1	1	1
0	1	0	0	1	0	1
0	1	1	0	1	1	1
1	0	0	0	1	1	1
1	0	1	0	1	1	1
1	1	0	1	0	0	0
1	1	1	1	1	1	1

By looking at the truth values in column 5 and 7, it can be seen that this is a tautology.

1.8.10 Contradiction

A contradiction is a statement that is false for all possible truth assignments of the atomic propositions involved. One classic example of a contradiction is a negation of a tautology. So the following is a contradiction.

$$\neg(((A \wedge B) \implies C) \iff (A \implies (B \implies C)))$$

1.8.11 Contingency

A proposition is **contingent** if and only if it is neither a **contradiction** nor a **tautology**

1.8.12 Logical Equivalence

Two propositions A and B are **equivalent** if and only if $A \implies B$ is a tautology. While the biconditional tests whether the two propositions are of equal value for a particular assignment of truth values, the equivalence is the test for all possible truth value assignments.

1.8.13 Validity

There are two ways in which an argument can go wrong:

1. The premise is false
2. The logical structure of the argument is wrong.

Since, in propositional logic, we cannot do anything to determine whether the premise is actually true, we define the validity of an argument in terms of the second idea.

$P_1, P_2, ..., P_N$ is said to **entail** C if and only if there is no truth assignment for which P_1, P_2, ..., P_n are all true and C is false.

We denote entailment as

$$\{P_1, P_2, \ldots, P_n\} \models C.$$

1.8.14 Logical Identities (logical inference):

Premise is the first part, while **conclusion** is the second.

1.	$P \vee \sim P$	Law of the excluded middle
2.	$\sim (P \wedge \sim P)$	Contradiction
3.	$[(P \rightarrow Q) \wedge P] \rightarrow Q$	Modus ponens
4.	$[(P \rightarrow Q) \wedge \sim Q] \rightarrow \sim P$	Modus tollens
5.	$\sim\sim P \leftrightarrow P$	Double negation
6.	$[(P \rightarrow Q) \wedge (Q \rightarrow R)] \rightarrow (P \rightarrow R)$	Law of the syllogism
7.	$(P \wedge Q) \rightarrow P$ $(P \wedge Q) \rightarrow Q$	Decomposing a conjunction Decomposing a conjunction
8.	$P \rightarrow (P \vee Q)$ $Q \rightarrow (P \vee Q)$	Constructing a disjunction Constructing a disjunction
9.	$(P \leftrightarrow Q) \leftrightarrow [(P \rightarrow Q) \wedge (Q \rightarrow P)]$	Definition of the biconditional
10.	$(P \wedge Q) \leftrightarrow (Q \wedge P)$	Commutative law for $\wedge$
11.	$(P \vee Q) \leftrightarrow (Q \vee P)$	Commutative law for $\vee$
12.	$[(P \wedge Q) \wedge R] \leftrightarrow [P \wedge (Q \wedge R)]$	Associative law for $\wedge$
13.	$[(P \vee Q) \vee R] \leftrightarrow [P \vee (Q \vee R)]$	Associative law for $\vee$
14.	$\sim (P \vee Q) \leftrightarrow (\sim P \wedge \sim Q)$	DeMorgan's law
15.	$\sim (P \wedge Q) \leftrightarrow (\sim P \vee \sim Q)$	DeMorgan's law
16.	$[P \wedge (Q \vee R)] \leftrightarrow [(P \wedge Q) \vee (P \wedge R)]$	Distributivity
17.	$[P \vee (Q \wedge R)] \leftrightarrow [(P \vee Q) \wedge (P \vee R)]$	Distributivity
18.	$(P \rightarrow Q) \leftrightarrow (\sim Q \rightarrow \sim P)$	Contrapositive
19.	$(P \rightarrow Q) \leftrightarrow (\sim P \vee Q)$	Conditional disjunction
20.	$[(P \vee Q) \wedge \sim P] \rightarrow Q$	Disjunctive syllogism
21.	$(P \vee P) \leftrightarrow P$	Simplification
22.	$(P \wedge P) \leftrightarrow P$	Simplification

Figure 2: Logical inference

1.8.15 Laws of logic

No.	Law	Name
1	$\neg\neg p \leftrightarrow p$	Law of Double Negation
2	$\neg(p \vee q) \leftrightarrow (\neg p \wedge \neg q)$ $\neg(p \wedge q) \leftrightarrow (\neg p \vee \neg q)$	DeMorgan's Laws
3	$p \vee q \leftrightarrow q \vee p$ $p \wedge q \leftrightarrow q \wedge p$	Commutative Laws
4	$p \vee (q \vee r) \leftrightarrow (p \vee q) \vee r$ $p \wedge (q \wedge r) \leftrightarrow (p \wedge q) \wedge r$	Associative Laws
5	$p \vee (q \wedge r) \leftrightarrow (p \vee q) \wedge (p \vee r)$ $p \wedge (q \vee r) \leftrightarrow (p \wedge q) \vee (p \wedge r)$	Distributive Laws
6	$p \vee p \leftrightarrow p$ $p \wedge p \leftrightarrow p$	Idempotent Laws
7	$p \vee F_0 \leftrightarrow p$ $p \wedge T_0 \leftrightarrow p$	Identity Laws

Table 2: Logical Equivalences
For any primitive statements p, q, r, any tautology T_0, and any contradiction F_0.

1.8.16 Consistency

A set of propositions is i**inconsistent** if it cannot be simultaneously all true. Otherwise, it is **consistent**.

1.9 Quiz

Question 1

What can be used as valid arguments in proofs?

- Unproven Theorems
- Axioms ✓
- Proofs
- Concepts
- Definitions ✓
- Proven Theorems ✓
- Interpretations

Question 2

1 / 1 pts
Consider the Peano axiom set and a theorem T that is proven to be true.
What happens if we make T an extra axiom?

- The resulting theory is inconsistent.
- The resulting theory has the same true and false statements. ✓
- The resulting theory is incomplete.
- The resulting theory has more false statements.
- The resulting theory has more true statements.

Question 3

1 / 1 pts
Is $a + b = b + a$ true for $\mathbb{N}_4$?

- No, this does not work for negative numbers.
- Yes, we can see it from the addition table. ✓
- Yes, this is common sense.
- This is undecidable because there are infinitely many numbers in $\mathbb{N}_4$.

Question 4

1 / 1 pts
How do Python and Java compare in terms of expressibility?

- Java has virtual code, so it covers more than Python.
- Python is interpreted, so it covers more than Java.
- Java has virtual code, so it covers less than Python.
- Python is interpreted, so it covers less than Java.
- The Church thesis states that both are the same. ✓

Question 5

1 / 1 pts
You should write a compiler, and the task includes making sure it always stops. Is this possible?

- The compiler has to be able to also compile a compiler and therefore this is impossible.
- The halting problem tells us that this is impossible.
- This is possible using formal methods, for example refinement. ✓
- This is possible if you use enough test cases to make sure the compiler works as expected.

Question 6

1 / 1 pts
Is it possible to decide halting for routines without input?

- Yes, because only the empty input has to be considered. ✓
- Yes, if the code of the routine is finite.
- No, the halting problem states that this is not possible.
- No, because the output can be infinite.

Question 7

1 / 1 pts
What is the truth value of the following formula?
$(1 = 2 \to 3 = 4) \to 5 = 6$

- True
- False ✓

The reason that this is the case, is:

- $1 = 2$ is false, so is $3 = 4$. $False \implies False = True$
- $5 = 6$ is false.
- $True \implies False = False$

$1 = 2$ is

Question 8

1 / 1 pts
Sort the following sets according to their subset relation, starting with the biggest. If they are not related, mark them as unrelated.

- All prime numbers: **unrelated**
- All natural numbers: **size a - biggest**
- All even natural numbers: **size b**
- All square numbers: **unrelated**
- All natural numbers divisible by 6: **size c**
- All natural numbers divisible by 1212: **size d**
- The empty set: **size e**
- All cubic numbers: unrelated

Question 9

1 / 1 pts
What happens if an axiom is false?

- If an axiom is false, all other axioms become false as well.
- By definition, an axiom is always true. ✓
- A false axiom makes the theory incomplete.
- A false axiom makes the theory inconsistent.

Question 10

1 / 1 pts
We consider the open statements $p(x) = x \geq 0$ and $q(x) = x^2 \geq 0$.
Which of the following statements describe the formula $\forall x[p(x) \rightarrow q(x)]$?

- For every real number x, if x>=0 then x∧2 >=0. ✓
- All nonnegative real numbers have nonnegative squares. ✓
- When a square is negative, then its number is also negative.
- The square of any nonnegative real number is a nonnegative real number. ✓
- All numbers are nonnegative.
- Every nonnegative real number has a nonnegative square. ✓
- Some nonnegative real numbers have nonnegative squares.
- All squares are nonnegative.

2 Induction and Recursion

2.1 Well-ordering principle

Definition: Every non-empty set S of positive integers has a smallest element. This means that there exists an element $m \in S$ such that $m \leq n$ for every $n \in S$. Often user as an **axiom** in reasoning.

Property as a set: A property can often be represented as a set of elements that satisfy that property. For instance, if we consider a property P over the natural numbers, we can define a set S containing all natural numbers that satisfy P.

Applying the Well-Ordering Principle: If our set S, representing the property P is non-empty, the Well-Ordering Principle, guarantees the existence of the smallest element in S. This element is the minimum natural number that satisfies the property P.

Example with Function: If f is a function from the natural numbers $\mathbb{N}$ to $\mathbb{N}$, the range of f is a set of natural numbers. According to the Well-Ordering Principle, this range (which is a set of numbers, representing the property of being output by f) has a smallest element. Thus, there exists an element in $\mathbb{N}$ which is the smallest value that f maps to.

PS: 0 IS NOT A NATURAL NUMBER

2.2 Peanos axioms

1. 0 is a number.
   $0 \in N$
   This axiom establishes a starting point for the set of natural numbers.
2. The successor of any number is a number.
   $\forall n(n \in N \rightarrow s(n) \in N)$
   Every natural number n has a successor, denoted as *S(n)*. The successor of a natural number is also a natural number.
3. if a and b are numbers, and their successors are equal, then a and b are equal.
   $\forall a \forall b(a \in N \land b \in N \land s(a) = s(b) \rightarrow a = b)$.
   This is commonly referred to as *distinctness of successors*. This axiom prevents two different numbers from having the same successor.
4. 0 is not the successor of any number.
   $\neg \exists n(n \in N \land s(n) = 0)$
   This distinguishes 0 as the first natural number.
5. If S is a set of numbers containing 0, and if the successor of any number in S is contained in S as well, then S contains all the numbers.
   $\forall S(0 \in S \land \forall n(n \in S \rightarrow s(n) \in S) \rightarrow N \subseteq S)$
   Because the successor of 0 is also included, everything is included.

S(n) is referred to as the **successor function**. It takes a natural number as an argument, and returns the next successive natural number as a result. In ordinary arithmetic notation this is

$$S(n) = n + 1$$

However, this notation is avoided when using the successor function. The goal is typically to define mathematical operations and results purely through the successor function, in particular, as a series of nested successor functions. For example:

1. = S(0)
2. = S(S(0))
3. = S(S(S(0)))

2.3 Peano addition

1. x + 0 = x
2. x + S(y) = S(x+y)

Example: Find S(S(0)) + S(S(S(0)))

1. S(S(0)) + S(S(S(0)))
2. S(S(S(0))) + S(S(0))
3. S(S(S(S(0)))) + S(0)
4. S(S(S(S(S(0)))))

This is actually 2 + 3 = 5 in normal symbols.

2.4 Peano multiplication

1. x * 0 = 0
2. x * S(y) = x + (x * y)

Example: Find 2 x 3

1. 2 x 3
2. = 2 + (2 x 2)
3. = 2 + (2 + (2 x 1))
4. = 2 + (2 + (2 + (2 x 0)))
5. = 2 + (2 + (2 + 0)
6. = 2 + (2 + (2))
7. = 2 + 4
8. = 6

2.5 Induction principle (Peano)

If a statement (property) about natural numbers is true for 0 and its truth for k implies the truth for k+1 (called s(k)), then it is true for all natural numbers.

We use three steps for inductive proofs.

1. Prove the property initially for 0 (or the start value(s))
2. Assume the implication condition
3. Prove the implication consequence

2.6 Recursion principle

If a function over natural numbers is defined for 0 and its value for k+1 can be calculated from its value for k, then it is defined for all natural numbers.

2.7 Let's be stupid

2.7.1 Recursive definition:

A recursive definition has two parts

- **Initial definition:** This is like the starting step of the recipe. It gives the basic beginning information needed. For example, if you're describing how to climb stairs, the initial definition is: "Start by standing on the ground".
- **Recursion:** This part is like repeating steps in the recipe. It tells you how to move from one step to the next. Continuing with the stairs example: "To reach the next floor, go up one step from where you are".

2.7.2 Explicit Definition (Characterization)

An explicit definition is more straightforward. It's like a simple formula or rule that tells you exactly what to do without repeating steps. Using the stairs example again: "The 10th floor is exactly 10 steps up from the ground floor". Here you have a direct way to find out where the 10th floor is without going through all the previous steps.

2.7.3 Comparing recursive and explicit definitions with induction:

When you want to see how a recursive definition (the step-by-step process) matches up with an explicit definition (the direct formula), you use induction. This is like checking every step of the recipe to make sure it leads to the right outcome. The method uses two principles:

- **Base step:** Verify that the statement holds true for the initial value, establishing the credibility of the proposition at its fundamental level.
- **Inductive step:** Assuming the statement is valid for a certain case and then proving its validity for the next case, thereby demonstrating the consistency and reliability of the proposition across all iterations.

2.8 String over an alphabet A

An alphabet is a set of symbols (which we often call letters).

- ε (the empty string) is a string
- If s is a string and l is a letter from the alphabet A, then $s \circ l$ (appending l at the end of s) is a string.
- **String exclusivity:** The only string that exist are those that can be formed by repeatedly appending letters from the alphabet A to the empty string ε, or by appending letters to the result of previous such operations.
- **Proof and Definitions by Induction and Recursion:** Because strings are built up from the empty string by successively adding one letter at a time, you can use mathematical induction to prove properties about strings. Similarly, you can define functions or properties of strings using recursion, a method where the function is defined in terms of itself, using simpler instances of the same function.

A simple recursion could be defining the length of a string: the length of the empty string ε is 0, and the length of any string $s \circ l$ is one more than the length of s.

2.9 String theory (no, not that kind)

2.9.1 Foundation

- **Symbol:** a, b ...
- **Alphabet (Σ):** A finite, nonempty set of symbols.
- **String (informal):** Finite number of symbols "put together.
- **Empty string (ε)**
- **$\Sigma*$:** Set of all strings over alphabet Σ.
  For example, {a, b}* = $\{\varepsilon, a, b, aa, ab, ...\}$

- **Length of string (|s|):** |abba| = 4, |a| = 1, $|\varepsilon| = 0$
  The set of strings of length n is denoted Z^n

2.9.2 Concatination of strings

- Written as $x \cdot y$ or just xy
- Just follow the symbols of x by the symbols of y
  $x = abba$, $y{=}b \implies xy = abbab$
- $x\varepsilon = \varepsilon x$ for any x
- The **reversal** x^R of a string x, is x written backwards.

2.9.3 Formal Inductive Definitions

Strings and their lengths:

- **Base case:** ε is a string of length 0.
- **Induction:** If x is a string of length n and $\sigma \in \Sigma$, then $x\sigma$ is a string of length $n+1$. So, x is a string of length n, and σ is an individual character in the language Σ, then the length of $x\sigma$ is $x+1$, which gives $n+1$
- We start with ε and add one symbol at a time, like $\varepsilon aaba$, but we don't write the initial ε unless the string is empty.

2.9.4 Proofs by Induction

To prove $P(n)$ for all $n \in \mathbb{N}$

1. **Base Case:** Prove $P(0)$
2. **Induction hypothesis:** Assume that $P(k)$ holds for all $k \leq n$, where n is fixed but arbitrary.
3. **Induction step:** Given induction hypothesis, prove that $P(n+1)$ holds.

2.10 EXAM questions

2.10.1 Do an inductive proof over numbers

Walkthrough:

1. **Base Case:**
   - Prove the statement is true for the initial value, usually for $n = 0$ or $n = 1$.
   - This establishes a starting point for the induction.
2. **Inductive Hypothesis:**
   - Assume the statement is true for some arbitrary natural number $n = k$, where k is a typical number in the domain of the statement.
   - This is not proving the statement; it's just assuming it's true for the purpose of the next step.
3. **Inductive Step:**
   - Prove that if the statement is true for $n = k$ (the inductive hypothesis), then it is also true for $n = k + 1$.
   - This step often involves algebraic manipulation and the use of the inductive hypothesis.
4. **Conclusion:**
   Conclude that since the statement is true for the base case and the truth of the statement for $n = k$ implies its truth for $n = k + 1$, the statement is true for all natural numbers.

Example:
Statement: For all natural numbers n, the sum of the first n natural numbers is $\frac{n(n+1)}{2}$

1. **Base case:** Prove the statement for $n = 1$
   - The sum of the first 1 natural number is 1.
   - According to the formula, $\frac{1(1+1)}{2} = 1$
   - The statement holds the base case.
2. **Inductive Hypothesis:** Assume the statement is true for some natural number k. That is, assume the sum of the first k natural numbers is $\frac{k(k+1)}{2}$
3. **Inductive Step:** Prove the statement for $k + 1$
   - The sum of the first $k + 1$ natural numbers is the sum of the first k natural numbers plus $k + 1$. By the inductive hypothesis, this sum is $\frac{k(k+1)}{2} + (k + 1)$
   - Simplifying, $\frac{k(k+1)}{2} + \frac{2(k+1)}{2} = \frac{k^2+3k+2}{2} = \frac{(k+1)(k+2)}{2}$
   - This is the same as $\frac{(k+1)((k+1)+1)}{2}$, which matches the formula, because $k + 1$ replaces k.
4. **Conclusion:**
   By mathematical induction, the statement is true for all natural numbers n.

2.10.2 Do an inductive proof over strings.

Inductive proofs for strings can be similar but often involve proving properties about strings of increasing length or complexity:

1. **Base Case:**
   Prove the statement for the simplest string, usually the empty string or a string with a single character.
2. **Inductive Hypothesis:**
   Assume the statement is true for strings of length k (or strings up to length k)
3. **Inductive Step:**
   - Prove that if the statement is true for strings of length k, it is also true for strings of length $k + 1$ or the next level of string complexity.
   - This step often involves analyzing the structure of the strings and how properties change (or remain true) when a character is added or altered.
4. **Conclusion:**
   Conclude that since the statement is true for the simplest string and true for a string of length k implies its truth for a string of length $k + 1$, the statement is true for all strings of that nature.

Example:
Statement: For any string composed of the characters 'a' and 'b', the number of 'a's is equal to or greater than the number of 'b's if and only if the string starts with 'a'.

1. **Base Case:**
   Prove the statement for the simplest string, the empty string, and a single-character string:
   - The empty string satisfies the condition trivially (0 'a's and 0 'b's).
   - A single-character string starting with 'a' also satisfies the condition.
2. **Inductive Hypothesis:**
   Assume the statement is true for strings of length k
3. **Inductive Step:**
   Prove the statement for strings of length $k + 1$
   - Consider a string of length $k + 1$

- If it starts with 'a', then regardless of the rest of the string, the condition holds (since it already holds for the string og length k)
- If it starts with 'b', then for the condition to hold, the following string of length k must start with 'a' (by inductive hypothesis)

4. **Conclusion:**
   By mathematical induction, the statement is true for all strings composed of 'a' and 'b'.

2.10.3 Prove by recursion P(P(S)) = P(S)

A projection function P in the context of strings, is used to transform a string by selectively keeping or discarding certain characters based on a specific criteria.

When projection functions are applied to strings, they typically involve creating a new string based on the original by including or excluding specific characters. The nature of the projection depends on the rule or set of rules that defines which characters are to be retained in the output string.

Projection Function P onto a Subset of Characters A'
When we talk about a projection function P onto a subset of characters A', we mean the following:

- **Subset of Characters A':** This is a specified group or set of characters.
- **Projection Function P:** This function takes a string as input and produces a new string. The function operates by examining each character in the input string and deciding whether to include it in the output string based on whether it belongs to the subset A'.
  - If a character in the input string is part of the subset A', it is included in the output string.
  - If a character is not part of A', it is excluded from the output string.

All rightey then.
We define projection P onto A' by $P(\varepsilon) = \varepsilon$ and $P(s \circ l) =$ if l $\in$ A' then $P(s) \circ l$ else P(s) for all strings s and letters l. Prove by recursion that P(P(s))=P(s)

The Projection Function P
The function P is defined as follows:

Base Case: For the empty string ε, $P(\varepsilon) = \varepsilon$.

Recursive Step: For any string s and letter l, $P(s \circ l)$ is defined as:

- If $l \in A'$, then $P(s \circ l) = P(s) \circ l$.
- If $l \notin A'$, then $P(s \circ l) = P(s)$.

Proving $P(P(s)) = P(s)$ by Induction

Base Case: Prove the statement for the empty string ε.

$P(P(\varepsilon)) = P(\varepsilon) = \varepsilon$ by the definition of P.
So, the statement holds for the base case.

Inductive Hypothesis: Assume the statement is true for a string of arbitrary length k, i.e., $P(P(s)) = P(s)$ for all strings s of length k.

Inductive Step: Prove the statement for a string of length $k+1$, i.e., show that $P(P(s \circ l)) = P(s \circ l)$ where s is a string of length k and l is a letter.

- Case 1: If $l \in A'$, then $P(s \circ l) = P(s) \circ l$ by the definition of P. By the inductive hypothesis, $P(P(s)) = P(s)$, so $P(P(s \circ l)) = P(P(s) \circ l) = P(s) \circ l = P(s \circ l)$.
- Case 2: If $l \notin A'$, then $P(s \circ l) = P(s)$. Again, by the inductive hypothesis, $P(P(s)) = P(s)$, so $P(P(s \circ l)) = P(P(s)) = P(s) = P(s \circ l)$.

Conclusion: By mathematical induction, $P(P(s)) = P(s)$ for all strings s.

0ther snacks:

- $\varepsilon + s = s$ for all strings s
- $s + (t + u) = (s + t) + u$, for all strings s, t, u
- $F(s + t) = F(s) + F(t)$
- $P(s + t) = P(s) + P(t)$
- $P(F(s)) = F(P(s))????$
- $P(P(s)) = P(s)$

2.10.4 Find a recursive definition of some function

Walkthrough:

1. **Identify the Base Case:**.
   - The base case(s) are the simplest instance(s) of the problem that can be solved directly without recursion.
   - This typically involves small, fixed input values for which the function's output is known.
2. **Recursive Step:**
   - Determine how a larger problem can be expressed in terms of one or more smaller problems.
   - This step involves calling the function itself with simpler inputs.
3. **Ensure Progress Towards the Base Case:**
   - The recursive calls must progress towards the base case(s)
   - This prevents infinite recursion.
4. **Combine the Results:**
   If the function involves combining results from recursive calls, define how these results are combined.

Test:
Let's find a recursive definition for a function $f(n)$ that calculates the factorial of a number n, where

$$n! = n * (n - 1) * (n - 2) * ... * 1$$

1. **Identify the Base Case:**
   - The simplest instance of a factorial is 0!, which is defined to be 1.
   - So, the base case is f(0) = 1.
2. **Recursive step:**
   - The factorial of n can be expressed as n times the factorial of $n - 1$
   - So the recursive step is $f(n) = n * f(n - 1)$
3. **Ensure Progress Towards the Base Case:**
   Each call to the function decreases n by 1, progressing towards the base case $f(0)$

Therefore, the recursive definition for $f(n)$, the factorial function, is:

- Base Case: $f(0) = 1$
- Recursive Step: $f(n) = n * f(n - 1)$ for $n > 0$

Sum of first n even numbers:
This is easy.

- First even number is 2. So f(1) = 2. This is the base case.
- Now, we can infer that the output of a function is double the n. Therefore we can say that $f(n) = 2n + f(n-1)$

Sum of squares for the first n natural numbers:

- Base Case: G(1) = 1, because 1^2 is 1.
- Recursive definition: G(n) = n^2 + G(n-1)

2.10.5 Transform numbers between bases

2.10.6 Calculate gcd and lcm.

2.10.7 Find prime representations of numbers.

2.11 Quiz

2.11.1 1

Which of the following sets have a smallest number?

- The set of all powers of 2 that are also powers of 3.
- The set of all powers of 2 that are also powers of 4.✓
- The set of all integer multiples of 17.
- The set of all solutions of the equation $\cos(x) = 0$.
- The set of all composite natural numbers. ✓
- The set of all natural numbers n such that $1/n$ is smaller than 0.0001. ✓
- The set of all Norwegian person numbers. ✓

Reasoning:

- **The set of all powers of 2 that are also powers of 3.**
  This set contains numbers that are both powers of 2 and powers of 3. The only number that satisfies this is 1 (since $2^0 = 1$ and $3^0 = 1$. However, since 1 is not typically considered a power of 2 or 3 in this context, some may argue this set is actually empty.
- **The set of all powers of 2 that are also powers of 4.**
  Every power of 4 is inherently a power of 2 (since 4 is 2^2). The smallest power of 4 is $4^0 = 1$, which is also a part of the set. Thus, this set has a smallest number 1.
- **The set of all integer multiples of 17.**
  This set includes all numbers that can be written as $17n$ where n is an integer. It extends infinitely in both positive and negative directions (including 0), and therefore has no smallest number.
- **The set of all solutions of the equation** $\cos(x) = 0$**.**
  The solutions to this equation is infinite and has no smallest element, as k can be any integer, negative or positive.
- **The set of all composite natural numbers.**
  Composite numbers are natural numbers greater than 1 that are not prime. The smallest number is 4.
- **The set of all natural numbers** n **such that** $1/n$ **is smaller than** 0.0001**.**
  This set contains natural numbers n for which $n > 1000$ (since $1/10000 = 0.0001$). The smallest number in this set is 10001.

- **The set of all Norwegian person numbers.**:
  Norwegian person numbers are a finite set of unique identification numbers assigned to individuals. Since there is a finite, countable number of such numbers, this set has a smallest number (the numerically smallest valid person number).

2.11.2 2

Is 8|418? Write 1 for yes and 0 for no
0
Reasoning:
The question asks if 8 divides 418. This means that we need to check if 418 is a multiple of 8, which it is not.

2.11.3 3

What is the GCD of 2364 and 4632?
12
Reasoning:
Kalkulator eller extended Euclidean Algorithm

2.11.4 4

What is the lcm of 330 and 225?
4950
Reasoning:
Kalkulator eller utregning

2.11.5 5

Match the following statements to elements of a recursive definition.

Statement	Type
$1! = 1$	Base case statement
$0! = 1$	Base case statement
$(n+1)! = n! \cdot (n+1)$	Recursive statement
$(n+2)! = n! \cdot (n+1) \cdot (n+2)$	Recursive statement
$(n+2)! = (n+1)! \cdot (n+2)$	Recursive statement
$0! = 0$	Wrong statement

2.11.6 6

Match the following proof steps to the steps of an inductive proof.

Proof Step	Inductive Proof Step
$0 + 0 = 0$	Basis step
$0 + s(0) = s(0 + 0) = s(0) = s(0)$	Basis step
$0 + k = k$	Induction hypothesis
$0 + (k + 1) = (0 + k) + 1 = k + 1$	Inductive step
$0 + s(k) = s(0 + k) = s(k)$	Inductive step

2.11.7 7

Translate the decimal number 2293 into base 7:
6454
Reasoning:
See guide on transforming bases.

2.11.8 8

Match the following numbers to their prime representation

Number	Factorization
4420	$2 \cdot 2 \cdot 5 \cdot 13 \cdot 17$
1218	$2 \cdot 3 \cdot 7 \cdot 29$
3034	$2 \cdot 37 \cdot 41$
2210	Something else

Reasoning:
You know this, you had this in school

2.11.9 9

What are the elements of a recursive definition?

- A default case statement
- One or more recursive statements ✓
- A uniqueness statement
- A fallback statement
- One or more base case statements ✓

2.11.10 10

What is true for inductive proofs?

- The inductive step uses the basis step.
- Each inductive proof has a basis step. ✓
- The inductive step uses the induction hypothesis.✓
- It is not possible to have two basis steps in the same inductive proof.
- Each inductive proof has an induction hypothesis.✓
- It is not possible to have two inductive steps.
- Each inductive proof has an inductive step.✓

3 Grammars

3.1 Introduction to Languages and Alphabets

3.1.1 Basic Concepts

- **Alphabet:** An alphabet in formal language theory is an arbitrary set of symbols or characters, typically denoted as A. The alphabet is a set of symbols or characters, and are the basic units used to construct strings or words.
- **Strings:** A string is a finite sequence of symbols from the alphabet. The set of all possible strings that can be formed from the alphabet **A** is denoted as **A***. This includes every conceivable combination of symbols from **A**, as well as the empty string.
- **Empty String:** Represented as ϵ, the empty string is a string with no symbols. It is part of **A*** for any alphabet A.
- **String formation:** If s is a string ($s \in A*$ and l is a symbol in A $l \in A$, then appending l to s (denoted as $s \circ l$ results in another string which is also in $A*$.

3.1.2 Language definition

- **Language:** A language L is defined as any subset of $A*$ ($L \subseteq A*$. It means a language consists of a selection (possibly all, possibly none) of strings from the set of all strings over the alphabet A.
- **Language dependence on Alphabet:** The nature of the language L is closely tied to the alphabet A, as L is a subset of $A*$. This implies that what constitutes a valid string in L depends on the symbols defined in A.
- **Decision for String Membership:** For each specific string, it can be decided whether or not it belongs to the language L.

3.1.3 Language Operations

- **Empty Language:** Set with no strings
- **Language with Only Empty Word:** The set containing only ϵ.
- **Union:** The set of all words in either of two languages. $L_1 \cup L_2$
- **Concatenation:** The set of all combinations of strings from two languages. L_1L_2 all words $l_1 + l_2$ where $l_1 \in L_1$ and $l_2 \in L_2$
- **Iterated Concatenation and Kleene Closure:** Generating sets of string by repeating concatenation. **Iterated concatenation**:
  - $L^0 = \epsilon = \epsilon$
  - $L^1 = L$
  - $L^{n+1} = L^n + L$

  Kleene Closure: $L^* : U_{i=0}^{\infty} L^i$ - zero or more concatenations of L.
- **Positive Closure:** Similar to Kleene closure but excludes the empty string. $L^* : U_{i=1}^{\infty} L^i$ - one or more concatenations of L

3.1.4 Equivalences for Language Operations

Term	Mathematical Notation	Description
Commutative	$L1 \cup L2 = L2 \cup L1$	The union of two languages is independent of the order in which they are combined.
Associative	$(L1 \cup L2) \cup L3 = L1 \cup (L2 \cup L3)$ $(L1L2)L3 = L1(L2L3)$	The union of multiple languages can be grouped in any order without affecting the result. Concatenation of multiple languages is also associative.
Distributive	$L1(L2 \cup L3) = (L1L2) \cup (L1L3)$	Concatenation distributes over union.
Identity	$L\epsilon = \epsilon L = L$ $L \cup \emptyset = \emptyset \cup L = L$	The language containing only the empty string acts as an identity for concatenation. The empty language acts as an identity for union.
Idempotence	$r^{**} = r^*$	Applying the Kleene star operation multiple times is equivalent to applying it once.
Kleene and Positive Closure	$r^* = (r \cup \epsilon)^* = (r \cup \epsilon)^+$	The Kleene closure of a language equals the closure of the union of the language and the empty string; this is also equal to the positive closure of the union.

3.1.5 Language Definition

Languages can be defined in two ways:

1. **Generation:** Defining a language by specifying how its strings can be generated.
2. **Recognition:** Defining a language by specifying a method to determine if a string is a member of the language.

3.2 Grammar

A grammar is a structure defined as (T, NT, s, R) where:

- T is a set of terminal symbols (actual symbols used in strings of the language)
- NT is a set of nonterminal symbols (used for constructing grammar rules, not appearing in the final language strings).
- s is the start symbol, a special nonterminal that begins the derivation process.
- R is a set of production rules, where each rule is a pair (l, r), and both l and r are strings that consist of both terminal and nonterminal symbols.

3.2.1 Example grammar

- **Example 1:** $(a, b, s, t, s, (s, at), (t, b))$
  The rules are $s \to at$ and $t \to b$.
- **Example 2:** $(a, s, s, (s, asas), (sas, a))$
  The rules are $s \to asas$ and $sas \to a$

3.2.2 Grammar and Languages

- Grammars define languages through a generation process.
- Generation involves starting from the start symbol s and applying derivation steps to replace nonterminal elements using the production rules.
- **The language of a grammar consists of all strings that can be derived using this process, containing only terminal symbols.**

3.2.3 Derivation (Generation) Process:

Derivation in a grammar is a finite sequence of strings where:

- The first string is the start symbol s.
- Each subsequent string is formed by replacing a nonterminal in the previous string with its corresponding rule in R.
- The final string consists only of terminal symbols.

3.2.4 Examples of Derivation

- For grammar $(a, b, s, t, s, (s, at), (t, b))$, the derivation $s \to at \to ab$ results in the language ab.
- For grammar $(a, s, s, (s, asas), (sas, a))$, multiple derivations are possible, resulting in the language $a*$ (the Kleene closure of a.

3.2.5 Well-Defined Grammars:

- A grammar must be well-defined, meaning its rules should allow for the generation of strings consisting only of terminal symbols.
- Ill-defined grammars may result in cycles or use undefined symbols in the rules, leading to ambiguous or empty languages.

3.2.6 Level of Grammar

Chomsky Hierarchy of grammars, presenting grammars categorized into different types or levels, each representing a certain power or complexity in generating languages. The goal is often to find a grammar for a language at the highest possible level in the hierarchy, balancing complexity and expressive power.

3.2.7 Simplification in Grammar Notation

- **Use of Alternatives:** $s \to A|B$ meaning s can be replaced by either A or B.
- **Kleene Closure:** $s \to A*$ implies that s acn be replaced by any number of A's, including none.
- **Positive Closure:** $s \to A+$ indicates s can be replaced by one or more A's.
- **Optional Parts:** $s \to [A]$ means s can be replaced by A or nothing.

3.2.8 How to Write Grammars

Extended Backus-Naur Form (EBNF)

- EBNF is a notation for writing grammar, where each rule is in the form $< symbol > \to < expression >$
- It uses | for alternatives, **[...]** for optional parts, **{...}** forzero or more repetitions, and **{...}+** for one or more repetitions.
- Nonterminals are symbols on the left-hand side of rules The first nonterminal is usually the start symbol. Other symbols are terminals.

3.2.9 What is the Alphabet in Grammrs:

- **Low-Level Alphabet:** Consists of basic characters, like ASCII characters.
- **High-Level Alphabet:** Comprises tokens or meaningful units, such as keywords, names, and symbols in programming languages.

3.2.10 EBNF to Tuple Grammar

Translation of EBNF Abbreviation Rules:

- **Alternatives:** In EBNF, alternatives (expressed as **A | B** translate into multiple rules in tuple grammar.
- **Repetitions:** EBNF repetitions (like **{A}** become new nonterminals in tuple grammar with a start rule and a recursive rule
- **Optional Parts:** The optional elements in EBNF (denoted by **[A]** lead to two rules in tuple grammar: one for the empty case and one for the case where the element is present.
- **Grouping:** Grouped elements in EBNF often become a new nonterminal with included rules in tuple grammar.

3.2.11 Grammar for a Language

Defining Languages with Grammars

- **Language Definition:** All grammars define a language. However, not all languages can be defined by a grammar.
- **Finding a Grammar:** To find a grammar for a language, identify the language pattern, refer to similar patterns in known languages, and adapt their grammar definitions. Utilize multiple nonterminals for clarity and simplicity.

3.2.12 Example of Finding a Grammar

Language Pattern $L = (ab)^n a$

- **Grammar Adaptation:** Adapt the concept to the pattern **L** by creating rules like $s \to aba$
- **Detailed Rules:** Create specific grammar rules, such as $s \to aRep$, $Rep \to Repba$ and $Rep \to \epsilon$

3.2.13 Language from a Grammar

Deriving the Language:

- **Derivation Process:** Generate several derivations from the grammar and collect the resulting terminal strings.
- **Generalization:** Abstract these results into a generalized language description.
- **Validation:** Ensure that the generalized language description fits the grammar.

3.2.14 Sample Language from Grammar

Example Grammar $s \to tuv$:

- **Derivations:** Derive strings like **ac**, **abcc**, **aabcc**, **aaabbccc** from the grammar.
- **Hypothesis:** Formulate a hypothesis for the language, such as $L = a^{n+1}b^m c^{m+1}$

3.2.15 Reasoning for the Hypothesis

Analyzing the Grammar Components:

1. **Component t:** Produces an arbitrary non-empty sequence of **a**'s
2. **Component u:** Ensures an equal number of **b**'s and **c**'s.
3. **Component v:** Translates directly into **c**, matching the number of **b**'s in **u**.
4. **Overall Structure:** The structure of **s** introduces an extra **c** leading to the language $a^{n+1} + b^m c^{m+1}$

En grammar er en struktur (T, NT, s, R), hvor

- **T** er et vilkårlig sett med **terminal** symboler. Terminal symboler er de grunnleggende symbolene som brukes for å forme stringen. De er de faktiske symboler i språket.
- **NT** er et vlkårlig sett med **non-terminal** symboler, som er *disjoint* med T. Det betyr at de ikke har noen felles elementer. Nonterminals brukes i regler for å representere mønstre eller kombinasjoner av terminals og nonterminals. De er ikke en del av selve språket, men er uunnværelige i prosessen med å generere strenger.
- **s** er **startsymbolet**, hvor $s \in NT$
- **R** er et sett med **rules**, hvor hver regel er et par (l,r), og $l, r \in (T \cup NT)*$.
  - l er left-hand, r er right-hand.
  - **l** kan inneholde en sekvens med **terminals** og **non-terminals**. Ofte en enkelt **non-terminal**.
  - **r (production rule)** er en regel som sier noe om hvordan et symbol (vanligvis en non-terminal) kan ersattes av en sekvens av symboler (som kan bestå av terminaler, ikke-terminaler, eller begge). Kan også være en tom sekvens.
  - $T \cup NT$ er en union mellom settene T og NT. Dette betyr at settet inneholder alle elementer som enter er i T, NT eller i begge.
  - Når **Kleene Star (*)** legges til settet, kan man lage strenger ved å slå sammen 0 eller flere elementer fra settet.
    For eksempel, hvis vi har settet A = {a,b}, så representerer A^* alle strengene som kan lages av 'a' og 'b', inkludert en emptry string() Dette gjelder både for l og r. Altså, både l og r kan være enten en tom string, eller en hvilken som helst sekvens av terminaler og ikke-terminaler, i hvilken som helst rekkefølge og antall.

Eksempel:
({a,b},{s,t},s,{(s,at),(t,b)}

- T ={a,b}
- NT ={s,t}
- s = s
- R = {(s,at),(t,b)}
- Dette blir til reglene s → at og t → b

Derivasjon:
En derivasjon av en setning i grammatikk, er en endelig sekvens av strenger bestående av terminaler og ikke-terminaler, hvor den første strengen kun består av startsymbolet, og hver påfølgende streng er resultatet av å bruke en regel for å erstatte l fra den forrige strengen med r. Den siste strengen består bare av terminaler.

Grammatikken vi har gjennomgått så langt er **Context-Free Grammar(CFG)**. Dette betyr at hver produksjonsregel har ett enkelt ikke-terminal-symbol på venstre side, og en string med terminaler og/eller på høyre side.

Derivasjon representert som et tre (Parse tree) i CFG Vi har en enkel grammatikk som følger formen ({a,b}, {S, A}, s, {(S,aA), (A,aA), (A,b)}. Reglene er da $S \to aA$, $A \to aA$, og $A \to b$. La oss konstruere setningen "aaab" ved hjelp av grammatikken.

1. **Start-symbol:** Vi begynner med S.
2. **S → aA:** Erstatt S med aA. Derivasjonstreet deler seg i a og A.
3. **A → aA:** Erstatt A med aA. Derivasjonstreet deler seg på samme måte.
4. **A → aA:**
5. **A → b:** Erstatt A med b. Derivasjonstreet er nå avsluttet og vi står igjen med Figur 3.

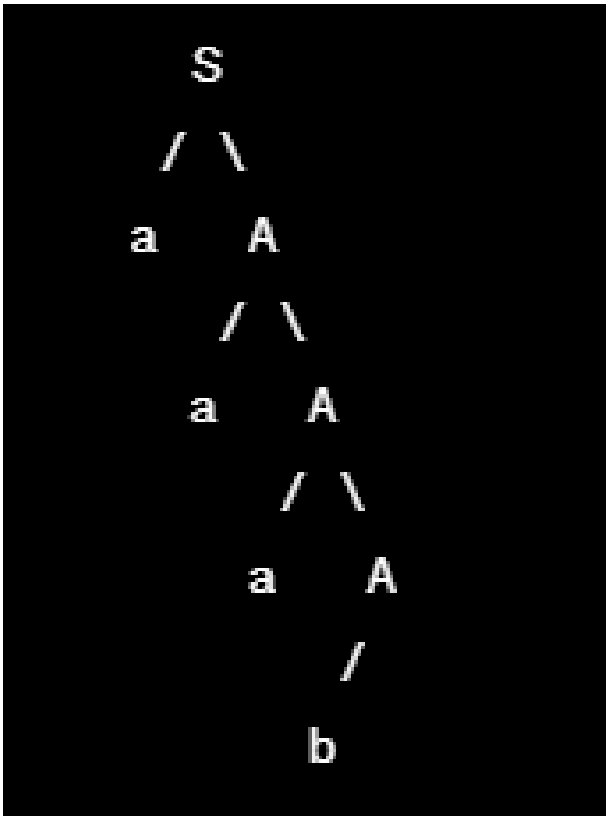

Figure 3: Tre

Så:

- Root er **start** symbolet-
- Vertex er **non-terminals**
- Leaves er **terminal** eller ϵ

Venstre og høyre derivasjonstrær:

- **Left derivation tree** blir laget ved å anvende produksjonsregel på variabelen lengst til venstre i hvert steg.
- **Right derivation tree** bruker variabelen lengst til høyre i hvert steg.

Vi vil generere stringen "aabaa" fra grammatikken $S \to aAS|aSS|\epsilon, A \to SbA|ba$

Figure 4: Left Tree

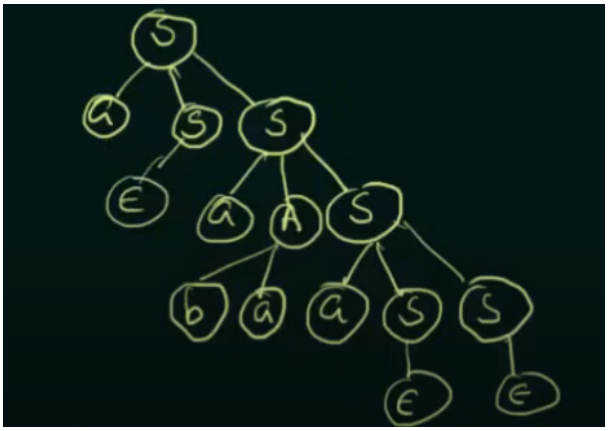

Figure 5: Right Tree

Ambiguity in grammars:
En **grammar** kalles **ambiguous** hvis det finnes to eller flere derivasjonstrær for en string **s** (to eller flere *left derivation trees*). La oss ta et eksempel: G = (T = {a+b, +, *}, NT={S}, S = S, R = {(S,S+S), (S, S*S), (S,a), (S,b)}). Generer strengen "a + a * b". Hvor mange ulike måter kan vi gjøre dette?
1.

- $S \to \underline{S} + S$
- $\to a + \underline{S}$
- $\to a + \underline{S} * S$
- $\to a + a * \underline{S}$
- $\to a + a * b$

2.

- $S \to \underline{S} * S$
- $\to \underline{S} + S * S$
- $\to a + \underline{S} * S$
- $\to a + a * \underline{S}$
- $\to a + a * b$

Fordi det er to muligheter, er grammatikken ambiguous.

3.3 Translate between EBNF and 4-tuple grammars

3.3.1 Extended Backus-Naur form (EBNF)

Nøkkelfunksjonalitet:

- **Optional elements:** Ved å bruke brackets '[ ]' kan man sette et syntax element som valgfritt. Altså, [element].
- **Repetisjon:** Ved å bruke curly braces '{}' kan man indikere repetisjon (0 eller flere ganger). For eksempel {element} betyr at elementet kan gjentas flere ganger, i tillegg til ikke i det hele tatt.
- **Grouping:** Paranteser '( )' brukes for å gruppere elementer. Dette er nyttig for å tydeliggjøre gramatikken, og sette repetisjon eller optionality til en gruppe av elementer.
- **Alternation:** Pipe symbolet '|' brukes for alternering (logical OR). Dette gir alternativer for produksjon. For eksempel 'element1 | element2' betyr enten element1 eller element2.
- **Special Sequences and Literals:** EBNF inkluderer ofte spesifikk notasjon for spesielle sekvenser og *string literals*, som tas som de er. For eksempel, "begin" representerer stringen "begin".
- **Comments:** EBNF støtter kommentarer, som gjør det mulig å annotere i grammatikken for tydelighet.

Eksempel:

```
    program ::= "program" identifier ";"
            { declaration ";" }
            "begin"
            { statement ";" }
            "end"
identifier ::= letter { letter | digit }
declaration ::= "var" identifier [ ":" type ]
statement ::= identifier ":=" expression | "print" "(" identifier ")"
type ::= "int" | "float"
letter ::= "a" | "b" | "c" | ... | "z"
digit ::= "0" | "1" | "2" | ... | "9"
```

Forklaring:

1. 'program' består av et nøkkelord "program", etterfulgt av en 'identifier', og flere 'deklarasjoner' og 'statements' og mellom "begin og "end".
2. 'identifier' er en kombinasjon av bokstaver og tall, som starter med en bokstav.
3. 'declaration' kan være en variabel declaration, med en 'type specifier' som option.
4. 'statement' kan være en assignment eller en printoperasjon
5. types er enten "int" eller "float"
6. 'letter' og 'digit' er definert med alternativer.

3.3.2 Konverter til 4-tuple-grammar

Denne prosessen er cirka like morsom den som virker:

1. **Identifiser non-terminals**: N={program, identifier, declaration, statement, type, letter, digit, expression}
2. **Identifiser terminals**: T = {"program", ";", "begin", "end", "var", ":", "int", "float", ":=", "print", "(", ")",} og alle de individuelle bokstavene og tallene.
3. **Definer regler**:
    - Håndter repetisjoner '{ }' og optional elements '[ ]' med nye nonterminaler og regler.
      { declarations ";" } blir 'declarations ::= declaration ";" declarations | ϵ'.
      Håndter optionals [":" type] som
      optional_type ::= ":" type | ϵ

4. **Start Symbol(S)**: S = program

Etter konverteringen kan vi da ha

- program → "program" identifier ";" declarations "begin" statements "end"
- declarations → declaration ";" declarations | ϵ
- declaration → "var" identifier optional_type
- optional_type → ":" type | ϵ
- statements → statement ";" statements | ϵ
- statement → identifier ":=" expression | "print" "(" identifier ")"

3.4 Find a grammar for a language and vice versa

3.5 Find a derivation for a string

3.6 Translate between syntax trees and derivations

3.7 Check ambiguity of a grammar

3.8 Quiz

3.8.1 1

What happens when we extend the alphabet of a grammar?
Let G=(A,NT,s,R) be a grammar over alphabet A. What happens when we extend the alphabet to A' = A u a and have G' = (A',NT,s,R)?
- The language of G is the same as the language of G'.
Reasoning:
If no new production rules are added for the new character, then it is the same.

3.8.2 2

What happens if the start symbol of a grammar is missing?
- The grammar is not well-defined.

3.8.3 3

A grammar contains a rule R ::= A | A . Which of the following statements are true?

- It depends on the use of R whether the grammar is ambiguous.
- If A is not empty, then R is ambiguous.

3.8.4 4

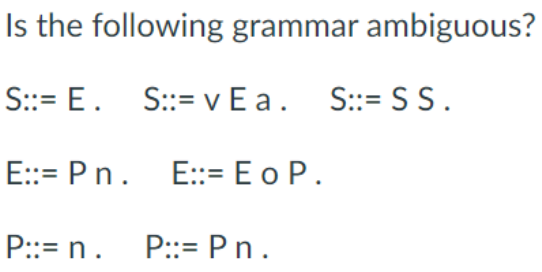

Figure 6: Question 4

- Yes, S is ambiguous, but not E and P.
Reasoning
We have two different derivations

1. Start with S
2. Replace S with vEa
3. Replace E with EoP
4. Replace the first E with Pn
5. Replace P with n

1. Start with S
2. Replace S with SS
3. Replace the first S with vEa
4. Replace E in the first S with Pn, P with n
5. Replace the second S with E
6. Replace E with Pn, P with n

When we say that a particular non-terminal symbol in a grammar, like S in your case, is ambiguous, we are referring to the fact that there are multiple distinct derivations or parse trees leading to some strings starting from that non-terminal symbol.

3.8.5 5

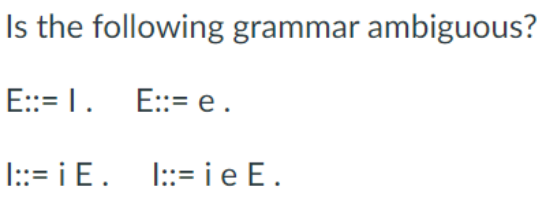

Figure 7: Enter Caption

- No, both E and I are unambiguous. **Reasoning:**
Same as the last paragraph from above.

3.8.6 6

In a grammar G, the string s has three different derivations. Is G ambiguous?
- Yes, if the different derivations lead to at least two different syntax trees.

3.8.7 7

Let L1 and L2 be two languages with L1 =< L2. What is the relation between L1+ and L2*? (=< denotes the subset relation)
- $L1+ =< L2*$
Reasoning:

- $L1+$ is the language containing all non-empty strings of concatenations of strings from L_1. Formally $L_1^+ = L_1L_1L_1...$ (one or more times).
- $L2*$ is the Kleene star of L_2, which contains all strings of concatenations of strings from L_2, including the empty string ϵ. Formally, $L_2^* = \{\epsilon\} \cup L_2L_2L_2$ (zero or more times)

Given the definition of $L_1 \subseteq L_2$, it is clear that any string in L_1^+ is also in L_2^* because:

- Every string in L_1^+ is made up of one or more strings from L_1.
- Since $L_1 \subseteq L_2$, every string in L_1 is also in L_2.
- Therefore, any concatenation of strings from L_1 is also a concatenation of strings from L_2.
- Additionally, L_2^* includes the empty string, while L_1^+ does not.

As a result, we can conclude that $L_1^+ \subseteq L_2^*$. However, the opposite is not necessarily true.

3.8.8 8

Let L1 and L2 be two languages with L1 =< L2. What is the relation between L1* and L2+? (=< denotes the subset relation)

- None of the alternatives ✓
- L1* $\subseteq$ L2+
- L1* $\subset$ L2+
- L1* = L2+

3.8.9 9

What is ϵ?

- Epsilon is a special symbol. It denotes that there is nothing.
- Epsilon is an empty string (of terminals and non-terminals).

3.8.10 10

Do you need to know the grammar in order to translate a derivation into a syntax tree?
- No, the derivation shows the rules used.

4 Languages, Machines, Regular expressions

4.1 Regular expressions

Et språk som følger de følgende delene er et **regular language:**
Grunnleggende elementer

1. **Ø:** denotes the empty language
2. ε**:** (the empty string) denotes the language with only one empty string: $\{\varepsilon\}$
3. **a** (if $a \in A$) denotes the language with only the string a: {a}

Operators

- **(r)|(s):** alternative denotes L(r) ∪ L(s)
- **(r)(s)**: Concatenation denotes L(r)L(s)
- **(r)*:** closure denotes L(r)*

Table 3: Regex Cheat Sheet

Pattern	Description
`^`	Start of a line
`$`	End of a line
`.`	Any single character
`[...]`	Any single character in the brackets
`[^...]`	Any single character not in the brackets
`|`	Alternation (logical OR)
`*`	0 or more of the preceding element
`+`	1 or more of the preceding element
`?`	0 or 1 of the preceding element
`{n}`	Exactly n occurrences
`{n,}`	At least n occurrences
`{n,m}`	Between n and m occurrences
`\\`	Escape character
`\d`	Any digit
`\D`	Any non-digit
`\w`	Any word character (letter, digit, underscore)
`\W`	Any non-word character
`\s`	Any whitespace character (space, tab, newline)
`\S`	Any non-whitespace character
`(...)`	Capturing group
`(?:...)`	Non-capturing group
`[a-z]`	Range (all lowercase letters)
`[A-Z]`	Range (all uppercase letters)
`[0-9]`	Range (all digits)
`\b`	Word boundary
`\B`	Non-word boundary
`(?=...)`	Positive lookahead
`(?!...)`	Negative lookahead
`(?<=...)`	Positive lookbehind
`(?<!...)`	Negative lookbehind

(?!abc1).{4} = Negative lookahead. Det som følger kan ikke være abc1.
b matches the boundary between a word and a non-word character. It is most useful in capturing entire words (for example by using the pattern
$w+$
b

4.2 Finite state machine (Finite automata)

Finite Automata without output, DFA, NFA og ϵ-NFA.

4.3 Deterministic finite automata

Simplest model of computation.
Has a very limited memory.

- **States** (verteces)
- **Edges** Transitions between states.
- Initial/starting state has an arrow pointing from nowhere
- Double circle is the final/terminating state

Formal definition: (S, A , T, s, F)

- **S** = Set of all states
- **A** = Alphabet (input)
- **T** = Transition function S x A -> S
- **s** = Set of start state $s \in S$
- **F** = Set of final (accepting) states $F \subseteq$ S

Similar to NFA, but transition function is deterministic. This means that for each state and each input symbol, there is exactly one transition defined. This determinism ensures that, given an input string and a starting state, there is only one possible path the automaton can follow through its states. DFA also needs to have transitions based on all inputs, enter dead states.

4.3.1 Minimize DFA

MINIMIZING CAN ALSO HAPPEN FOR THE FINAL STATES. CHECK EXAMPLES IN EXERCISES FOR CHAP 4. Minimizing means converting a given DFA to its equivalent DFA with minimum number of states.

1. Remove unreachable states
2. Divide the set of states into two sets: final states and non-final states. This partition is called P_0
3. Find P_{k+1} recursively by partitioning the sets of P_k. In each set of P_k, separate the states that are distinguishable, thus getting P_{k+1}.
4. Stop when $P_{k+1} = P_k$ (No change in partition)
5. The sets of the last P_k are the states of the minimal DFA and have their transitions.

Two states s_i and s_j are distinguishable in partition P_{k+1} if for any input symbol a, their transitions T(s_i, a) and T(s_j, a) are in different sets in partition P_k.

Simplified approach: Two states can be combined only when they are equivalent. This can happen in two ways:

1. If $T(A, X) \rightarrow F$ (transition from state A, given input string X, goes to final state F) AND $T(B, X) \rightarrow F$
2. If $T(A, X)! \rightarrow F$ AND $T(B, X)! \rightarrow F$

- If |X| (length of input string) = 0, then A and B are said to be 0 equivalent
- If |X| = 1, then A and B are said to be 1 equivalent
- If |X| = 2, then A and B are said to be 2 equivalent

- If |X| = n, then A and B are said to be n equivalent.

Example:

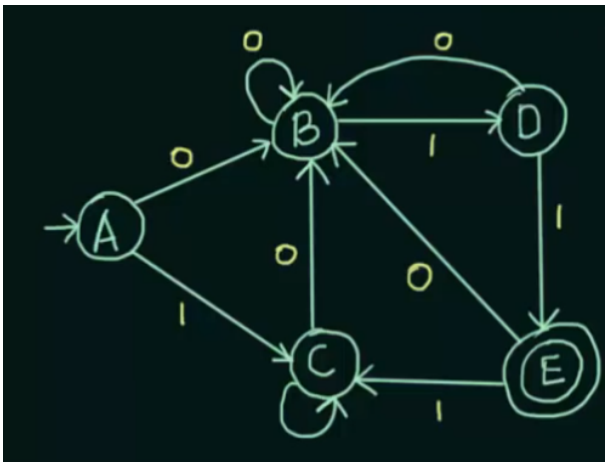

Figure 8: DFA to minimize

1. Create state transition table: The table shows the transitions from the different states based on input.

	0	1
→**A**	B	C
B	B	D
C	B	C
D	B	E
***E**	B	C

Table 4: State Transition Table

2. **0 equivalence**:
Separate the non-final states and the final states:
NFS = {A,B,C,D} FS = {E}

3. **1 equivalence**:
Check if NFS set contains 1 equivalent states. Check the transition table to see if the elements in NFS transition from states in the same set, and transition to states in the same set. Compare two and two elements.

1. (A,B): A goes to B and C, and B goes to B and D.
   B and B falls in the same set, and B and D falls in the same set. This means they are 1 equivalent to each other ✓
2. (A,C): A → (B,C), C → (B,C) ✓
3. (A,D): A → (B,C), D →. (B,E)
   B and B falls in the same set, but C and E does not. Therefore A and D are not 1 equivalent. D is therefore not 1 equivalent to B and C either. Take D out of NFS set.

{A,B,C}, {D}, {E}

3. **2 equivalence**:
Use the set from 1 equivalence. Only check A, B and C.

1. (A,B): A goes to B and C, and B goes to B and D.
   B and B falls in the same set, but B and D does not.
   A and B are not 2 equivalent
2. Now, we need to check C against both A and B.
3. (A, C): A goes to B and C, and C goes to B and C ✓
4. (B, C): B goes to B and D, and C goes to B and C.
   D and C are not in the same set, therefore
   B and C are not 2 equivalent

{A, C} {B} {D} {E}

3. **3 equivalence**:
Use the set from 2 equivalence. Only check A, C, as the rest are separate sets and cannot be combined with other sets.
As A → C → (B,C), we stop. This is because two rows of sets are similar (see below)
{A, C} {B} {D} {E}

A and C can be combined into one state. Based on the sets now remaining, and the transition table we can draw a new DFA:

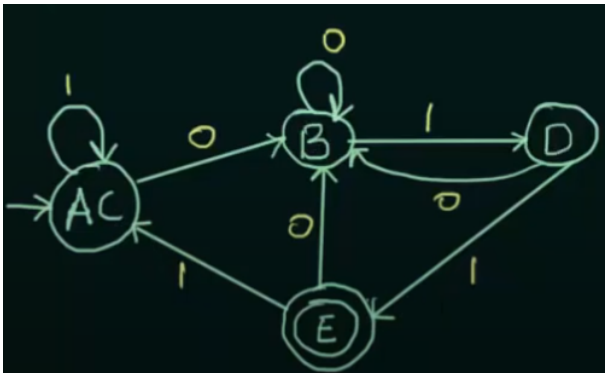

Figure 9: Minimized DFA

4.4 Non-deterministic Finite Automata

What separates NFA from DFA is the transition state, is the transitions.

- There can be two equal transitions/edges (labels) from one single state
- The empty string can be a label
- Transition function T : $Sx(A \cup \{\varepsilon\}) \to P(S)$
- We accept a string if there is a path to a final state

4.5 Translate from NFA to DFA:

NOTE: IF THERE ARE EPSILON TRANSITIONS, TRANSLATE FROM THESE FIRST, SEE BELOW

Formal definition: (S, A , T, s, F)

- **S** = Set of all states
- **A** = Alphabet (input)
- **T** = Transition function S x A -> S
- **s** = Set of start state $s \in S$
- **F** = Set of final (accepting) states $F \subseteq$ S

In this example, we will use $M = [A, B, C, (a, b), \delta, A, C]$
T is defined as:

	a	b
A	A,B	C
B	A	B
∘C	ϕ	A,B

This forms the following NFA state diagram:

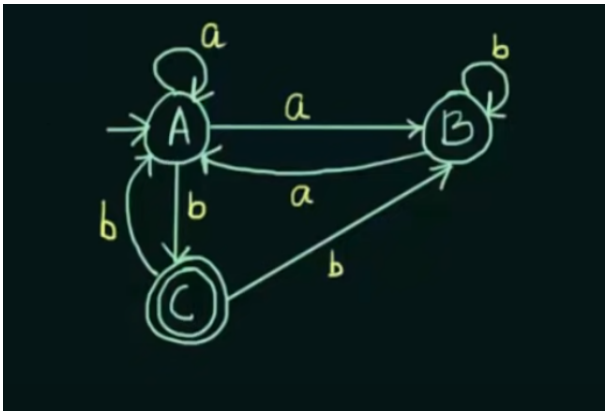

Figure 10: Initial NFA diagram

We start with mapping state A:
When **A** gets input **a**, it goes to **A,B**. This is not legal in DFA, as it can only go to one. We therefore combine the states into **AB**.
That leaves us with the following transition diagram after A has been mapped.

	a	b
A	AB	C

Next, we map AB, as this is one of the two reachable states (AB and C).
In order to know where AB goes, we have to look at the NFA transition table for both A and B, and perform the union operation on A and B.
As A goes to A,B for input 'a' and B goes to A for input 'a', we get $A, B \cup A = AB$.
A goes to C for input 'b' and B goes to C for input 'b', so we get $B \cup C = BC$.

	a	b
A	AB	C
AB	AB	BC

Next reachable state is BC.
On input 'a', $B \to A$ and $C \to \phi$, therefore just A.

On input 'b', $B \to B$ and $C \to A, B$, therefore AB.

	a	b
A	AB	C
AB	AB	BC
BC	A	AB

Now what states are reachable?
Just state C.
For input 'a', $C \to \phi$. In DFA this is not legal, so we introduce a DEAD STATE D.
For input 'b', $C \to AB$.

	a	b
A	AB	C
AB	AB	BC
BC	A	AB
C	D	AB

Is it finished? No, we are missing the newly introduced dead state.
Whatever comes to the dead state stays in the dead state.

	a	b
A	AB	C
AB	AB	BC
BC	A	AB
C	D	AB
D	D	D

Now, we know what the initial state is, but not the final state.
As C was the final state in the NFA, we transform the states that contain a C in the DFA to final states.
This means that both **BC** and **C** will be final states.

	a	b
A	AB	C
AB	AB	BC
∘BC	A	AB
∘C	D	AB
D	D	D

The state diagram in NFA looks like this:

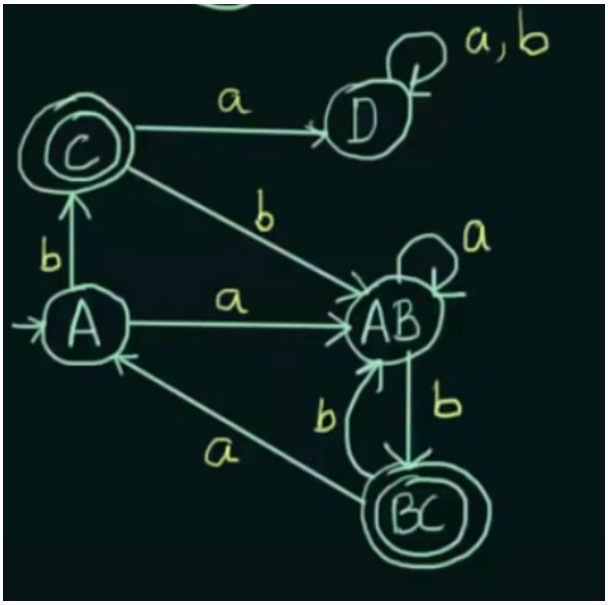

Figure 11: Final DFA

4.5.1 What if it is an epsilon NFA?

Convert epsilon NFA to NFA
We attempt it through this beast:

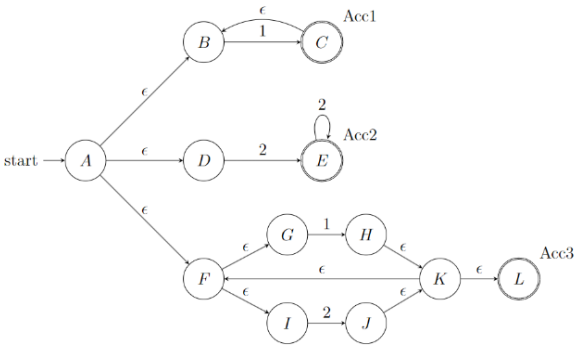

Figure 12: Beast NFA

THE FOLLOWING IS MORE OR LESS DIRECT TRANSCRIPT FROM SOLUTION. REWRITE.

1. We start by defining new states with lower case letters, where we indicate related the NFA states.

4.6 Translate from Regex to NFA

1. **Understanding the Regular Expression**:
   Before beginning, make sure you understand the regular expression you want to convert. For example:
   - **Literals:** representing themselves (e.g., 'a', 'b')
   - **Concatenation:** sequence of characters (e.g., 'ab')
   - **Alternation:** choice between characters (e.g., 'a|b')
   - **Kleene Star:** zero or more occurrences of the previous character (e.g., 'a*')

2. **Basic NFA Constructions**

   - For a Literal (e.g., 'a'): Create an NFA with a start state, an accept state, and a transition from the start to the accept state on the literal.
   - **For an epsilon (ϵ):** An NFA that transitions from its start state to its accept state on an epsilon move (a move without consuming any input).

3. **Applying Operators:**

   - **Concatenation (e.g., 'ab'):** Connect the accept state of the NFA for the first character to the start state of the NFA for the second character with an epsilon transition.
   - **Alternation (e.g., 'a|b'):** Create a new start state and a new accept state. Add epsilon transitions from the new start to the old starts of the 'a' and 'b' NFAs, and from the old accepts of these NFAs to the new accepts.
   - **Kleene Star (e.g., 'a*'):** Add new start and accept states. Add epsilon transitions from the new start to the old start, from the old accept to the new accept, from the old accept to the old start, and from the new start to the new accept.

4. **Build the NFA Step by Step**
   For a given regular expression, decompose it into its basic components and apply the above rules. Start with the innermost expressions and work outwards, combining NFAs using rules for concatenation, alternation, and the Kleene star.

5. **Example: Converting '(ab)*|c':**

   (a) **NFA for 'a' and 'b':** Construct basic NFAs for 'a' and 'b'.

   (b) **Concatenate for 'ab':** Connect 'a' NFA to 'b' NFA with an epsilon transition

   (c) **Kleene Star for '(ab)*':** Enclose 'ab' NFA in a new NFA with epsilon transitions for the star.

   (d) **NFA for 'c':** Construct a basic NFA for 'c'.

   (e) **Alternation for '(ab)*|c':** Combine the NFAs for '(ab)*' and 'c' with a new start and accept state, connecting them with epsilon transfers.

4.7 Translate from NFA to regex

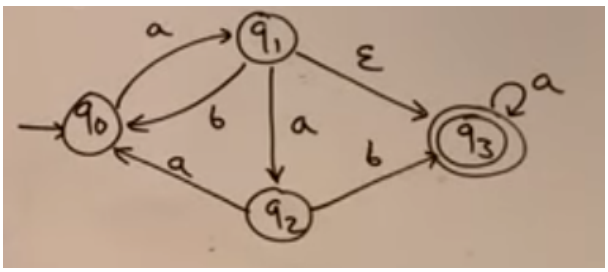

Figure 13: Initial NFA

There are three main steps to the translation:

1. Normalize NFA.
2. Pick states repeatedly with few transitions
3. Repeat

4.7.1 Normalize the NFA

To normalize the NFA you need to

1. Introduce a new start state with an epsilon transition to the old start state.
2. Introduce a new final state with an epsilon transition from the old start state.

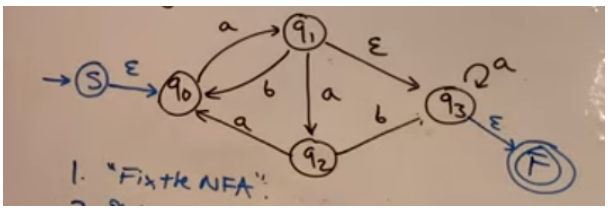

Figure 14: Enter Caption

4.7.2 Get to ripperoni

PS: HUSK AT $(epsilon|b) = b$? You can pick any state you want as long as you apply the method correctly.
We begin by ripping out q_3. To do this, we create an **in list** and an **out list**. In the **in list** we put all the states that have transitions in to the state (q_3), and on the **out list** we put all the states that the state transitions to.
These are the lists for q_3

In	Out
q_1	F
q_2	

Now, form a pairs between the in and out list:

- q_1 -> F
  - We are trying to emulate as though we go through q_3.
  - We have a self loop on q_3 for a. This converts to **a***
  - As $q_1 \to q_3 = \epsilon$ and $q_3 \to F = \epsilon$, we only have the **a***.
  - The final transition from $q_1 \to F$ consists of **a***.
- $q_2 \to F$
  - From $q_2 \to q_3$ we have a **b**
  - We have the self loop on q_3 for **a**, which gives to **a***
  - From $q_3 \to F$ we have an ϵ transition.
  - We are left with **ba***

Now, we can delete any transitions related to q_3:

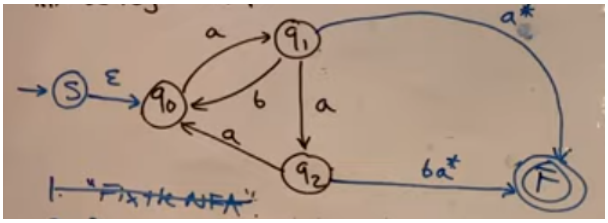

Figure 15: Removed q_3

Now, we pick the next state, choosing q_2.

In	Out
q_1	q_0
	F

- $q_1 \to q_0$
  - We already have an existing path between q_1 and q_0 consisting of **b**.
  - By going from q_1 through q_2 to q_0, we get **a** from $q_1 \to q_2$ and **a** from $q_2 \to q_0$.
  - As we have two different paths from $q_1 \to q_0$ (**aa** and **b**) we use the '|' operator, and get **aa|b**.
- $q_1 \to F$
  - We already have an existing path from $q_1 \to F$ consisting of **a***.
  - If we go from q_1 through q_2 to F, we get **a** from $q_1 \to q_2$ and **ba*** from $q_2 \to F$.
  - As we have two different paths, we get **a*|aba***

Now, we remove q_2 and its transitions.

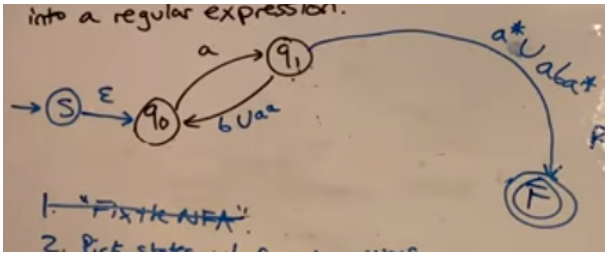

Figure 16: q_2 removed

Next, we rip out q_0

In	Out
S	q_1
q_1	

- $S \to q_1$
  - From $S \to q_0$ we have ϵ.
  - From $q_0 \to q_1$ we have **a**
  - The transition $S \to q_1$ becomes **a**.
- $q_1 \to q1$
  - From $q_1 \to q_0$ we have **b|aa**
  - From $q_0 \to q_1$ we have **a**
  - As this transition goes from q_1 to q_1 it is a self loop. As we have an alternative in **b|aa**, this should be in parantheses. We therefore get a self loop of **(b|aa)a**

Then we rip out q_0

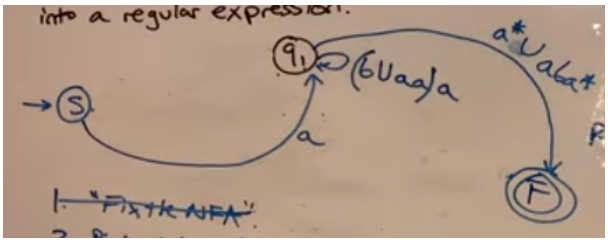

Figure 17: $q_0 removed$

Finally, we have q_1

In	Out
S	F

- $S \to F$

- $S \to q_1$ gives **a**
- Self loop on q_1 gives **((b|aa)a)***
- $q_1 \to F$ gives **a*|aba***

We are then left with **b((b|aa)a)*(a*|aba*)**
Take the regex out of the diagram, and present it isolated.

4.8 Translate from grammar to NFA

1. All Non-Terminals are states
2. All rules are transitions
   - $A \to \epsilon$: Create a transition from A to an unlabeled final state with label ϵ.
   - $A \to t$: Create a transition from A to an unlabeled final state with label t
   - $A \to tB$: Create a transition from A to B with label t
   - $A \to B$: Create a transition from A to B with label ϵ

Example: The following grammar can be converted to an NFA

- S ::= aS | aX | a
- X ::= bS | aY
- Y ::= bS | X

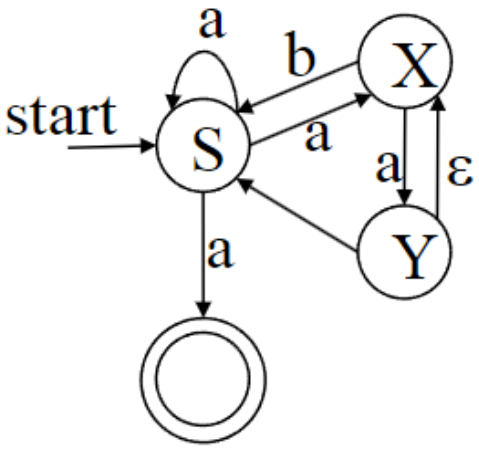

Figure 18: Enter Caption

4.9 NFA to grammar

We translate the NFA into grammar as follows:

1. All states are Non-Terminals
2. All transitions are rules:
   - A transition from A to B with label t: $A \to tB$
   - A transition from A to B with label ϵ: $A \to B$
   - A final state A: $A \to \epsilon$

The following NFA:

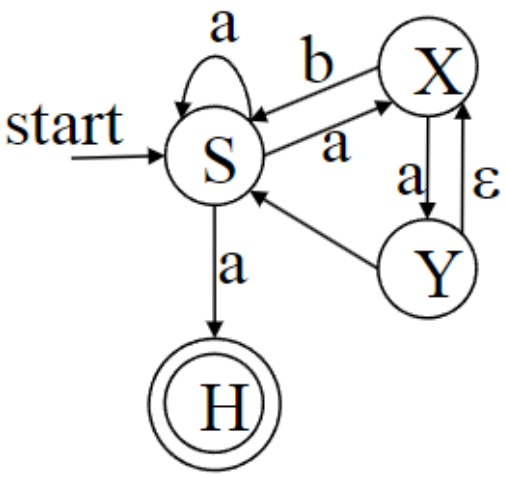

Figure 19: Enter Caption

will be this grammar:

- S::= aS | aX | aH
- X::= bS | aY
- Y::= bS | X

4.10 EXAM Questions

- Find a regex for a language and vice versa.
- Translate between regex and NFA.
- Translate from NFA to DFA. ✓
- Minimize a DFA. ✓
- Translate between grammar and NFA.

4.11 Quiz

4.12 1

Can a DFA be compared to a grammar?
- Yes, both define a language that can be compared.

4.13 2

Which regular expressions cover all of the following strings? ac, aabc, aaabbc, aaabbcc

- a*(ab)?(bc)?c?
- a+b*cc?
- a+b*c+
- a*b*c*
- a*(ab)?(bc)?c*

4.14 3

Which regular expression does not cover the following strings? aac, aabc, aabbcc, aaabbccc
- a+bb?c+

4.15 4

What is the relation between NFA and DFA?
- All DFA are also NFA

4.16 5

What is the relation between the languages that can be defined using NFA and DFA?
- The languages that can be defined with NFA and DFA are the same.

4.17 6

What happens when two lexical entities define the same strings? For example 'class', which is a keyword and an identifier.
- The first definition takes precedence.

4.18 7

How many transitions can leave a state in an NFA? - The number of outgoing transitions for a state is not limited.

4.19 8

Is it possible to reduce the size of an NFA?
- Yes, a similar procedure as for DFA works also for NFA.

4.20 9

For a given NFA, how many equivalent DFAs exist? - One

4.21 10

Which parts of regular expressions in computer science go beyond regular languages?

- start of line with ∧
- end of line with'$'
- references to sub-expressions with \1

5 Correct programs

5.1 Correctness

5.1.1 Why Does Software Fail?

Software failures can be attributed to several factors, distinct from the causes of hardware failures:

1. Latent design errors in software are fundamentally different from hardware failures.
2. Traditional strategies for mitigating hardware failures, such as duplicative redundancy, do not effectively address software errors.
3. The complexity of modern software systems makes 'complete' testing coverage unfeasible.
4. Testing the interactions between multiple processes and different subsystems presents significant challenges.
5. Performance optimizations in software can inadvertently introduce new errors.
6. Reusing software components in new contexts, even if previously qualified, may not be safe.

5.1.2 What to Achieve in Software Development?

The goals in software development should focus on reliability and correctness:

1. **Reliability:**
   Reliability in software means the level and frequency of failure are acceptable. It does not imply a complete absence of failures, but rather an acceptable level. Failures are measured pragmatically.
2. **Correctness:**
   Software is correct if its behavior aligns with its specification. Correctness is formally defined as the implementation matching the specification. The challenge often lies in obtaining a precise initial specification. Correctness is a means to achieve overall reliability.
3. **Correct Protocols:**
   There are no universally "correct" protocols; their correctness is context-specific.
   - Correctness in protocols is relative to specific requirements.
   - For example, the Alternating Bit Protocol ensures unique message reception and guarantees that every message sent is received at least once.
   - Protocols can be checked against standard requirements like the absence of deadlocks, unspecified receptions, dead code, and buffer overruns.

Five Basic Elements of Protocols

Protocols are essentially messaging algorithms with specific purposes and are comprised of the following elements:

- **The Service Provided by the Protocol**
  This defines the purpose and function of the protocol.
- **Constraints of the Environment**
  Understanding the operational environment and its limitations is crucial for the protocol's effectiveness.
- **Vocabulary of Messages**
  The set of messages used in the protocol for communication.
- **Encoding of Messages**
  The syntax or format of each message within the protocol's vocabulary.
- **Procedure Rules**
  These govern the message exchange process, forming the behavior or grammar of the protocol.

5.2 Temporal Logic

Focuses on reasoning about time and the ordering of events.

5.2.1 Temporal operators:

5.2.2 Linear Temporal Logic (LTL)

In LTL, time is treated in a specific way: It is considered implicit, meaning time progression is understood, but not explicitly stated; it is discrete, consisting of distinct, separate moments, it starts from an initial moment without any prior states; and it is theoretically infinite towards the future, as **described by Peano's axioms**.

The structure of LTL is based on an infinite sequence of states, denoted as π: s_0, s_1, s_2 etc. This sequence represents the progression of states over time. LTL incorporates various elements, including **atomic propositions** (basic, indivisible, statements), **Boolean Operators** (like AND($\wedge$, OR$\vee$, NOT$\neg$, IMPLIES) $\Longrightarrow$, and a set of temporal operators. These temporal operators include 'G' (Globally, meaning always true), 'F' (Finally, meaning eventually), 'X'(neXt, referring to the immediate next state), 'U' (Until) and 'R' (Release).

In simpler terms, the semantic intuition of LTL can be broken down as follows:

- **G f:** This means that 'f' is always true.
- **F f:** This means that 'f' will eventually be true.
  Words to look out for **eventually**, **finally**, **in the future**.
- **X f:** This refers to 'f' being true until the next state
  Words to look out for **Until**, **As Long As**, **Wait Until**
- **f U r:** This means 'f' holds true until 'r' becomes true
  Words to look out for **Release**, **Unless**, **Prevented By**
- **f R r:** This means 'r' releases 'f' from holding true
  Words to look out for **Never**, **Not in any future state**

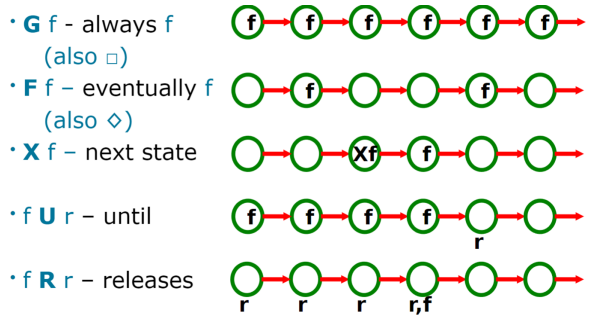

Figure 20: Semantic Intuition of LTL

Elements and Syntax

Element/Syntax	Description
P	Atomic Proposition
$\neg\Phi$	Negation of Φ
$\phi \wedge \psi$	Logical AND of ϕ and ψ
$\phi \vee \psi$	Logical OR of ϕ and ψ
$\Phi \rightarrow \Phi$	Logical implication
G Φ	Globally (always) Φ holds
F Φ	Eventually (in the future) Φ holds
X Φ	In the next state, Φ holds
$\phi U \psi$	ϕ Until ψ
$\phi R \psi$	ϕ Releases ψ

Typical LTL Formulae

LTL Formula	Description
G $\neg(critical1 \wedge critical2)$	Mutual exclusion: no two processes in the critical section simultaneously
GF myTurn	It is infinitely often 'my turn'
G(try $\rightarrow$ F critical)	Entering 'try' eventually leads to 'critical'
G(try $\rightarrow$ (critical R try))	'Try' holds as long as 'critical' does not hold
F G initialised	Once initialised, the system stays initialised

5.2.3 Specification

Process Description:

- Formal specification involves describing a system and its desired properties using a mathematically defined language.
- It focuses on writing these descriptions precisely and abstractly, simplifying and ignoring irrelevant details while highlighting central properties and characteristics.
- It avoids premature commitment to specific design and implementation choices.

5.2.4 Formal Proofs

Logic-Based System Analysis:

- Both the system and its properties are expressed as formulas in a mathematical logic framework.
- The framework includes a set of axioms and inference rules.
- Correctness proof involves proving properties from the system's axioms, often requiring human assistance.

5.2.5 Model Checking

Verification Technique:

- An automatic method for verifying finite-state systems. It involves building a finite model of a system and checking if a desired property holds within that model.
- The system is modeled as a state machine, and properties are expressed as logical formulas.
- The model checker verifies if the model satisfies the requirements, exploring all reachable paths in the computational tree derived from the state machine model, which can be computationally expensive.

5.3 Abstraction and Substitution

5.3.1 Abstraction and Information Hiding

- **Process Abstraction:** Hiding the implementation details of specific functionalities (code). It allows focusing on what the function does rather than how it does it.

- **Data Abstraction:** Concealing the representation of data types (like integers, stacks, etc.). It involves creating constructors for data creation, operators for data manipulation, and observers for returning simpler values.
- **Data Format Abstraction:** Hiding the low-level details of data transport and formatting.

5.3.2 Abstraction by Encapsulation

- **Reusable Entities:** Encapsulation involves creating self-contained entities like modules or classes that can be reused across different parts of a program.
- **Interface and Specification:** Modules typicaly have two parts: an interface defining how the module interacts with the rest of the system, and a specification detailing the module's functionality.
- **Component-Based Programming:** This approach allows building programs out of components, enhancing maintainability and scalability
- **Method-basd Access:** It's recommended to avoid direct attribute access in interfaces. Instead, using methods (getters and setters) offers better flexibility for changes.

5.3.3 Abstract Data Types (ADTs)

- **Type and Operation Protocols:** ADTs define a type and the operations on objects for that type within a single syntactic unit.
- **Representation Hiding:** The internal representation of the type's object is hidden from the units that use the type, ensuring a clear separation between the type's interface and its implementation.
- **Example: Peano in Java:** The **Peano interface** defines methods for basic arithmetic operations, with the actual implementation details provided in a class like **PeanoImpl**

Table of ADTs

ADT	Description
Stack	A collection that allows adding and removing elements in a last-in, first-out manner.
Queue	A collection allowing addition of elements at the end and removal from the front, implementing firs
List	An ordered collection of elements that can be accessed by their position.
Set	A collection that does not allow duplicate elements and does not preserve the order of elements.
Map/Dictionary	A collection of key-value pairs, allowing for fast retrieval based on the key.
Tree	A hierarchical structure with a root value and subtrees of children, with a parent node.
Graph	A set of nodes connected by edges, representing relationships between elements.
Priority Queue	Similar to a queue, but each element is assigned a priority and the element with the highest priority

5.3.4 Peano Interface in Java

```
public interface Peano {
    Peano zero();
    Peano succ(Peano p);
    Peano pred(Peano p);
    Peano add(Peano p1, Peano p2);
}
```

5.3.5 Peano Implementation in Java

```
    public class PeanoImpl implements Peano {
    private final PeanoImpl predecessor;

    // Constructor for zero (no predecessor)
    public PeanoImpl() {
        this.predecessor = null;
    }

    // Private constructor used for successor
    private PeanoImpl(PeanoImpl predecessor) {
```

```
        this.predecessor = predecessor;
    }

    // Static method to create zero
    public static PeanoImpl zero() {
        return new PeanoImpl();
    }

    // Returns the successor of this Peano number
    @Override
    public PeanoImpl succ(Peano p) {
        return new PeanoImpl((PeanoImpl) p);
    }

    // Returns the predecessor of this Peano number
    @Override
    public PeanoImpl pred(Peano p) {
        PeanoImpl impl = (PeanoImpl) p;
        if (impl.predecessor != null) {
            return impl.predecessor;
        }
        throw new IllegalStateException("Cannot find predecessor of zero.");
    }

    // Adds two Peano numbers
    @Override
    public PeanoImpl add(Peano p1, Peano p2) {
        if (p1 == null) return (PeanoImpl) p2;
        return add(pred(p1), succ(p2));
    }
}
```

In this implementation:

- Each **PeanoImpl** object has a **predecssor** which is **null** for the **zero** representation
- The **succ** method creates a new **PeanoImpl** instance with the current instance as its predecessor.
- The **pred** method returns the predecessor of a given **Peano** number, and it throws an exception if we try to find the predecessor of zero.
- The **add** method recursively defines addition based on **succ** and **pred**.

5.3.6 Liskov Substitution Principle

- **Substitutability of Implementations:** This princple states that implementations of the same interface should be interchangeable without altering the program's desirable properties.
- **Two Levels of Substitutability:**
  - Static Level: The compiler allows the substitution without errors.
  - Behavioral Level: The substituted implementation should offer the same behavior as the original.

5.4 Design by Contract

The fundamental idea behind DbC is to view software correctness in terms of "contracts" between different parts of a software system, particularly between a method and its callers.

5.4.1 Key Concepts of Design by Contract

1. **Correctness as Consistency:**
   Correctness in DbC is not an absolute notion but is relative to the given specification. It is better understood as consistency between the software's implementation and specification.
2. **Correctness Formula P A Q:**
   This formula encapsulates the essence of DbC. **P** represents the precondition that must be true before the execution of action **A**. **Q** is the postcondition that must be true after the execution of **A**. The formula states that any execution of **A** starting in a state where **P** holds, will terminate in a state where **Q** holds.

   Example: **x>=9 x:=x+5 x>=14** implies that **x** is at least 9 before the operation, it will be at least 14 afterwards.
3. **Strengthening and Weakening Conditions:**
   - A condition **A** is stronger than a condition **B** iif **A** implies **B**. In DbC, you can strengthen the precondition and weaken the postcondition without changing the underlying code.
   - Example: Strengthening a precondition from **a>0** to **a>10**, and weakening a postcondition from **x==6** to **2|x** (x is divisible by 2).
4. **Responsibility and Correctness:**
   - In DbC, both the client and the supplier have obligations and benefits. For example, in a **put()** operation on a stack, the client's obligation is to call **put(x)** only on a non-full stack. The benefit for the client is that the stack gets updated with **x** on top. The supplier's obligation is to ensure the stack is updated, and the benefit is simpler processing due to the guaranteed non-full stack.
5. **Invariant:**
   - An invariant is a condition that remains true throughout the life of an object (or system). It must hold before and after any public method call, provided the method's preconditions are met.
   - Invariants help to ensure that an object never enters an illegal state.

5.4.2 Client and Supplier

Client:

1. **Definition:**
   In DbC, a "client" refers to the part of the software that calls or uses a function, method, or a class. It's the consumer of a service provided by another part of the software.
2. **Responsibilities:**
   - The client is responsible for ensuring that the preconditions of a method or function are met before calling it. These preconditions are part of the contract established by the supplier (the method or function being called).
   - For example, if a method requires an input parameter to be non-null, it'ss the client's responsibility to check this before making the call.

   Rationale:
   This approach ensures that the responsibility for correct usage of a method or function is clearly defined. It avoids redundant checks within the method (defensive programming), promoting cleaner and more efficient code.

Supplier:

1. **Definition:**
   The supplier in DbC is the part of the software that provides a function, method, or class. It's the producer of a service that the client consumes.
2. **Responsibilities:**

   - The supplier is responsible for ensuring that the postconditions and invariants are met. In other words, once the method's preconditions are satisfied, the supplier guarantees that certain conditions will be true after the method's execution.
   - The supplier also defines the preconditions and invariants that must hold true.
3. **Rationale:**
   This delineation ensures that the responsibility of maintaining the integrity and expected behavior of a method or function is clearly with the supplier. The supplier guarantees that the method will behave as specified, provided the preconditions are met.

5.5 Peano in JML (Java Modeling Language)

JML is a behavioral interface specification language that can be used to specify the behavior of Java moduls (classes and interfaces). It's used to formally specify the pre-conditions, post-conditions, and invariants in Java code.
JML adopts a design-by-contract approach, where the contract for a method or class is specified by pre-conditions, post-conditions, and invariants in a way that's both human-readable and machine-checkable.

5.5.1 Key Concepts

1. **Pre-Conditions:** These are conditions that must be true before a method is executed. They are specified using the //@ **requires** clause in JML.
2. **Post-Conditions:** These are conditions that must be true after a method has finished executing. They are specified using the //@ **ensure** clause in JML.
3. **Invariants:** These are conditions that must always hold true for an instance of a class. They describe the consistent state of an object.

5.5.2 JML Overview of the Set Interface

Original Code

```
import java.util.Vector;
import java.util.Iterator;

public interface Set {
    //@ ensures empty()
    public Set();

    //@ ensures \result != null;
    public Iterator iterator();

    //@ ensures \result == \not \exists(Object x; is_in(x));
    public boolean empty();

    //@ requires o != null;
    public boolean is_in(Object o);

    /*@ requires o != null;
      @ ensures is_in(o) && \forall(Object x;
      \old(is_in(x)) ==> is_in(x);) */
    public void add(Object o);

    /*@ ensures not is_in(o) &&
      \forall(Object x; x \= o &&
      \old(is_in(x)) ==> is_in(x);) */
    public void remove(Object o);

    /*@ requires otherSet != null;
      @ ensures \forall(Object x;
      is_in(x) || otherSet.is_in(x) <==> \result.is_in(x)); */
    public Set union(Set otherSet);

    /*@ requires otherSet != null;
      @ ensures \forall(Object x;
      is_in(x) && otherSet.is_in(x) <==> \result.is_in(x)); */
    public Set intersection(Set otherSet);
}
```

Method Explanations

Constructor Set()

- **Ensures:** The newly created set is empty. This is a post-condition ensuring that after calling the constructor, the ‘empty()‘ method will return true.

Method iterator()

- **Ensures:** The returned iterator is not null. This post-condition guarantees the non-nullity of the iterator for the set.

Method empty()

- **Ensures:** Returns true if and only if there are no elements in the set. The post-condition uses logical negation and existential quantification to express this.

Method is_in(Object o)

- **Requires:** The object ‘o‘ being checked must not be null. This is a pre-condition for the method.

- **Behavior:** Checks if the specified object is in the set.

Method add(Object o)

- **Requires:** The object ‘o‘ to be added must not be null.
- **Ensures:** After adding ‘o‘, it is in the set, and all other elements that were in the set remain. It uses universal quantification and the ‘\old‘ construct to refer to the state before the method call.

Method remove(Object o)

- **Ensures:** The object ‘o‘ is not in the set after execution, and all other elements remain unchanged in the set.

Method union(Set otherSet

- **Requires:** The other set to union with must not be null.
- **Ensures:** The resulting set contains all elements that were either in the original set or in ‘otherSet‘.

Method intersection(Set otherSet)

- **Requires:** The other set to intersect with must not be null.
- **Ensures:** The resulting set contains only those elements that were in both the original set and ‘otherSet‘.

5.5.3 JML Overview of the Peano Interface

Original Code

```
public interface Peano {
    //@ ensures \result != null
    public Peano zero();

    //@ requires p != null;
    //@ ensures \result != null && \result != zero()
    public Peano succ(Peano p);

    //@ requires p != null && p != zero();
    //@ ensures \result != null && succ(\result) == p
    public Peano pred(Peano p);

    //@ requires p1 != null && p2 != null;
    //@ ensures \result == (p2==zero()) ? p1 : succ(add(p1,pred(p2)))
    public Peano add(Peano p1, Peano p2);
}
```

5.5.4 Method Explanations

Method zero()

- **Ensures:** The result is not null. This post-condition ensures that calling ‘zero()‘ will always return a non-null Peano object, representing the number zero in Peano arithmetic.

Method succ(Peano p)

- **Requires:** The Peano object ‘p‘ must not be null.
- **Ensures:** The result is a non-null Peano object and is not the zero object. This method represents the successor function in Peano arithmetic, which returns the next number.

Method pred(Peano p)

- **Requires:** The Peano object ‘p‘ must not be null and must not be the zero object.
- **Ensures:** The result is a non-null Peano object such that its successor is equal to ‘p‘. This method represents the predecessor function, returning the previous number in Peano arithmetic.

Method add(Peano p1, Peano p2)

- **Requires:** Both Peano objects ‘p1‘ and ‘p2‘ must not be null.
- **Ensures:** The result is equal to the sum of ‘p1‘ and ‘p2‘ in Peano arithmetic. The method recursively defines addition based on the ‘zero‘, ‘succ‘, and ‘pred‘ functions.

5.6 Refinement

5.6.1 Initial clarifications

- **[**= is often used in the contect of refinement calculus and formal methods to denote a refinement step. In this context, is it used to indicate that the statement or program segment on the right side is a refinement of the one on the left side. Essentially means, "is refined to" or "can be transformed into".
- **What is "rel"?**
  "Rel" typically stands for a relation or a relational expression. It represents a condition or a set of conditions that relate different variables or states in a program.

  In a program statement like **v:=v'|rel**, **rel** would be the condition that defines how the new value **v** is related to other avriables or to the previous state of **v**. It's a way of specifying the rules or constraints that the assignment must fullow. The relational expression helps ensure that the assignment is done correctly and that the program's logic is maintained.

  In summary, "rel" is used to express the logical constrains or relationships that must hold true for the operation (like an assignment) to be considered valid or correct in the given context.

5.6.2 Logical Symbols

Symbol	Description
/\	Logical AND
\/	Logical OR
~	Logical NOT
==>	Logical implication (if ... then ...)
==	Equality
false	Logical constant representing falsehood
true	Logical constant representing truth
(!x . p)	Universal quantification (for all x, p holds)
(?x . p)	Existential quantification (there exists an x for which p holds)

5.6.3 Inference Rule

Symbol	Description
p —— q	p implies q (if p holds, then q can be inferred)

5.6.4 Sets and Functions

Symbol	Description
in	Membership in a set
{ x | p }	Set comprehension (set of all x such that condition p holds)
dom R	Domain of relation R
ran R	Range of relation R
f(x)	Function application
s[i]	Array access (element i of array s)
#s	Size of the array s
E <= F	Ordering (E is less than or equal to F)
E >= F	Ordering (E is greater than or equal to F)

5.6.5 Programming Constructs

Symbol	Description
skip	Do nothing
x := E	Assignment (assign expression E to variable x)
seq(;)	Sequential composition of statements
if-then-else	Conditional execution
DO g -> S OD	Loop with guard g and body S
VAR x:X	Declaration of variable x of type X
CON x:X	Declaration of constant x of type X
{ p }	Assertion (p should hold at this point in the program)

5.6.6 Generalized Assignment

Symbol	Description
x := x' | q	x is assigned a value x' that satisfies condition q.

5.6.7 Specification Statement

Symbol	Description
{ p } x := x' | q	Statement with precondition p and postcondition q.

5.6.8 Refinement Rules Lookup Table

Refinement Rules Lookup Table

Rule Name	**Description**	**When to Use**
Introduce Local Variable (InV)	Introduces a new local variable into a program segment, maintaining the original behavior.	Use when a temporary or additional variable is needed for clarity or computation.
Introduce Assertion (InA)	Adds an assertion, a condition that should hold true at a specific point in the program.	Use for documentation, validation, or when a specific condition must hold at a certain point.
Insert Leading Assignment (Las)	Adds a new assignment before the main part of a program segment.	Use when initialization or preparation is required before the main operations.
Strengthen relation/postcondition (Str)	Makes the relation or postcondition in a program segment more specific or restrictive.	Use to tighten the behavior of a program segment or to make postconditions more precise.
Contract Frame (Fra)	Reduces the set of variables affected by a program segment.	Use when focusing on specific variables or simplifying a program's logic.
Introduce Assignment (Ass)	Makes the assignment of a new value to a variable explicit and specific.	Use when the assignment in a program needs to be clarified or made more direct.
Rewrite (Rew)	Replaces one expression with another equivalent expression within a program.	Use for optimization, simplification, or when an equivalent expression enhances clarity.
Weaken Assert/precondition (Wea)	Relaxes or weakens the precondition or assertion in a program segment.	Use to make the program more general or to reflect a broader range of applicability.
Introduce IF (InI)	Introduces conditional branching (if-then-else) into a program.	Use when different scenarios or conditions need to be handled explicitly.

5.6.9 Introduce Local Variable (InV)

This rule is about introducing a new local variable into a program while preserving its behavior. The syntax and the process can be explained as follows: **Original Statement Syntax:**

`{pre} v:=v'|rel`

- **pre** represents the precondition that must hold before the execution of this statement.
- **v:=v'|rel** indicates that variable **v** is being assigned a new value **v'**, subject to the relation **rel**. This relation is a condition that describes how **v'** is related to other variables or states in the program.

Refined Statement with Local Variable:

`VAR l:T. l:=E; {pre} v,l:=v',l'|rel`

- **VAR 1:T** introduces a new variable **1** of type **T**.
- **I:=E** assigns an initial value **E** to **1**.
- **pre v,1=v'1'|rel** is the refined statement where **v** and the new local variable **1** are being assigned new values (**v'** and **1'**, respectively), again subjet to possibly updated relation **rel**.

Example:

`x,y:=x',y' | x'=y /\ y'=x [= VAR t:T t:=x; x,y,t:=x',y',t' | x'=y /\ y'=x`

This example swaps the values of **x** and **y** using temporary variable **t**.

5.6.10 Introduce Assertion (InA)

The "Introduce Assertion (InA) rule is used to add an assertion into a program. An assertion is a statement that declares a condition that should always hold true at that point in the program.

Original Statement Syntax:

`v:=E`

v:=E simple assigns the value of expression **E** to the variable **v**.

Refined Statement with Assertion:

`v:=E; {v=E}`

After assigning **E** to **v**, an assertion **e=E** is introduced. This asserts that immediately after the assignment, the value of **v** should be equal to the expression **E**.

Example:

`x:=2*3+y [= x:=2*3+y; {x=2*3+y}`

This example assigns the value of **2*3+y** to **x** and then asserts that **x** must be equal to **2*3+y**.

5.6.11 Insert Leading Assignment (LAS)

This rule is about adding a new assignment to the beginning of a program segment. The purpose of this rule is to prepare certain variables or set up conditions before the main part of the program segment executes.

Original Statement Syntax:

`v,x:=v',x'|rel`

Here **v** and **x** are variables being assigned new values (**v'** and **x'** respectively), under the condition **rel**.

Refined Statement with Leading Assignment:

```
x:=E; v,x:=v',x'|rel
```

- **x:=E** is the new leading assignment. Here, the variable **x** is assigned a value **E** before the original operation.
- **v,x:=v',x'|rel** is the original statement, now following the assignment.

Example:

```
x,y,t:=x',y',t' | x'=y [= x:=y; x,y,t:=x',y',t' | x'=y
```

In this example, the value of **y** is first assigned to **x**, and then the original operation is carried out.

5.6.12 Strengthen Relation / Post-Condition (STR)

This rule involves strengthening the relation or postcondition in a program segment. Strengthening a postcondition means making it more specific or restrictive, which can be useful for refining the behavior of a program.

Original Statement Syntax:

```
{pre} v:=v'|rel1
```

- **pre** is the precondition for the statement
- **v:=v'reli** means that **v** is assigned a new value **v'** under the condition **rel1**

Refined Statement with Strengthened Postcondition:

```
{pre} v:=v'|rel2
```

- **pre remains the same postcondition.**
- **v:=v'|rel2** is the refined statement where **rel2** is a strengthened (or more specific) version of **rel1**. The implication is that **pre** /\ **rel2** ==> **rel1** meaning **rel2** is a stronger or more restrictive relation compared to **rel1**
- IMPROVE LAST STATEMENT
- INSERT OBLIGATION, WHICH MEANS SHOWING THAT THE CHANGE IS TRUE

Example:

```
x,y,t:=x',y',t' | x'=x [= x,y,t:=x',y',t' | t'=t /\ x'=x
```

In this example, the original relation **x'=x** is strengthened to **t'=t** /\ **x'=x**, adding an additional condition involving **t**.

In both LAS and STR rules, the idea is to refine and adjust the program in a way that makes it more precise or prepares the environment better for subsequent operations. LAS is about setting up initial conditions or variable states, while STR is about making the postconditions more specific to tighten the program's behavior and ensure more precise outcomes.

5.6.13 Contract Frame (FRA)

The "Contract Frame" rule is used to reduce the scope or 'frame' of the variables affected by a particular program segment. This is often done to simplify the program or focus on the variables that are actually being modified.

Original Statement Syntax:

```
x,v:=x',v'|rel /\ x'=x
```

This syntax indicates that both **x** and **v** are assigned new values (**x'** and **v'** respectively), under a condition **rel**. Additionally **x'=x** specifies that the new value of **x**, **x'** is the same as its old value **x**, essentially meaning **x** is unchanged.

Refined Statement with Contracted Frame:

`v:=v'|rel[x'\x]`

In this refined version, the assignment focuses only on **v**. The expression **rel[x'\x]** indicates that the relation **rel** is now expressed solely in terms of **v** with **x'** being replaced by **x** in the relation, acknowledging that **x** remains unchanged.

Example:

`x,y:=x',y' | x'=x /\ y'=t [= y:=y' | y'=t`

This example shows a refinement where the original assignment updates both **x** and **y**, but since **x** remains unchanged (**x'**=**x**), the refined statement only updates **y**.

5.6.14 Introduce Assignment (ASS)

The ASS rule is used to explicitly specify the new value of a variable. This rule makes the assignment more concrete and specific.

Original Statement Syntax:

`x:=x'|x'=E`

Here, **x** is assigned a new value, **x'**, and the condition **x'**=**E** specifies what the new value **x'** should be.

Refined Statement with Explicit Assignment:

`x:=E`

The refined statement directly assigns the expression **E** to **x**. This makes the assignment explicit and clear, removing any ambiguity about the value to be assigned to **x**.

Example:

`z:=z' | z' = 2*x [= z:=2*x`

In this example **z** is initially assigned a value **z'** such that **z'** = **2*x**. The refinement simplifies this to a direct assignment **z:**=**2*x**.

In both FRA and ASS rules, the goal is to make the program more concise and focused. FRA is about narrowing down the set of variables that are actively involved in a computation, thereby simplifying the program's logic. ASS on the other hand, is about making the assignments in the program more explicit and straightforward, thereby enhancing clarity and reducing ambiguity.

5.6.15 Rewrite (Rew)

The REW rule in refinement calculus is used to replace one expression with another equivalent expression within a program. This is often done for clarity, optimalization, or to facilitate proofs.

Original Statement Syntax:

`{E1=E2} v:=v'|rel[E1]`

- **E1=E2** is an assertion that **E1** is equal to **E2**.
- **v:=v'|rel[E1]** indicates that **v** is being assigned a new value **v'** under the condition **rel[E1]**, where **E1** is part of the condition.

Refined Statement with Rewrite:

```
v:=v'|rel[E2]
```

In the refined version, **rel[E1]** is replaced by **rel[E2]**. Since **E1** is asserted to be equal to **E2**, this replacement does not change the meaning of the program.

Example:

```
{max x y = x} z:=z' | z' = max x y [= z:=z' | z' = x
```

In this example, the condition **z' = max x y** is rewritten as **z' = x**, based on the assertion **max x y = x**.

5.6.16 Weaken Assert/Precondition (WEA)

The WEA rule is about relaxing or weakening the precondition or assertion in a program segment. This is typically done to make the program more general or to reflect a broader range of applicability.

Original Statement Syntax:

```
{pre1}
```

pre1 is a precondition or assertion that must hold at this point in the program.

Refined Statement with Weakened Precondition:

```
{pre2}
```

pre2 is a weaker or more general precondition than **pre1**. The implication is that **pre1** implies **pre2** (i.e., **pre1** ==> **pre2**), meaning that every situation where **pre1** holds, **pre2** will also hold.

Example:

```
{x>y} z:=z' | z' = x [= {max x y = x} z:=z' | z' = x
```

Here, the original precondition **x>y** is weakened to **max x y = x**. The weakening is valid if **x>y** implies **max x y = x**.

In both REW and WEA rules, the focus is on modifying the program to enhance its clarity, generality, or applicability. REW involves replacing expressions with equivalent ones to possibly simplify or clarify the program, while WEA involves loosening the constraints impoes by preconditions or assertions, thereby broadening the scope or applicability of the program segment.

5.6.17 Introduce IF (InI)

The IF rule is used to introduce conditional branching (an **if-the-else** structure) into a program. This is often done to handle different scenarios or conditions more explicitly within the program logic.

Original Statement Syntax:

```
{pre} v:=v'|rel
```

- **pre** represents the precondition that must be true before executing the statement.
- **v:=v'|rel** indicates that the variable **v** is being assigned a new value **v'** subject to the condition **rel**.

Refined Statement with IF

```
if G then {G /\ pre} v:=v'|rel else {~G /\ pre} v:=v'|rel
```

- The **if G then ... else ...** structure introduces a conditional branch based on the guard **G**.
- **G /\pre** is the precondition when **G** is true.
- **~G /\pre** is the precondition when **G** is false.
- In both branches, **v** is assigned a new value **v'**, but the condition under which this assignment happens is guided by whether **G** is true or false.

Example:

```
z:=z' | z' = max x y [= if x>y then {x>y} z:=z' | z' = max x y else {x<=y} z:=z' | z' = max x y
```

- In this example, an **if-then-else** structure is introduced based on the comparison **x>y**.
- If **x>y** is true, then the assignment is made under the condition **x>y**.
- If **x>y** is false, (i.e., **x <= y**, then the assignment is made under the condition **x<=y**.
- In both cases **z** is assigned a value **z'** such that **z' = max x y**, but the distinction is made clear by the conditional branching.

The INI rule is particulary useful for refining program that need to handle different cases or conditions distinctly. By introducting condiitional branching, the program can be made more robust and adaptable to varying scenarios, improving clarity and maintainability.

5.6.18 Correctness of Loops

1. **Initial value:** The state of all relevant variables at the start of the loop.
2. **State Where to Start:** The conditions or state of the system when the loop begins execution.
3. **Loop Invariant:** A condition that holds true before and after each iteration of the loop. It is essential for proving the correctness of the loop.
4. **What Stays Constant (In Loop Invariant):** The aspect of the program state that remains unchanged throughout the execution of the loop.
5. **Loop Variant:** A measure (usually a positive integer) that decreases with each iteration of the loop. It is used to ensure that the loop terminates.
6. **What is Changing and How (In Loop Variant):** Describes how the loop variant changes (decreases) with each iteration.
7. **Exit Condition:** The condition under which the loop will stop executing. This condition is met after applying the loop variant.
8. **Loop Body:** The set of statements or operations that are executed in each iteration of the loop.
    - **Init:** The initialization part of the loop.
    - **Post:** The postcondition that should hold after the loop terminates.

Introduce DO (InD)
When introducing a **DO** loop (a loop with a guard and a body), the following components are essential:

1. **Guard (G):** A condition stating when the loop will continue its execution.
2. **Loop Invariant:** A condition that must be true at the start of the loop and should remain true after each iteration. At the end of the loop, the invariant should imply the postcondition.

3. **Loop Variant (E):** A measure that ensures termination. It should start as positive integer and decrease with each loop iteration.
4. **Correctness Condition:** The following correctness conditions must be met:
   - **post does not mention v:** The postcondition should not depend on the variable **v** being modified in the loop.
   - **pre ==> inv:** The precondition **pre** should imply the loop invariant **inv**.
   - **inv & ~G ==> post[v'\v]**: The loop invariant combined with the negation of the guard (**~G**) should imply the postcondition, with **v** replaced by **v**.
5. **Introduce DO Statement Syntax:**

```
{pre} v:=v' | post [=
DO G -> {inv & G} v:=v' | inv[v\v'] & 0<=E[v\v']<E OD
```

   - **pre** is the precondition before the loop starts.
   - **DO G-> ... OD** is the loop structure.
   - **G** is the guard condition for continuing the loop.
   - **inv & G** ensures that the loop invariant and guard hold true at the beginning of each iteration.
   - **v:=v' | inv[v \ v'] & 0<=E[v \ v']<E** is the loop body, where **v** is assigned a new value **v'** that satisfies the loop invariant with the updated variables, and the loop variant **E** decreases.

In summary, introducing a **DO** loop involves carefully defining the guard, invariant, and variant to ensure the loop behaves correctly, maintains certain properties, and eventually terminates. This rigorous approach is crucial in formal methods to guarantee the correctness and termination of loops in programs.

5.6.19 Example: Finding the Maximum

Specification:

```
Max(x, y, z: NUM) z:=z' | z' = max x y
```

This specifies a function to find the maximum of two numbers, x and y, and assign it to z. The postcondition $z' = maxxy$ states that the new value of z (denoted z') should be the maximum of x and y.

Step 1: Introduce IF (InI)

```
[= (InI) if x>y then {x>y} z:=z' | z' = max x y
else {x<=y} z:=z' | z' = max x y
```

Introduces an 'if-then-else' structure based on the comparison $x > y$. If x is greater than y, the maximum is determined under the condition $x > y$; otherwise, under $x \leq y$.

Step 2: Weaken Assert/precondition (WeA)

```
[= (WeA) if x>y then {max x y = x} z:=z' | z' = max x y
else {x<=y} z:=z' | z' = max x y
Obligation: x>y ==> max x y = x
```

Weakens the assertion in the 'then' branch to $\{maxxy = x\}$, based on the obligation that if $x > y$, then $maxxy$ is equal to x.

Step 3: Weaken Assert/precondition (WeA) Continued

```
[= (WeA) if x>y then {max x y = x} z:=z' | z' = max x y
else {max x y = y} z:=z' | z' = max x y
Obligation: x<=y ==> max x y = y
```

Similarly, weakens the assertion in the 'else' branch to $\{maxxy = y\}$, based on the obligation that if $x \leq y$, then $maxxy$ is equal to y.

Step 4: Rewrite (Rew)

```
[= (Rew) if x>y then z:=z' | z' = x
else {max x y = y} z:=z' | z' = max x y
```

Rewrites the assignment in the 'then' branch to $z := z'|z' = x$, simplifying the expression based on the previously weakened assertion.

Step 5: Introduce Assignment (Ass)

```
[= (Ass) if x>y then z:=x
else {max x y = y} z:=z' | z' = max x y
```

Introduces a direct assignment in the 'then' branch: $z := x$. This simplifies the code by explicitly assigning the value of x to z.

Step 6: Rewrite (Rew) Continued

```
[= (Rew) if x>y then z:=x else z:=z' | z' = y
```

Applies a similar rewrite to the 'else' branch, changing the assignment to $z := z'|z' = y$.

Step 7: Introduce Assignment (Ass) Continued

```
[= (Ass) if x>y then z:=x else z:=y
```

Finally, introduces a direct assignment in the 'else' branch: $z := y$. This completes the refinement, resulting in a clear, executable 'if-then-else' structure that assigns the maximum of x and y to z.

5.7 EXAM questions

5.7.1 Translate natural language into temporal logic

1. **Identify Key Components:**
   Read the natural language statement and identify the main components, such as events, conditions, and temporal aspects (e.g., always, eventually, next, until).
2. **Determine Temporal Operators:** Map the identified temporal aspects to corresponding LTL operators:
   - **Always (Globally):** Use **G** for statements that always hold.
   - **Eventually (Future):** Use **F** for events that will occur at some point in the future.
   - **Next:** Use **X** for something that will happen in the immediate next state.
   - **Until:** Use **U** for situations where one condition holds until another becomes true.
   - **Release:** Use **R** in scenarios where one condition ensures another holds until it itself becomes true.
3. **Formulate LTL Expression:** Combine the temporal operators and conditions to create the LTL expression. Pay attention to logical connectors (AND, OR, NOT, IMPLICATION) that might be implicit in the natural language statement.

Example:
Natural Language Statement: "Whenever the system enters a critical state, it will eventually return to a safe state."

Translation:

1. **Key Components:** System entering a critical state, eventually returning to a safe state.
2. **Temporal Operator: G** for the ongoing condition (whenever) and **F** for the future event (eventually)
3. **Conditions:** Critical state **critical**, safe state **safe**.
4. **LTL Expression: G(critical -> F safe)**
5. **Review:** The expression reads as "Globally, if the system is in a critical state, it is followed by an eventual transition to a safe state".

Other examples

Natural Language	Temporal Logic
The professor is never late.	G !late
The light eventually turns off.	F off
After sending a request, a response is received next.	sendRequest -> X receiveResponse
The door remains closed until someone opens it.	closed U open
If the alarm sounds, it will keep ringing until it is turned off.	alarm -> (ringing U turnedOff)

5.7.2 Translate temporal logic into natural language

1. **Understand the LTL Expression:** Break down the LTL expression into its basic components: temporal operators and propositions.
2. **Interpret Temporal Operators:** Translate the temporal operators into their natural language equivalents (Globally, Eventually, Next, Until, Release).
3. **Translate Conditions:** Turn the atomic propositions and logical expressions into natural language statements and conditions.
4. **Construct the Natural Language Statement:** Combine the temporal descriptions and conditions into a coherent natural language statement.

Example:
LTL Expression: F G !error

Translation:

- **Components: 'F', 'G' '!error'**
- **Temporal Interpretation:** Eventually **F**, Always **G**
- **Conditions:** Error state (**!error** meaning no error).
- **Natural Language Statement:** "Eventually, the system will always be free of errors".

Other examples

Temporal Logic	Natural Language
G !rain	It never rains.
F arrive	Eventually, they will arrive.
X taskCompleted	The task will be completed in the next state.
work U lunchTime	Keep working until it is lunchtime.
hungry -> (eating U notHungry)	If you are hungry, you will keep eating until you are not hungry anymore.

5.7.3 Write JML for a function

1. **Identify Function Parameters and Return Type:**

2. **Define Preconditions (Optional)**
   - **If** there are specific conditions or constraints that must be true before the function is called: **Use @requires** to specify each precondition.
   - **Else:** Skip this step if the function has no preconditions
3. **Define Postconditions:**
   - Determine what conditions must hold true after the function execution.
   - Use **@ensures** to specify each postcondition. This often involves:
     - Stating relationships between the return value and the input parameters.
     - Describing any changes or lack of changes to the object's state (if the function is a method of a class).
4. **Consider Invariants (For Class Methods):**
   If the function is a method of a class:
   - Determine if there are any conditions that always hold true for the class's state.
   - Use **@invariant** to specify these conditions.

   Else: Skip this step for standalone functions.
5. **Write JML Comments:**
   - Place the JML specifications as comments above the function declaration.
   - Use the JML syntax for preconditions, postconditions, and invariants.

Example:
For a function **int max(int a, int b), the JML specification might look like this:**

```
/**
 * Returns the larger of two integers.
 * @param a the first integer
 * @param b the second integer
```

```
 * @return the larger of a and b
 * @ensures \result >= a && \result >= b // The result is greater than or equal to both
 * @ensures (\result == a) || (\result == b) // The result is either a or b
 */
public int max(int a, int b);
```

Tips:

- Be as precise as possible in your preconditions and postconditions.
- If the function modifies the state of an object, include postconditions that describe these modifications.
- For class methods, always consider whether there are class invariants that should be included.

5.7.4 Apply one or more refinement rules

Refinement rules are used to systematically transform a specification or code into a more detailed or concrete version while preserving its correctness. Follow these steps to apply refinement rules effectively:

Identify the Goal of Refinement

- Determine what aspect of the software needs refinement (e.g., functionality, performance, readability).
- Understand the current state of the specification or code and what the end goal is.

Choose Appropriate Refinement Rules

1. Review available refinement rules and understand their purposes. Common rules include:
    - Introduce Local Variable (InV)
    - Introduce Assertion (InA)
    - Insert Leading Assignment (Las)
    - Strengthen relation/postcondition (Str)
    - Contract Frame (Fra)
    - Introduce Assignment (Ass)
    - Rewrite (Rew)
    - Weaken Assert/precondition (Wea)
    - Introduce IF (InI)
2. Select the rule(s) that best fit the refinement goals.

Apply the Selected Rule(s)

- Carefully apply the chosen refinement rule to the specification or code.
- Ensure that each application of the rule is done systematically and correctly.
- For each refinement step, document the changes and rationale behind them.

Validate the Refinement

1. After applying the refinement, check that the refined specification or code still meets the original requirements.
2. Ensure that the refinement has not introduced any new errors or issues.
3. Verify that the refined version enhances the aspect (functionality, performance, etc.) it was intended to improve.

Iterate if Necessary

- If the refinement does not fully meet the goals, or if further improvements are needed, repeat the process with additional or alternative rules.
- Continue refining until the desired level of detail and correctness is achieved.

5.7.5 Check some code for correctness

Check correctness of code:

1. Review the JML specifications (if available) to understand the expected behavior of the code.
2. Examine the implementation of abstract data types (ADTs) and ensure they adhere to their definitions.
3. If applicable, verify the correct implementation of temporal logic expressions.
4. Check for adherence to established coding standards and best practices.
5. Use static analysis tools to detect potential coding errors or bugs.
6. Perform unit testing and ensure high test coverage, particularly focusing on edge cases.
7. Validate the implementation against the formal specifications or contracts.
8. Ensure the code handles error conditions and edge cases gracefully.
9. Review any use of design patterns or architectural principles for consistency.
10. Conduct peer reviews to get an independent assessment of the code.
11. Validate the final implementation against the initial requirements or specifications.

Checklist:

- ☐ JML specifications are correctly implemented.
- ☐ ADTs conform to their definitions and usage.
- ☐ Temporal logic, if used, is correctly applied.
- ☐ Code follows best practices and coding standards.
- ☐ Static analysis does not reveal critical issues.
- ☐ Unit tests cover a wide range of scenarios and pass successfully.
- ☐ Error handling and edge cases are adequately addressed.
- ☐ Design patterns and architectural principles are correctly applied.
- ☐ Code has been peer-reviewed and approved.
- ☐ Final implementation meets the specified requirements.

5.8 Quiz

5.8.1 1

In Design by Contract, what is good for the supplier?
A weak postcondition, because then you have to deliver less.

5.8.2 2

Is FIFO the same as LIFO?
A stack (LIFO storage) is given in an abstract definition by the following operations.
pop: STACK -> STACK
push: STACK , ELEMENT -> STACK
top: STACK -> ELEMENT
empty: STACK -> BOOLEAN
A queue (FIFO) has a specification like this.
dequeue: QUEUE -> QUEUE

enqueue: QUEUE , ELEMENT -> QUEUE
front: QUEUE -> ELEMENT
empty: QUEUE -> BOOLEAN
Apart from renaming, the specifications are similar. Does it mean they can replace (substitute) each other?

- Yes, both can be implementations of the same abstract type.
Reasoning:
This might not intuitively make sense, however:

- An ADT defines a set of operation on data without specifying the details of how these operations are implemented. It describes what operations are available and what they should do, but not how they do it.
- Both can be viewed as different implementations of a more generalized ADT that manages a collection of elements.

5.8.3 3

In Design by Contract, what is good for the client?
A strong postcondition, because then the supplier has to deliver more.

5.8.4 4

What is true for type safety and the Liskov substitution principle? Mark all that apply.

- Type safety is an issue in language design
- Type safety ensures that the runtime type adheres to the compile time type, which is the same as the Liskov principle

Reasoning:

- Type safety means that the language enforces rules on how types are used and combined, preventing type errors and enhancing program reliability.
- Type safety ensures that objects or variables used at runtime match the types determined at compile-time. This aligns with the Liskov Substitution Principle, which states that objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program.

5.8.5 5

Why are preconditions and postconditions not checked automatically against the code?
It is undecidable.
Reasoning:

- **Computational Limitations:** In computational theory, an undecidable problem is one that cannot be solved by any algorithm for all possible cases. This means there's no general algorithm that can always determine correctness in finite time.
- **Complex Behaviors:** Software can exhibit a wide range of behaviors with different inputs and states. Predicting every possible outcome to validate rpeconditions and postconditions for all scenarios is akin to solving the Halting Problem, a classic example of an undecidable problem.
- **Halting Problem:** The Halting Problem is a fundamental problem in computer science which states that there is no general algorithm that can determine for every program and input whether the program will eventually stop running or continue to run indefinitely. This problem is undecidable, meaning it's impossible to algorithmically predict in all cases whether a given program will halt or run forever.

5.8.6 6

What is true about invariants, preconditions and postconditions?

- An invariant is considered to be in both precondition and postcondition of all routines.
- An invariant is considered to be both precondition and postcondition of all exported routines.

5.8.7 7

What change is correct in refinement?

- Strengthen the postcondition.
- Weaken the precondition.

5.8.8 8

What is characteristic for an abstract data type?

- Other programs are allowed to create variable of the type
- The representation of objects is hidden from the user
- The type's interface does not depend on the implementation
- The declaration of the type and operations are contained in a single syntactic unit.

5.8.9 9

Why are getters and setters better than direct access to the hidden data?

- Constraints can be included in setters, allowing the setter to enforce particular ranges.
- Read-only access can be provided by having a getter but no setter.
- The actual implementation of the data member can be changed without affecting clients.

5.8.10 10

Sort the following conditions according to strength.

- **strongest** - false
- **strong** - $x > y > 0$
- **medium** - $x - y >= 0$
- **weak** - $x * x >= y * y$
- **weakest** - true

1. **False (Strongest):**
   The condition **false** is the strongest because it's always false and never satisfied, ruling out all possibilities.
2. $x > y > 0$ **(Strong)**
   This condition is quite specific. It requires that x be greater than y, and y be greater than 0, which excludes many possibilities.
3. $x - y \geq 0$ **(Medium)**
   This condition states that x is greater than or equal to y. It's less restrictive than $x > y > 0$ because it also includes cases where x equals y.
4. $x^2 \geq y^2$ **(Weak)**
   This condition is weaker as it allows more possibilities. It is true when x is either greater than or equal to y or less than or equal to $-y$.

5. **True (Weakest)**
   The condition **true** is the weakest because it's always satisfied, regardless of the values of x and y.

6 Relations

6.1 Basics

- **Set A:** Well-defined collection of objects. These objects can be numbers, characters, points, or any other identifiable entities.
- **Relation R:** A relation on set A is a collection of ordered pairs of elements from A. Each pair in the relation consists of two elements where the first element is related to the second element in some way. This relation is a subset of the *Cartesian product* AxA, which consists of all possible ordered pairs of elements from A.
- **Cartesian Product A x A:** Given a set A, the *Cartesian product* A x A is defined as the set of all possible ordered pairs, where the first and second elements of each pair are from A. For example if $A = \{1, 2, 3\}$, then the Cartesian product A x A is {(1,1), (1,2), (1,3), (2,1), (2,2), (2,3), (3,1), (3,2), (3,3)}. This set includes every combination of elements from A paired with every other element, including themselves.
  If $\emptyset$ x S = $\emptyset$ for any set S.
- **Properties of cartesian products:**
  1. $A \times (B \cup C) = (A \times B) \cup (A \times C)$
  2. $(A \cup B) \times C = (A \times C) \cup (B \times C)$
  3. $A \times (B \cap C) = (A \times B) \cap (A \times C)$
  4. $(A \cap B) \times C = (A \times C) \cap (B \times C)$
  5. $A \times \emptyset = \emptyset$
  6. $\emptyset \times A = \emptyset$
  7. $A \times B = B \times A$ iff $A = B$ or $A \times B = \emptyset$
- **Relation as a Subset:** A relation R on a set A is a subset of this Cartesian product. This means that every element (which is an ordered pair) in relation R is also an element of the Cartesian product A x A, but not every element of A x A needs to be in R. The relation R selects certain pairs from A x A that satisfy a specific criterion or relationship.
- **Example of a Relation:** $A = \{1, 2, 3\}$. Suppose we define a relation R to include pairs where the first element is less than the second element. Then R woulc be {(1,2),(1,3),(2,3)}. Here, R is a subset of A x A because every pair in R is also in the Cartesian product A x A, but not all pairs in A x A are in R.
- Any subset of AxB is called a (binary) relation **FROM A to B**.
- AxA means everything is related, $\emptyset$ means nothing is related.

In conclusion, **a relation R on A** is therefore a **subset of the cartesian product of A**.

6.2 Binary relations

- **Basic Concept:** A binary relation is a collection of ordered pairs. It's called "binary" because it related elements from two sets (or the same set in some cases).
- **Mathematical Formulation:** If R is a binary relation, and a and b are two elements (where a is from set A, and b is from set B), then aRb (read as "a is related to b by R") if the pair (a,b) is in the relation R.

Types of binary relations

- **Reflexive:** Every element is related to itself. For a set A, aRa for all a in A.
- **Symmetric:** If aRb, then bRa.
- **Anti-symmetric:** If aRb and bRa, then $a = b$.

- **Transitive:** If aRb and bRc, then aRc.

These relations are often represented with symbols to express the nature of the relationship between elements of sets.
THE R IS SUBSTITUTED WITH THE SYMBOL OF THE RELATION

Example Set 1

- $\emptyset$ is the empty binary relation.
- $\mathrm{Id}(A) = \{(a, a) \mid a \in A\}$ is the identity relation on set A.
- $A \times A$ is the complete binary relation on set A.
- Relations on sets include $\subseteq, \subset, \supseteq$, and $\supset$.
- $<$ and $\leq$ are relations on integers.
- $>$ and $\geq$ are relations on integers.
- $=$ and $\neq$ are general relations. Note: $=$ is the same as Id.

Example Set 2

$++$ **Relation:** A relation on integers defined as $a++b$ iff $a = b + 1$.

True Examples: $2++1$, $1++0$, $1001++1000$, $0++-1$, $-1++-2$, $-99++-100$.

False Examples: $2++2$, $1++3$, $3++1$, $-1++1$, $100++200$, $-22++-11$.

Example Set 3

$*+$ **Relation:** A relation on integers defined as $a*+b$ iff $a \times b \geq 0$.

True Examples: $0*+0$, $1*+1$, $0*+1$, $1*+2$, $1001*+12$, $-9*+-100$, $-2*+-3$, $0*+-2$.

False Examples: $2*+-2$, $-1*+3$, $-3*+1$, $-1*+1$, $100*+-200$, $-22*+11$.

Example Set 4

$=$**%n**$=$ **Relation:** For each natural number n, a relation on integers is defined as $a = \%n = b$ iff $a\%n = b\%n$ iff $(a - b)\%n = 0$.

True Examples: $9 = \%10 = 9$, $8 = \%2 = 6$, $-1 = \%2 = -3$, $1 = \%10 = -9$, $1 = \%1 = 2$.

False Examples: $8 = \%10 = 5$, $1 = \%2 = 4$, $-1 = \%2 = -2$, $1 = \%10 = -1$, $1 = \%3 = 6$.

6.3 Properties of relations

6.3.1 Reflexivity

A relation R on A is reflexive if

$$(a, a) \in Rforalla \in A$$

Definition in words: for every element a in the set A, the ordered pair (a, a) is in the relation R. This means that each element in the set A is related to itself within the context of the relation.

Interpretation of Reflexivity: In a reflexive relation, there is a kind of 'self-loop'. Every element must be connected to itself.

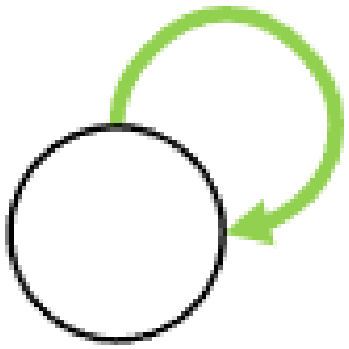

Figure 21: Self-loop

Subset of the Cartesian Product: Given that R is a subset of the Cartesian product A x A, for R to be reflexive, it must contain all pairs of the form (a, a) for each a in A. However, R can also include other pairs where the elements are not the same, as long as it includes all the self-pairs.

Trivialization: In a real world context, consider a relation "is a sibling of". For this relation to be reflexive in a family set, it would mean that every person is considered a sibling of themselves, which is typically not the case. So, this relation would not be reflexive.

In summary, a reflexive relation on a set A requires that every element in A is related to itself. It's about the presence of these 'self-pairings' in the relation.

6.3.2 Symmetry

A relation R on A is symmetric if

$$(a, b) \in Rimplies(b, a) \in Rforalla, b \in A$$

Defined in words: A relation R on a set A is symmetric if, whenever a pair (a, b) is in R, the reverse pair (b, a) is also in R. This means that the relation works both ways; if a is related to b, then b must also be related to a.

Understanding Symmetry in Relations:

- If you think of a relation as a kind of connection or link between elements, symmetry implies that this link is bidirectional.
- For example, if the relation R represents "is a friend of", then the symmetry means that if person a is a friend of person b, then person b must also be a friend of person a.

Symmetry in the Context of the Cartesian Product: In the Cartesian product A x A, which includes all possible ordered pairs from A, a symmetric relation R will have the property that whenever (a, b) is in R, (b, a) will also be in R. However, it's important to note that not every pair A x A needs to be in R for R to be symmetric; symmetry only concerns pairs that are actually in R.

Examples:
If

$$A = \{1, 2, 3$$

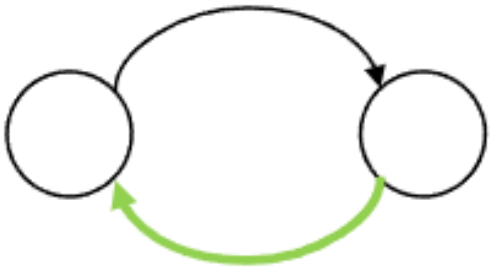

Figure 22: Enter Caption

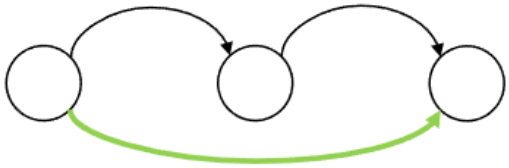

Figure 23: Enter Caption

and

$$R = \{(1,2),(2,1),(2,3)(3,2)\}$$

then R is symmetric, because for every pair (a,b) in R, the pair (b,a) is also in R.

6.3.3 Transitivity

A relation R on A is transitive if

$$(a,b),(b,c) \in Rimplies(a,c) \in Rforalla,b,c \in A$$

Defined in words:
A relation R on a set A is transitive if, whenever two pairs (a,b) and (b,c) are in R, the pair (a,c) is also in R. In other words, if a is related to b, then b is related to c. then transitivity dictates that a must also be related to c.

Understanding Transitivity in Relations:

- Transitivity can be thought of as a way of establishing a chain of relationships. If one link of the chain connects a to b, and another link connects b to c, then there must be a direct link connecting a to c.

Examples:
If

$$A = \{1,2,3\}R = \{(1,2),(2,3),(1,3)\}$$

then R is transitive because the presence of $(1,2)$ and $(2,3)$ in R implies the presence of $(1,3)$

In essence, the friend of my friend is my friend.

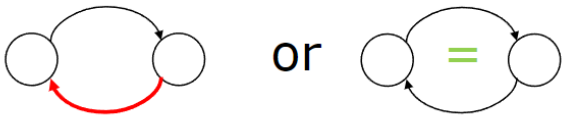

Figure 24: Enter Caption

6.3.4 Antisymmetry

A relation R on A is antisymmetric if

$$(a,b),(b,a) \in Rimpliesa = bforalla, b \in A$$

Defined in words:
A relation R on a set A is antisymmetric if, whenever both pairs (a,b) and (b,a) are in R, it must be the case that $a = b$. In other words, the only way both (a,b) and (b,a) can coexist in R is if they are essentially the same element, i.e., a and b are identical.

Understanding Antisymmetry in Relations:

- Antisymmetry is about restricting the bidirectionality that is allowed in symmetry. If there is a two-way relationship between different elements, it violates antisymmetry.
- This property is often seen in order and hierarchical relationships. For example, in a "less than or equal too" relation, the relation is antisymmetric, because if $a \leq b$ and $b \leq a$, then a must be equal to b.

Antisymmetry in the Context of the Cartesian Product: In the Cartesian product A x A, a relation R being antisymmetric means that if (a,b) and (b,a) are both in R for some a and b, then a and b must be the same element. It is perfectly fine for R to have pairs like (a,a), but it cannot have (a,b) and (b,a) with $a \neq b$

Example:
Consider the set

$$A = \{1,2,3\} R = \{(1,1),(2,2),(1,2),(2,3)\}$$

The relation is antisymmetric because it does not contain any pair (a,b) and (b,a) where $a \neq b$. In contrast, if R included both $(1,2)$ and $(2,1)$, it would not be antisymmtric unless $1 = 2$, which is not true.

6.3.5 Overview of operators and other stuff

Rel	Property	Reasoning
$\leq$	Reflexive	True. For any element a, $a \leq a$ is always true.
	Symmetric	False. If $a \leq b$, it does not imply $b \leq a$.
	Transitive	True. If $a \leq b$ and $b \leq c$, then $a \leq c$.
	Antisymmetric	True. If $a \leq b$ and $b \leq a$, then $a = b$.
$=$	Reflexive	True. For any element a, $a = a$ is always true.
	Symmetric	True. If $a = b$, then $b = a$.
	Transitive	True. If $a = b$ and $b = c$, then $a = c$.
	Antisymmetric	True. Redundant for equality, but if $a = b$ and $b = a$, then $a = b$.
$\geq$	Reflexive	True. For any element a, $a \geq a$ is always true.
	Symmetric	False. If $a \geq b$, it does not imply $b \geq a$.
	Transitive	True. If $a \geq b$ and $b \geq c$, then $a \geq c$.
	Antisymmetric	True. If $a \geq b$ and $b \geq a$, then $a = b$.
$\subseteq$	Reflexive	True. A set is always a subset of itself.
	Symmetric	False. If $A \subseteq B$, it does not imply $B \subseteq A$.

	Transitive	True. If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.
	Antisymmetric	True. If $A \subseteq B$ and $B \subseteq A$, then $A = B$.
$\supseteq$	Reflexive	True. A set is always a superset of itself.
	Symmetric	False. If $A \supseteq B$, it does not imply $B \supseteq A$.
	Transitive	True. If $A \supseteq B$ and $B \supseteq C$, then $A \supseteq C$.
	Antisymmetric	True. If $A \supseteq B$ and $B \supseteq A$, then $A = B$.
Rel	**Property**	**Reasoning**
$<$	Reflexive	False. For any element a, $a < a$ is never true.
	Symmetric	False. If $a < b$, then $b < a$ is not true.
	Transitive	True. If $a < b$ and $b < c$, then $a < c$.
	Antisymmetric	False. $a < b$ and $b < a$ cannot both be true.
$>$	Reflexive	False. For any element a, $a > a$ is never true.
	Symmetric	False. If $a > b$, then $b > a$ is not true.
	Transitive	True. If $a > b$ and $b > c$, then $a > c$.
	Antisymmetric	False. $a > b$ and $b > a$ cannot both be true.
$\subset$	Reflexive	False. A set is not a proper subset of itself.
	Symmetric	False. If $A \subset B$, it does not imply $B \subset A$.
	Transitive	True. If $A \subset B$ and $B \subset C$, then $A \subset C$.
	Antisymmetric	False. $A \subset B$ and $B \subset A$ cannot both be true.
$\supset$	Reflexive	False. A set is not a proper superset of itself.
	Symmetric	False. If $A \supset B$, it does not imply $B \supset A$.
	Transitive	True. If $A \supset B$ and $B \supset C$, then $A \supset C$.
	Antisymmetric	False. $A \supset B$ and $B \supset A$ cannot both be true.
$\neq$	Reflexive	False. For any element a, $a \neq a$ is never true.
	Symmetric	True. If $a \neq b$, then $b \neq a$.
	Transitive	False. If $a \neq b$ and $b \neq c$, it doesn't necessarily mean $a \neq c$.
	Antisymmetric	Not applicable. Inequality doesn't establish a kind of order or hierarchy.
++ (Placeholder)	Reflexive	false
	Symmetric	false
	Transitive	false
	Antisymmetric	true
$*+$	Reflexive	true
	Symmetric	true
	Transitive	false
	Antisymmetric	false
$\emptyset$	Reflexive	Not applicable, as $\emptyset$ is a set, not a relation.
	Symmetric	Not applicable, as $\emptyset$ is a set, not a relation.
	Transitive	Not applicable, as $\emptyset$ is a set, not a relation.
	Antisymmetric	Not applicable, as $\emptyset$ is a set, not a relation.
$A \times A$ (Cartesian Product)	Reflexive	Not directly applicable, as $A \times A$ is a set of ordered pairs, not a relation itself.
	Symmetric	Not directly applicable, as $A \times A$ is a set of ordered pairs, not a relation itself.
	Transitive	Not directly applicable, as $A \times A$ is a set of ordered pairs, not a relation itself.
	Antisymmetric	Not directly applicable, as $A \times A$ is a set of ordered pairs, not a relation itself.

6.4 Relation matrix

A **relation matrix** is a way of representing a relation using a matrix.

- **Representation:** In a relation matrix for a relation R on a set A, the rows and columns represent the elements of A. The matrix is typically a square matric, and its size is determined by the number of elements in A.
- **Entities in Matrix:** Each entry in the matrix corresponds to a pair of elements from A. If the

pair (a_i, a_j) is in the relation R, the entry in the i-th row and the j-th column is 1 (or true). Otherwise it's 0 (or false)

- **Translation to Set or Formula:** To translate a relation matrix back to a set or a formula, you interpret the 1s and 0s in the matrix as indicating the presence or absence of the corresponding ordered pairs in the relation.

6.5 Equivalence Relation and Partitions

An equivalence relation R on a set A is a relation that is **reflexive**, **symmetric**, and **transitive**. Equivalence relations can partition a set into disjoint subsets.

- **Partitioning a Set:** A partition of a set A is a collection of non-empty, disjoint subsets of A whose union is A. Each element of A belongs to exactly one of these subsets.
- **Equivalence Classes:** For an equivalence relation R on A, each element a in A belongs to an equivalence class, which is the set of elements in A that are related to a by R. The set of all such equivalence classes forms a partition of A

6.6 Equivalence Classes

Imagine you have a bunch of objects, and you want to group them based on some rule. This rule is an *equivalence relation*. It's a way of saying that objects in each group share something in common. The main rules for this relation are:

1. **Reflexive**: Everything is similar to itself.
2. **Symmetric**: If one object is similar to another, the second one is similar to the first one.
3. **Transitive**: If one object is similar to a second, and that second is similar to a third, then the first and third objects are also similar.

Now, an **equivalence class** is a group formed by this rule. In each equivalence class, every object is considered similar based on the rule.

Example

Consider your objects are numbers, and your rule is *"numbers that leave the same remainder when divided by 5"*.

- Numbers like 1, 6, 11, 16, etc., form one equivalence class because they all leave a remainder of 1 when divided by 5.
- Numbers like 2, 7, 12, 17, etc., form another equivalence class for leaving a remainder of 2.

Each group is an equivalence class, where all numbers follow the same rule.

Key Points

- Every object belongs to exactly one equivalence class for a given rule.
- Equivalence classes are either completely disjoint or identical.
- All equivalence classes together cover every possible object without overlap or duplication.

In simple terms, equivalence classes are like buckets where you sort objects based on a specific rule, with each object fitting into exactly one bucket.

6.7 Orders and Extending Relations

- **Order Relations:** An order relation is a relation that is **reflexive**, **antisymmetric**, and **transitive**. The most common examples are partial orders and total orders.
- **Extending a Relation:** To extend a relation R to become an order or an equivalence, you would add pairs to R to ensure it satisfies the required properties (reflexivity, antisymmetry, transitivity **for order**; reflexivity, symmetry, transitivity for equivalence).

- **Total and Partial Orders:** A total order (or linear order) is an order relation in which every pair of distinct elements is comparable. A partial order allows for some pairs of distinct elements to be incomparable.

6.8 Total order

A total order is a type of binary relation that is used to compare elements within a set. It's a more specific form of a partial order.
What is Total Order?

- **Binary relation:** A total order is a binary relation, denoted typically by $\leq$ on a set S.
- **Properties:** For a relation to be a total order, it must satisfy certain properties for all elements, a, b and c in S:
  - **Reflexivity** $a \leq a$
  - **Anti-symmetry:** if $a \leq b$ and $b \leq a$ then $a = b$ If two elements are mutually related, they are actually the same.
  - **Transitivity:** If $a \leq b$ and $b \leq c$, then $a \leq c$
  - **Totality:** For any two elements a and b in S, either $a \leq b$ or $b \leq a$ must hold. This is what distinguishes a total order from a partial order.

Comparison with Partial Order

- **Partial Order** is a binary relation that is reflexive, antisymmetric, and transitive. However, it does not require the property of totality. In partially ordered sets, not very pair must be comparable.
  Total vs Partial Order:
  - **Comparability:** In a total order, every pair of elements is comparable. This is not the case with partial orders, where some elements may not be related.
  - **Examples:**
    - **Total Order:** The usual ordering of numbers ($\leq$ on integers or real numbers) is a total order. Every pair of numbers can be compared.
    - **Partial Order:** The subset relation among sets is a partial order. Some sets are not subsets of others, and hence they are not comparable.

6.9 Negative numbers: =%=

=%= is an equivalence relation and induces integer partitions. Negative numbers are the partitions of $= \%_{16} =$

- **Equivalence Relation (=%16=):** Two integers a and b are considered equivalent in this relation if and only if they have the same remainder when divided by 16. In formal terms, $a = \%16 = b$ if and only if $(a - b)\%16 = 0$. This means that for any integer a, the equivalence class $[a]$ under $= \%16 =$ contains all integers x such that $x\%16 = a\%16$.
- **Negative Numbers as Partitions:** The statement "Negative numbers are the partitions of $= \%16 =$" refers to the fact that each negative integer, when considered in the context of this relation, forms a distinct equivalence class, a set of numbers that are all equivalent to each other under $= \%16 =$.

0 = {-32, -16, 0, 16, 32}

-1 = {-17, -1, 15, 31}

-2 = {-18, -2, 14, 30}

-3 = {-19, -3, 13, 29}

-4 = {-20, -4, 12, 28}

-5 = {-21, -5, 11, 27}

-6 = {-22, -6, 10, 26}

-7 = {-23, -7, 9, 25}

-8 = {-24, -8, 8, 24}

-9 = {-25, -9, 7, 23}

-10 = {-26, -10, 6, 22}

-11 = {-27, -11, 5, 21}

-12 = {-28, -12, 4, 20}

-13 = {-29, -13, 3, 19}

-14 = {-30, -14, 2, 18}

-15 = {-31, -15, 1, 17}

6.10 Quiz

6.10.1 1

Let P be a relation on $\{a, b, c, d\}$ with $P = \{(a, a), (a, b), (a, c), (b, c)\}$. Which elements do we need to add to P at least to make it reflexive?

(b, b), (c, c) and (d, d). **The latter needs to be included because it is for EVERY $x \in S$.**

6.10.2 2

Let P be a relation on $\{a, b, c, d\}$ with $P = \{(a, a), (a, b), (a, c), (b, c)\}$. Which elements do we need to add to P at least to make it symmetric?
$(b, a), (c, a), (c, b)$

6.10.3 3

Let P be a relation on $\{a, b, c, d\}$ with $P = \{(a, a), (a, b), (a, c), (b, c), (c, b)\}$. Which elements do we need to add to P at least to make it transitive?
This is an interesting one. Because we have (b, c) and (c, b) we also need to add (c, c) and (b, b). REMEMBER: ONLY IF YOU HAVE THE TWO FIRST JOINTS. DO NOT CONSTRUCT FROM JOINTS ONE AND THREE

6.10.4 4

Let P be a relation on $\{a, b, c\}$ with $P = \{(a, a), (a, b), (a, c), (b, c)\}$. Which elements can we add to P to keep it anti-symmetric?
The only ones that can be added, are the ones who will not cause symmetry (c, c) and (b, b)

6.10.5 5

Let P be a relation on $\{a, b, c, d\}$ with $P = \{(a, a), (a, b), (a, c), (b, c)\}$. Which elements do we need to add to P at least to make it an equivalence relation?
Equivalence relations are **reflexive**, **symmetric** and **transitive**. Add the corresponding ones. (c,c), (c,b), (b,b), (b,a), (c,a) and (d,d).

6.10.6 6

Let P be a relation on $\{a, b, c, d\}$ with $P = \{(a, a), (a, b), (d, a), (b, c)\}$. Which elements do we need to add to P to make it a partial order?
Partial orders are **reflexive**, **antisymmetric**, and **transitive**.

- Add for reflexivity: (b,b), (c,c), (d,d)

- Add for transitive:
  - We have (a,b) and (b,c), therefore, add (a,c)
  - We have (d,a) and (a,b), therefore, add (d,b)
  - We have (d,a) and (a,c), therefore, add (d,c

Add the corresponding ones. (b,b), (c,c), (a,c), (d,d)

6.10.7 7

Let P be a partial order on $\{a, b, c, d\}$ with $P = \{(a, a), (b, b), (c, c), (d, d), (a, b), (c, d)\}$. Which elements are maximal in P?

In a partial order, an element is considered maximal if there is no other element in the set that is strictly greater than it within the context of the partial order. To find the maximal elements in the partial order, we need to look for elements that are not 'less than' any other elements in the set according to the order.

- **a** is related to **b** in (a,b), which means **a** is less than **b** in the order. a is not maximal.
- **b**: There is no pair in P, where (b,x), where x is different from b. Therefore, no element is strictly greater than b in P.
- **c** same logic as with a, but with d.
- **d** same logic as with b, but with c.

Therefore, the answer is b and d.

6.10.8 8

Let D be the relation on sets such that xDy is true if no elements of x appear in y. Which properties are true for D?

Symmetry? Really?

6.10.9 9

Consider the relation =%10=. How many sets are in its partition of the integers?
The relation "=%10=" is an equivalence relation on the set of integers where two integers a and b are related if a%10=b%10 This means two integers are related if they have the same remainder when divided by 10.

Since there are only 10 possible remainders when dividing by 10 (specifically, 0 through 9), this relation partitions the set of integers into 10 equivalence classes. Each class consists of integers that share the same remainder when divided by 10.

6.10.10 10

Let O be a total order of 8 elements. How many pairs are in O?
$\binom{8+1}{2}$.
So the formula is

$$\binom{n+1}{2}$$

This means order in the pairs is UNimportant, and repetitions are allowed

7 Recurrence relations

7.1 Introduction

A recurrence relation is a way of defining a sequence of numbers where each number in the sequence is determined by a formula involving its predecessors.

7.1.1 Key concepts

- **Predecessor:** These are the numbers in the sequence that come before the current number.
- **Order of the Relation:** This refers to how many predecessors are used to calculate the next number. If a sequence has two predecessors (like the Fibonacci sequence), it's a second-order relation.
- **Initial Conditions:** These are the starting values of the sequence. They are essential because without them, you can't start building the sequence. A set of initial values and the recurrence relation together determine the entire sequence.

7.1.2 Examples:

- **Fibonacci Sequence:** This is a classic example of a second-order recurrence relation.
- **Factorial Sequence:** This is an example of a first-order recurrence relation. Each number is the product of the preceeding number and its position in the sequence. The initial condition is 1. $a_n = n \cdot a_{n-1}$ is the recurrence relation for $a_n = n!$
- **Geometric Progression:** This is a sequence where each term is a constant multipe of the previous term (2, 6, 18, 54).

7.1.3 Exercise 2.3

Show by induction that solutions to recurrence relations with given initial value are unique. In other words, if $a_0, a_1, \ldots$ and $b_0, b_1, \ldots$ are two sequences which both solve a given recurrence relation (and its initial condition), show that $a_n = b_n$ for all $n \geq 0$.

Important for understanding how sequences are formed, and how they evolve.
A recurrence relation is essentially a mathematical expression that describes a sequence in which each term is defined in terms of its preceding terms, known as its predecessors.
By understanding recurrence relations, one can often predict or calculate subsequent terms in a sequence without having to enumerate each term explicitly.
Fibonacci is an example of recurrence relation.

Defining a recurrence relation is not sufficient to fully describe a sequence. One must also provide initial conditions, which are the starting terms of the sequence. For instance, the Fibonacci sequence is defined along with the initial conditions that the first two terms are 1.

7.2 Initial value problem

A **recurrence relation** together **with** an **initial condition** is often called an *initial value problem.*

7.3 Non-homogeneous linear relations of order 2

7.4 Solving recurrence relations

ORDER: The order of a recurrence relation is determined by the number of previous terms it references.

Initial conditions: Initial conditions are necessary to uniquely determine the sequence. They specify the first few terms of the sequence, allowing the rest of the terms to be calculated using the recurrence relations.

7.4.1 Constant coefficients

Definition: A recurrence relations has constant coefficients if the coefficients (the multipliers of the terms) are constant, i.e., they do not depend on the position n of the term in the sequence. For example $a_n = 3a_{n-1} + 5a_{n-2}$ both 3 and 5 are constant coefficients.

Opposite: Variable coefficients. In this case, the coefficients depend on n. An example would be $a_n = na_{n-1} + (n-1)a_{n-1}$, where the coefficients n and $n-1$ vary with each term.

7.4.2 Homogeneous:

Definition: A homogeneous recurrence relation is one in which every term is a lienar combination of previous terms without any additional constant or independent term. For instance $a_n = 4a_{n-1} - 2an - 2$ is homogeneous since there are no terms outside the recursive pattern.

Opposite: Non-homogeneous or inhomogeneous. These relations include additional terms that are not part of the recursive sequence. For example $a_n = 4a_{n-1} - 2a_{n-2} + 3^n$ is non-homogeneous due to the presence of the 3^n term.

7.4.3 Linear:

A linear recurrence relation is one where each term is the sum of previous terms, each multiplied by a constant coefficient, and potentially plus a constant (in the case of non-homogeneous relations). The relation is linear because each term is a linear function of the previous terms. An example is $a_n = 6a_{n-1} + 3a_{n-2}$.

Opposite: Non-linear. In a non-linear recurrence relation, terms are related in a way that is not a simple linear combination. This can include relations where terms are multiplied or divided by each other, raised to powers, or involve more complex functions. For example $a_n = (a_{n-1})^2 + 3a_{n-2}$ is non-linear because of the squared term.

7.5 Solving recurrence relations

A recurrence relation combined with a starting point (initial condition), forms what we call an **Initial value problem**.
A specific formula that directly calculates any term in the sequence is known as a **particular solution** to this problem.
If we find a formula that solves such problems for a specific recurrence relation, we call it a **general solution**.

7.5.1 Geometric Progressions:

Example: In a sequence where each term is a multiple of the previous term (common ratio **r**), and starting with a_0, the formula for any term is $a_n = a_0 \cdot r^n$. This formula is the general solution for any geometric progression with a common ratio r.

Specific Case: For a sequence starting 2 $a_0 = 2$ and a common ratio of 3 $r = 3$ the formula becomes $a_n = 2 \cdot 3^n$

7.5.2 Inspection Method:

Sometimes, we can spot a pattern by looking at the first few terms of a sequence. We can then guess the solution and use induction (a proof method) to confirm it.
Example: For the recurrence relation $a_n = (n-2)/n * a_{n-1} + \frac{2}{n} starting with a_1 = 1$, we observe that the sequence seems to be 1, 1, 1... for each term we calculate. We guess that an = 1 for all n and prove it using induction.

7.5.3 Generating functions

A sequence can be represented as an infinite sum, known as its generating function. For example, for a sequence s, its generating function is $f(x) = \Sigma$ (from n=0 to infinity) of $a_n \cdot x^n$.
This approach is particularly useful in finding non-recursive formulas for sequence terms.

Fibonacci Sequence Example: Using algebraic manipulation, we find that the generating function for the Fibonacci sequence is $f(x) = 1/(1-x-x^2)$. This compact form leads us to a more convenient expression for each Fibonacci number. By further analysis, involving roots of a polynomial and partial fractions, we derive an explicit formula for the Fibonacci numbers: $Fn = 1/\sqrt{5} * ((1+\sqrt{5}/2)^(n+1) - (1-\sqrt{5}/2)^(n+1))$

7.5.4 Homogeneous Linear Recurrence Relations with Constant Coefficients

PS: IF THE FIRST TERM (a_n) HAS A CONSTANT $\neq 1$, DIVIDE THE ENTIRE EXPRESSION BY THIS CONSTANT
PS2: IF THERE IS A CONSTANT AT THE END WITHOUT ANYTHING TO DO WITH a_{n-k}, IT IS NOT HOMOGENEOUS. Step 1: Formulate the Characteristic Equation:

1. Replace a_n with r^n (where r is a constant we're trying to find). For example, turn $a_n = c_1 a_{n-1} + c_2 an - 2$ into $r^n = c_1 r^{n-1} + c_2 r^{n-2}$.
2. Rearrange this into a standard polynomial form: $r^n - c_1 r^{n-1} - c_2 r^{n-2} = 0$. Notice that the right hand terms are switched to the left side, and therefore the operator is '-'.
3. This becomes your characteristic equation.

Step 2: Find the Roots of the Characteristic Equation Solve the equation for r. The roots may be real or complex, and they might be distinct or repeated. Use the formula

$$\frac{-b + -\sqrt{b^2 - 4ac}}{2a}$$

Remember that $-b+$ and $-b-$ provides two different roots.

Step 3: Construct the General Solution: Assuming you find two distinct roots r_1 and r_2 from the characteristic equation:

- If the roots are different, the general solution is a combination these roots raised to the nth power, like: $a_n = A \cdot 2^n + B \cdot 3^n$, where A and B are constants we need to determine.
- If the roots are the same (a repeated root), then the solution would include a term multiplied by n: $a_n = A \cdot 2^n + B \cdot n \cdot 2^n$
- If the roots are complex, the solution will take the form $a_n = r^n(A\cos(n\theta) + B\sin(n\theta))$ where $r = \sqrt{a^2 + b^2}$ and $\theta = \arctan\left(\frac{b}{a}\right)$.

Step 4: Applying Initial Conditions

Create two separate equations
First, if we set $a_0 = x$, then $x = A \cdot r_1^0 + B \cdot r_1^0$. Notice that the n is swapped with 0, as we have a_0.
Next, if we set $a_1 = y$, then $y = A \cdot r_2^1 + B \cdot r_2^1$.
Solve the two.

Example: $a_n = 5a_{n-1} - 6a_{n-2}$ with initial conditions $a_0 = 1$ and $a_1 = 7$.

1. Formulate the characteristic equation: $r^2 - 5r + 6 = 0$
2. Find the roots, in this case $(r-2)(r-3)$, so the roots are 2 and 3.
3. Construct the general solution $a_n = A \cdot 2^n + B \cdot 3^n$
4. Apply initial conditions: Use $a_0 = 1$ and $a_1 = 7$ to find A and B.
5. As $a_0 = 1$, $a_0 = A \cdot 2^0 + B \cdot 3^0$. The equation becomes $a_0 = 1 = A + B$
   As $a_1 = 7$, $a_1 = A \cdot 2^1 + B \cdot 3^1$. The equation becomes $a_1 = 7 = 2A + 3B$
6. Now, we can use the substitution method to solve this:

$$|A = 1 - B||7 = 2(1 - B) + 3B||7 = 2 - 2B + 3B||5 = B||B = 5|A = 1 - 5|A = -4$$

Now, write the final solution with A and B, so that

$$a_n = -4 \cdot 2^n + 5 \cdot 3^n$$

7.5.5 Non-homogeneous Linear Recurrence Relations

Step 1. Identify the Non-Homogeneous Relation
Typically looks like $a_n = c_1a_{n-1} + c_2a_{n-2} + ... + c_ka_{n-k} + f(n)$, where $f(n)$ is a non-zero function that makes the relation non-homogeneous.

Step 2: Solve the Corresponding Homogeneous Relation
First, solve the corresponding homogeneous part of the relation $a_n = c_1a_{n-1} + c_2a_{n-2} + ... + c_ka_{n-k}$ by using the method for homogeneous relations.

Step 3: Find a Particular Solution
A particular solution is a specific solution to the non-homogeneous equation. Its form depends on the nature of $f(n)$. **Common methods for finding a particular solution:**

- **Method of Undetermined Coefficients:** Guess a form similar to $f(n)$ and solve for the coefficients.
- **Method of Annihilators:** Apply a differential or difference operator that makes $f(n)$ zero, then solve the resulting homogeneous equation.

Step 4: Form the General Solution
The general solution of a non-homogeneous linear recurrence relation is the sum of the general solution of the homogeneous part and the particular solution.

Step 5: Apply Initial Conditions

- Use given initial conditions to find the specific constants in the solution.
- Substitute these initial conditions into the general solution and solve the resulting system of equations.

Example:
Suppose you have a non-homogeneous linear recurrence relation $a_n = 3a_{n-1} - 2a_{n-2} + 2^n$, with the initial conditions $a_0 = 1$ and $a_1 = 5$.
Step 1: Solve the Homogeneous Part:

- $a_n = 3a_{n-1} - 2a_{n-2}$
- The characteristic equation is $r^2 - 3r + 2 = 0$
- Solving this, we get the roots $r_1 = 1$ and $r_2 = 2$
- The general solution to the homogeneous part is $a_n = A \cdot 1^n + B \cdot 2^n$ or $a_n = A \cdot + B \cdot 2^n$, as 1^n is 1 for all n.

Step 2: Find a Particular Solution:

- Since the non-homogeneous term is 2^n, let's guess a particular solution of the form $a_n = C \cdot 2^n$
  PS: THIS MIGHT NOT WORK. IF SO, THEN TRY $C \cdot n \cdot 2^n$
- Substitute a_n into the non-homogeneous relation: $C \cdot 2^n = 3C \cdot 2^{n-1} - 2C \cdot 2^{n-2} + 2^n$
- Simplify and solve for C.
  $C = \frac{1}{4}$, since all the terms involving 2^n will cancel out except for the 2^n on the right hand side.

Step 3: Form the general solution
The general solution is the sum of the homogeneous and particular solutions:

- $a_n = A + B \cdot 2^n + \frac{1}{4} \cdot 2^n$
- Simplify to $a_n = A + (B + \frac{1}{4}) \cdot 2^n$

Step 4: Apply Initial Conditions
Now apply the initial conditions to solve for A and B

- For $a_0 = 1$: $1 = A + B + \frac{1}{4}$

- For $a_1 = 5$: $5 = A + 2B + \frac{1}{4}$

Step 5: Solve for A and B
Find values for A and B. In this example $A = -2$ and $B = \frac{11}{4}$.

Step 6: Write the Final Solution
Substitute A and B back: $a_n = -2 + (\frac{11}{4} + \frac{1}{4}) \cdot 2^n$.
Simplify to $a_n = -2 + 3 \cdot 2^n$

7.5.6 Recurrence Relations with Variable Coefficients

Understanding the Relation
A recurrence relation with variable coefficients typically looks like
$a_n = f(n) \cdot a_{n-1} + g(n) \cdot a_{n-2} + ...$
where $f(n)$, $g(n)$ are functions of n.

Example
Consider a simple variable coefficient recurrence relation:
$a_n = n \cdot a_{n-1}$
with an initial condition $a_0 = 1$

Step 1: Direct computation for Few Terms

- $a_1 = 1 \cdot a_0 = 1$
- $a_2 = 2 \cdot a_1 = 2$
- $a_3 = 3 \cdot a_2 = 6$
- $a_4 = 4 \cdot a_3 = 24$

Step 2: Pattern Recognition:
Notice that $a_n = n!$

Verification by Induction:
You can prove that $a_n = n!$ using induction.

1. **Base Case: For $n = 0$, $a_0 = 1$, which is true.**
2. **Inductive Step:** Assume $a_k = k!$ is true. Show it for a_{k+1}
   $a_{k+1} = (k+1) \cdot a_k = (k+1) \cdot k! = (k+1)!$, which is the definition of factorial

7.5.7 Relationship Between Sequences and Power Series

??

7.5.8 Differential Equations

7.5.9 Characteristic Equation

7.5.10 General solution

7.5.11 Applying Initial Conditions

7.6 EXAM Questions

7.6.1 Solve given equations

7.6.2 Prove that a given sequence is a solution to a given equation.

7.7 Quiz

Question 1

Choose the recurrence relation satisfied by the sequence $a_n = 2^n + 1$.
Group of answer choices:

- $a_n = 2a_{n-1} + 1$
- $a_n = a_{n-1} + a_{n-2}$
- $a_n = 2^{n-1}$
- $a_n = 2^n + 1$
- $a_n = 2a_{n-1} - 1$✓

Set up the sequences, and test them against each other.

Question 2

Choose the recurrence relation satisfied by the sequence $a_n = \left(1 + \sqrt[3]{4}\right)^n + \left(1 - \sqrt[3]{4}\right)^n$.
Group of answer choices:

- $a_n = \frac{1}{2}a_{n-1} + 2a_{n-2}$✓
- $a_n = \frac{1}{2}a_{n-1} - 2a_{n-2}$
- $a_n = 2a_{n-1} - \frac{1}{2}a_{n-2}$
- $a_n = 2a_{n-1} + \frac{1}{2}a_{n-2}$

Question 3

Given a sequence which starts out with the following numbers, written in binary form:
1
10
100
1000
10000
100000
...
Which of the following recurrence relations describes this sequence?
Group of answer choices:

- $a_n = 2^n a_{n-1}$
- $a_n = \frac{1}{2}na_{n-1}$
- $a_n = \frac{1}{2}a_{n-1}$
- $a_n = 2 \cdot a_{n-1}$✓

By the properties of the binary numbers, we have the numbers 1, 2, 4, 8, 16, 32.

Question 4

Find the solution to the recurrence relation $a_n = 7a_{n-1} - 10a_{n-2}$ with initial condition $a_0 = 2, a_1 = 3$.
Group of answer choices:

- $a_n = \left(\frac{7}{3}\right)^n - \left(\frac{1}{3}\right)^n$
- $a_n = 7 \cdot 2^n - 10 \cdot 5^n$
- $a_n = \frac{7}{3}2^n - \frac{1}{3}5^n$ ✓
- $a_n = 7^n \cos\left(n \cdot \frac{\pi}{3}\right) + 7^n \sin\left(n \cdot \frac{\pi}{3}\right)$

Regular homogeneous linear stuff

Question 5

Assume that a_n for $n \geq 0$ is the solution to a homogeneous 2. order linear recurrence relation with constant coefficients, which starts out like
-3, 2, -1, 1, 0, 1, 1, 2, 3,...
Choose the correct value for a_{25}:
10946
Just perform the damned calculations from 0 - 25.

Question 6

Find the solution to the recurrence relation $a_n = 3a_{n-1} + 4a_{n-2}$ with initial terms $a_0 = 5$ and $a_1 = 8$.
Group of answer choices:

- $a_n = 4^n - (-1)^n$
- $a_n = 4^n - 2^n$
- $a_n = \frac{5}{13}4^n - \frac{5}{12}(-1)^n$
- $a_n = \frac{13}{5}4^n + \frac{12}{5}(-1)^n$

Homogeneous stuff again.

Question 7

Find the solution to the recurrence relation $a_n = 2a_{n-1} - a_{n-2}$ with initial terms $a_0 = 1$ and $a_1 = 2$, and use it to compute a_{1000}.
1001.
Single root

Question 8

Find a_{120} of the number sequence which satisfies the recurrence relation $a_n = a_{n-1} - a_{n-2}$ and has initial term $a_0 = 1$.
(We do not need both initial terms to solve this problem.) 1.
Don't know

Question 9

Find the solution to the recurrence relation $a_n = 2a_{n-1} + 2^n$ with initial terms $a_0 = 0$ and use it to compute a_{17}. 2,228,224

Given the recurrence relation

$$a_n = 2a_{n-1} + 2^n$$

with the initial condition

$$a_0 = 0,$$

we want to find the general form of a_n and compute a_{17}.

Step 1: Homogeneous Part

First, consider the homogeneous part of the recurrence relation:

$$a_n = 2a_{n-1}.$$

The solution to this is a geometric sequence:

$$a_n = C \cdot 2^n,$$

where C is a constant.

Step 2: Non-Homogeneous Part

For the non-homogeneous part, we try a solution of the form

$$a_n = D \cdot n \cdot 2^n.$$

Substitute this into the original recurrence relation:

$$D \cdot n \cdot 2^n = 2 \cdot D \cdot (n-1) \cdot 2^{n-1} + 2^n.$$

Simplify the equation:

$$D \cdot n \cdot 2^n = D \cdot (n-1) \cdot 2^n + 2^n.$$

$$D \cdot n \cdot 2^n = D \cdot n \cdot 2^n - D \cdot 2^n + 2^n.$$

$$0 = -D \cdot 2^n + 2^n.$$

Solving for D gives

$$D = 1.$$

General Solution

The general solution is the sum of the homogeneous and particular solutions:

$$a_n = C \cdot 2^n + n \cdot 2^n.$$

Using the initial condition $a_0 = 0$:

$$0 = C \cdot 2^0 + 0 \cdot 2^0.$$

$$C = 0.$$

Thus, the solution simplifies to

$$a_n = n \cdot 2^n.$$

Computing a_{17}

Finally, to compute a_{17}:

$$a_{17} = 17 \cdot 2^{17} = 2228224.$$

8 Introduction to group theory

A group is a mathematical structure consisting of a set G along with an operation * that combines any two elements of the set to form a third element. To qualify as a group, the set and operation must satisfy four key properties:

1. **Closure:** For any two elements a and b in G, the result of the operation $a * b$ is also in G.
2. **Associativity:** The operation is associative if, for any a, b and c in G, $(a * b) * c = a * (b * c)$
3. **Identity element:** There exists an element e in G (known as the identity/neutral element) such that for every element a in G, $a * e = a = e * a$
4. **Inverse element:** For every element a in G, there exists an element b in G such that $a * b = e$, where e is the identity element.

When we say closed under the operation, this operation is the operation defined in the group. For example G = $(\mathbb{Z}, +)$ is closed under addition, meaning any results involving from adding two arbitrary integers, must also be in the group (**closure**). This operation further dictates the key properties defined above, so for addition:

- **Associativity:** Addition of integers is associative, $(a + b) + c = a + (b + c)$
- **Identity Element:** The identity element for addition is 0, since a + 0 = a = 0 + a for an integer a.
- **Inverse Element:** The inverse of any integer a under addition is its negation -a, since a + (-a) = 0.

8.1 Examples of groups

- **Integers under addition $(\mathbb{Z}, +)$:** This groups consists of all integers with the operation of addition
- **Integers modulo n under addition ($\frac{\mathbb{Z}}{n} + mod\ n$):** This is a finite group of integers under addition modulo n.
- **Permutations of n Symbols (S_n):** This group consists of all possible permutations of n symbols.
- **Symmetries of an n-Polygon(D_n):** This group represents the symmetries of an n-sided polygon.
- **Group of Invertible Matrices (G$L_n(\mathbb{R})$):** This is the group of all n x n matrices over the real numbers.

8.2 Subgroups

A subgroup is a subset of a group that itself forms a group under the same operation. To be a subgroup, a subset H of a group G must be closed under the group operation and must include the identity element and inverses of its elements.

8.3 Key concepts

- **Abelian Groups:** Groups where the operation is **commutative** ($a * b = b * a$ for all a, b in G)
- **Multiplication Tables:** A way to visualize the operation in a group, especially useful for finite groups
- **Cancellation Property:** In a group, if $a * b = a * c$, then $b = c$.
- **Cyclic Groups and Subgroups:** Groups generated by a single element.
- **Order of an Element and a Group:** The order of an element is the smallest positive integer m such that $a^m = e$, and the order of a group is its cardinality (the number of elements in the group).

8.4 Preliminaries on Set Theory

Basic Set Theory Concepts:

- **Membership Notation:** $s \in S$ means an element s belongs to the set S
- **Cartesian Product:** SxT represents all the pairs (s, t) where s is from S, and t is from T.
- **Function Equality:** Functions f and g are equal if $f(r) = g(r)$ for all $r \in R$
- **Isomorphism Between Sets:** Sets R and S are isomorphic if there are functions $f : R \implies S$ and $g : S \implies R$ such that composing them equals the identity function in each set.

The notation $f : R \implies S$ means that there is a function f which takes elements from the set R and maps each of them to all elements in the set S. For example, if R was a set of real numbers (including negative numbers), and S was a set of positive numbers, then the function $f : R \implies S$ could be something like $f(x) = x^2$, which maps every real number x to its square, a positive number.

8.5 Definition of a Group

Group Structure:

- **Closed Binary Operation:** A group G has a closed binary operation $\mu : GxG \implies G$
- **Associativity:** The operation is associative, meaning $a \cdot (b \cdot c) = (a \cdot b) \cdot c$ for all $a, b, c \in G$
- **Neutral Element:** There exists a neutral element e in G such that $e \cdot a = a = a \cdot e$
- **Inverse Element:** Each element $a \in G$ has an inverse a^{-1} such that $b \cdot a = e = b \cdot a$

8.6 Subgroups:

Definition and Properties of Subgroups:

- A subset A of a group G is a subgroup if it's closed under the group operation and inverses.
- A subgroup must contain the neutral element of G.

8.7 Homomorphisms and Isomorphisms

Definition Homomorphism:

- A homomorphism between two groups G and H is a function $\alpha : G \implies H$ that preserves the group operation. This means that for any elements g, h in G, the homomorphism satisfies $\alpha(g \cdot h) = \alpha(g) * \alpha(h)$
- Essentially, a homomorphism is a map that respects and preserves the structure of groups.

Definition Isomorphism:

- An Isomorphism is a special type of homomorphism that is bijective (one-to-one and onto).
- If $\alpha : G \to H$ is an isomorphism, there exists another homomorphism $\beta : H \to G$ such that for all g in G and h in H, $\alpha(\beta(h)) = h$ and $\beta(\alpha(g)) = g$. This means that α and β are inverse functions of each other.
- Isomorphisms indicate a very strong level of similarity between groups: they are structurally the same, just with possibly different elements.

8.8 Finite groups

- **Order of a group:** The order of a group G (denoted |G|) is the number of elements in G.
- **Order of an element:** The order of an element g in a group G is the smallest natural number n such that $g^n = e$, where e is the neutral element of the group. If no such n exists, the element's order is infinite.
- **Theorem 7.3:** Every finite group G is a subgroup of the symmetric group S_n, where $n = |G|$. The proof involves showing that there's an injective group homomorphism from G to S_n

8.9 Symmetric groups

- A **symmetric group** S_n represents all possible permutations of a set of n distinct elements. If S_4, then the distinct elements are {1,2,3,4}, and the possible permutations are $4! = 24$. S_4 therefore has 24 elements.
- **Identity element:** In any group there is an identity element that, when combined with any other element, leaves that element unchanged. In S_4, the identity element is the permutation that leaves all elements in their original position, often denoted as id or e, and can be represented as (1 2 3 4)
- **Abelian:** If the group is abelian, then it is commutative, meaning $a \cdot b = b \cdot a$
- **Inverse and Order of Elements:** The inverse is the element that when combined with the original element yields the identity element.

8.10 Cyclic Groups

- **Cyclic Groups:** A group G is cyclic if there exists an element g in G such that every element in G can be expressed as g^k for some integer k.

$$G = g^k | k \in \mathbb{Z}$$

  g is called a generator of G , and we write G = $\langle g \rangle$.

Example:

8.10.1 Theorems on Cyclic Groups:

- If A is a subgroup of a finite cyclic group G with generator g, then A is also cyclic and its order divides |G|.
-

8.10.2 Understanding $\mathbb{Z}/10$

PS: If the group operation was multiplication, it would be $(\mathbb{Z}/n)^*$ instead.

1. **Group Structure:**
   $\mathbb{Z}/10$ is a cyclic group consisting of the equivalence classes of integers modulo 10. This group can be represented by the set 0,1,2,3,4,5,6,7,8,9, with the group operation being addition modulo 10.
2. **Order of the Group:**
   The order of $\mathbb{Z}/10$ is 10, as there are 10 distinct elements in the group.

Criteria for Generators in Cyclic Groups

1. **Generator Definition:** An element g in a cyclic group G is a generator if every element G can be expressed as some power (in this case, a multiple since the operation is addition) of g.
2. **Condition for Generators in $\mathbb{Z}$:** In the group $\mathbb{Z}/n$, an element a is a generator if and only if it is coprime to n. Being coprime means that a and n have no common positive integer factors other than 1, or equivalently, the greatest common divisor (gcd) of a and n is 1.

Finding Generators of $\mathbb{Z}/10$

1. **Checking for Coprimality:** We need to identify the elements in 0,1,2,3,4,5,6,7,8,9 that are coprime to 10.
2. **Elements Coprime to 10:** In this set, the numbers that are coprime to 10 are 1,3,7 and 9.

8.11 Questions

8.11.1 Compute subgroups generated by a given group element.

8.11.2 Check if a group is abelian or not.

8.11.3 Prove that a given set and binary operation satisfies the group axioms.

8.11.4 Computations in known groups: $\mathbb{Z}, \mathbb{Z}/n\mathbb{Z}, D_n, S_n$.

8.12 Quiz

8.12.1 1

Let G be a group, and let $a, b, c \in G$ elements such that $b \cdot a \cdot b^-1 = c$.
What is a?

We need to focus on isolating a by creating identity elements e. **Remember** $b \cdot b^{-1} = e$

$$
\begin{aligned}
&b \cdot a \cdot b^{-1} = c \\
&b^{-1} \cdot (b \cdot a \cdot b^{-1}) = b^{-1} \cdot c \\
&(b^{-1} \cdot b) \cdot a \cdot b^{-1} = b^{-1} \cdot c \\
&e \cdot a \cdot b^{-1} = b^{-1} \cdot c \\
&a \cdot b^{-1} = b^{-1} \cdot c \\
&a \cdot b^{-1} \cdot b = b^{-1} \cdot c \cdot b \\
&a \cdot e = b^{-1} \cdot c \cdot b \\
&a = b^{-1} \cdot c \cdot b
\end{aligned}
$$

8.12.2 2

: **Let G be a group, and let $a, b \in G$ be elements. Simplify the following expression as much as possible:** $a^{-1}aab^{-1}aa^{-1}bb^{-1}bba$

$$
\begin{aligned}
&eab^{-1}ebeba \\
&ab^{-1}bba \\
&aeba \\
&aba
\end{aligned}
$$

The logic is the same as before. Create identity elements, and by doing so, reduce the size of the expression.

8.12.3 3

Compute the following operations within the group $\mathbb{Z}/7$:

- 5+4=2
- 1-2=6
- -4=3
- 6+1=0
- 1+1+1+1+1+1+1+1=1

Remember the wrap-around. That's it.
Example $\frac{5+4}{7}$ gives 2 in rest, therefore 2.
Example 2 $1 - 2 = -1 \implies 7 - 1 = 6$

8.12.4 4

Let g∈G be an element of a group with the property that $g^{12} = e$ is the neutral element. What is the set of possible orders of g?
{1,2,3,4,6,12}
Here you basically look at the factors of 12. We know that when g is multiplied by itself 12 times, it forms the neutral element. However, this does not only mean that the order can only be 12. If by having the order as either 1, 2, 3, 4, or 6 we will achieve the same results, but we will perform respectively 12, 6, 4, 3, and 2 cycles.

8.12.5 5

How many generators are there in the group $\mathbb{Z}/10$?
4
Reasoning, check for coprimes between the elements and 10. I.e., elements that only share the common factor of 1. In this case it is {1, 3, 7, 9}.

8.12.6 6

Let $\mathbb{Z}/6 = \{0, 1, 2, 3, 4, 5\}$ be the set of integers modulo 6.
List all the elements of the subgroup generated by the element $4 \in \mathbb{Z}/6$.
0,2,4.
Se på wrap around

8.12.7 7

Let $Z/6 = \{0, 1, 2, 3, 4, 5\}$ be the cyclic group of integers modulo 6.
List all the elements of the subgroup generated by the element $5 \in \mathbb{Z}/6$
{0,1,2,3,4,5}
Samme som forrige, se på wrap around. Kan også se at 5 og 6 ikke er coprimes.

8.12.8 8

Let $F/2$=0,1 be the cyclic group of integers modulo 2.
List all the elements of the subgroup generated by the element [1,0,1]$\in F/2 \times F/2 \times F/2$

{[0,0,0], [1,0,1]}

Teori:

- **Group $\mathbb{Z}/2$:** This is the set {0,1} with the addition modulo 2 as the operation. It's a cyclic group with 1 as a generator.
- **Group $\mathbb{Z}/2\text{x}\mathbb{Z}/2\text{x}\mathbb{Z}/2$:** This is the direct product of three copies of $\mathbb{Z}/2$. The elements are all possible ordered triples of elements from $\mathbb{Z}/2$, and the group operation is component-wise addition modulo 2.

Subgroup Generated by [1,0,1]
To list all the elements of the subgroup generated by [1,0,1], we repeatedly apply the group operation to [1,0,1].

1. **Zero Applications of the Operation (Identity Element):** [0,0,0]. Every group contains the identity element, which, in this case, is the triple of all zeros.
2. **One Application of the operation:** [1,0,1]. This is the generator itself.
3. **Two applications of the Operation:** Adding [1,0,1] to itself (remember, we are doing modulo 2 for each component):
   $[1, 0, 1] + [1, 0, 1] = [0, 0, 0]$
   This brings us back to the identity element.

Therefore, the subgroup generated by [1,0,1] in $\mathbb{Z}/2\text{x}\mathbb{Z}/2\text{x}\mathbb{Z}/2$ contains only two elements: [0,0,0] and [1,0,1].

9 The multiplicative group of integers mod(n) and Lagrange's theorem

$\equiv$ = is congruent to. Used to express that two numbers have the same remainder when divided by a certain number. $a*b \equiv 1(modn)$ - $a*b$ has the same remainder as 1 when divided by n. Or even simpler, $a*b$ divided by n is 1.

9.1 Introduction to modulo arithmetic

Modulo arithmetic is like looking at a clock. Imagine a clock that instead of going up to 12, goes up to a number n. In modulo n arithmetic:

- When you reach n, you go back to 0.
- Every number you count is actually the remainder you get when you divide that number by n.

9.1.1 Example: Modulo 5 arithmetic

- If you count "1, 2, 3, 4, 5" in modulo 5 arithmetic, 5 would actually be 0 (because 5 divided by 5 leaves a remainder of 0).
- If you keep counting, "6, 7, 8" these would actually be "1, 2, 3" in modulo 5, because 6 divided by 5 leaves a remainder of 1 etc.

9.1.2 Operations in modulo arithmetic

- **Addition:** If you add two numbers, you take the sum and find the remainder when divided by n. For example, in modulo 5 arithmetic, 3 + 4 would be 2, because 3 + 4 = 7, and 7 divided by 5 leaves a remainder of 2.
- **Multiplication:** Similarly for multiplication, multiply the numbers and then find the remainder. In modulo 5, 2*3 is 1, because 2*3=6, and 6 divided by 5 leaves a remainder of 1.
- **Inverses:** A number a has an inverse b if a x b equals 1 modulo n. For instance, in modulo 5, the inverse of 2 is 3 because 2*3=6 which is 1 in modulo 5. **In order for a to have an inverse,** a **and** n **must be coprime**. Once this is established, there exists a number b such that a x b $\equiv$ 1 (mod n).

9.2 Concepts of Multiplication Modulo n

Integers m and n: Consider two numbers, m and n, where m is smaller than n and they have no common factors (except for 1). This is denoted as gcd(m,n)=1, where gcd means 'greatest common divisor'.
Bezout's identity: This principle says that for such numbers m and n, you can always find two integers, x and y, making the equation $xm + yn = 1$ true.

9.2.1 Corollary of Bezout's Identity

Concept: If $gcd(m, n) = 1$, there exists an integer x such that $xm \equiv 1mod(n)$.The yn disappears here, as any multiple of n is congruent to 0. By dividing yn by n no remainder would be left. In the context of modulo n arithmetic, we therefore ignore yn, and say that there exists an x such that xm is equivalent to 1 (mod n), or $xm \equiv 1(modn)$.

The existence of such an x shows that m has a multiplicative inverse in the modulo n system. In simpler terms, multiplying m by this x and reducing the result modulo n yields 1, demonstrating the fundamental property of multiplicative inverses in this arithmetic system.

9.3 New Group P_n

Definition:

$$P_n = \{x|0 < x < nandgcd(x, n) = 1\}$$

with the group operation defined as

$$x * y = xymod(n)$$

.
Let's dissect this a bit first.

- P_n is a set of numbers x, such that $0 < x < n$ and x must be a coprime with n. Being a coprime means that the only number that evenly divides both x and n is 1.

9.3.1 Group operation $x * y = xymod(n)$

A group operation is a special way we can combine two members of the the group P_n, to get another element in the group. Here, we multiply two numbers from the group x and y. Instead of taking the result as is, we use modulo n. This ensures that the outcome is also a member of the group P_n. As a group has a set n, the max value in the group will be $n - 1$.

Group Theory: This set forms a group under modular multiplication. This means that the set, along with the operation (in this case, modular multiplication), meets the criteria that qualify it as a mathematical group. This means that the combination of the set and operation satisfies four key properties:

1. **Closure:** For any two elements a and b in the set, the result of the operation on a and b (denoted a x b in the case of multiplication) is also an element of the set.
2. **Associativity:** The operation is associative. This means that for any three elements a, b and c in the set,
$$(axb)xc = ax(bxc)$$
3. **Identity element:** There is an element in the set (called identity element) that, when used in the operation with any element of the set, leaves the other element unchanged. For multiplication, this identity element is typically 1, sine multiplying any number by 1 leaves it unchanged.
4. **Inverse element:** For every element in the set, there exists another element in the set (its inverse) such that when they are operated together, the result is the identity element. In modular multiplication, this means that for each element a, there exists an element b, such that
$$axb \equiv 1(modn)$$

9.4 Extended Euclidean Algorithm

9.4.1 Bezout's Identity

Bezout's Identity states that for any two natural numbers m and n, there exist integers x and y such that $x \cdot m + y \cdot n = gcd(m, n)$

9.4.2 Extended Euclidean Algorithm

The extended algorithm finds both the gcd, and the integers x and y satisfying Bezout's Identity.

How it works:

1. **Start with Two Linear Equations:**
   - $1 \cdot a + 0 \cdot b = m$
   - $0 \cdot a + 1 \cdot b = n$
   - Represented by the augmented matrix
$$\left[\begin{array}{cc|c} 1 & 0 & m \\ 0 & 1 & n \end{array}\right]$$
2. **Perform Row Operations**

- The extended Euclidean algorithm applies the Euclidean algorithm to the right hand column (containing m and n) by using row operations on the entire matrix.
- If $m < n$, subtract the first row from the second row, and vice versa if $m > n$.
- Repeat this process until the values in the right-hand column are both equal to the gcd of m and n.

3. **Final Matrix and Solutions:**
   - When the process terminates, the matrix takes the form:

$$\left[\begin{array}{cc|c} x & y & gcd(m,n) \\ z & w & gcd(m,n) \end{array}\right]$$

   - This represents a new system of equations:
     - $x \cdot a + y \cdot b = gcd(m,n)$
     - $z \cdot a + w \cdot b = gcd(m,n)$
   - Since we reached this system by elementary row operations from the initial syystem, the solutions for a and b remain m and n, respectively.
4. **Verifying Bezout's Identity:**
   - The final equations become:

$$x \cdot m + y \cdot n = gcd(m,n)$$
$$z \cdot m + w \cdot n = gcd(m,n)$$

   - Both pairs (x,y) and (z,w) satisfy Bezout's Identity.

9.5 Solving modular equation using extended Euclidean algorithm

We can solve modular equations such as $x \cdot 14 \equiv 1 mod 29$ using the algorithm. To do this we need to find the modular multiplicative inverse of 14 modulo 29. The modular inverse of a number a modulo m, if it exists, is the integer x such that $a \cdot x \equiv 1 mod m$

In this case, we are looking for an integer x that satisfies $14 \cdot x \equiv 1 mod 29$. This is equivalent to solving the linear Diophantine equation $14x - 29y = 1$ for integers x and y, since finding x such that $14x - 29y = 1$ will also mean that $14x \equiv 1 mod 29$.

9.6 Modular inverse

9.6.1 EXAM example 2:

Use the extended Euclidean algorithm to find integers x and y such that $18x + 12y = gcd(18, 12)$.

Answer

We begin by creating a set of linear equations based on the initial equation:

1. $1 * a + 0 * b = 18$
2. $0 * a + 1 * b = 12$

Then we change the format to the augmented coefficient matrix of the system of linear equations

$$\left[\begin{array}{cc|c} 1 & 0 & 18 \\ 0 & 1 & 12 \end{array}\right]$$

Then the Euclidean algorithm is performed on the rightmost column, through row operations until m = n:

$$\left[\begin{array}{cc|c} 1 & -1 & 6 \\ 0 & 1 & 12 \end{array}\right]$$

$$\left[\begin{array}{cc|c} 1 & -1 & 6 \\ -1 & 2 & 6 \end{array}\right]$$

From the final matrix we derive the equations

1. $1 * 18 + (-1) * 12 = 18 - 12 = 6$
2. $(-1) * 18 + 2 * 12 = -18 + 24 = 6$

Based on this there are two solutions to the equation $18x + 12y = 6$:

1. $$\underline{\underline{x = 1,\ y = -1}}$$
2. $$\underline{\underline{x = -1,\ y = 2}}$$

9.7 Cosets and Lagrange's Theorem

- **What are cosets?** Cosets are a way of dividing a group into equal-sized, non-overlapping pieces using one of its subgroups.
- **Types of Cosets**
  - **Left Coset/Leading Coset:** Formed by taking an element from the group and multiplying it on the left side of each element of the subgroup.
    $gH = gh : h \in H$
  - **Right Coset:** Formed by multiplying the group element on the right side of each element of the subgroup.
    $Hg = hg : h \in H$
- **Forming a Coset:**
  - Suppose you have a group G and a subgroup H, and you pick an element g from G.
  - The left coset of H in G with respect to g is all the elements you get by multiplying g with each element of H.
- **Properties:**
  - **Equal Size:** All cosets of a subgroup have the same number of elements.
  - **Partitioning the Group:** Cosets partition the group into disjoint sets. This means the whole group is divided into distinct cosets with no overlap.
  - **Number of Cosets:** The number of left cosets of a subgroup in a group is the same as the number of right cosets.

Cosets: If H is a subgroup of G, and g is an element of G, the coset of g and H in G is the subset $gH = \{g * h | h \in H\}$

9.8 Understanding Cosets

1. A coset is a form of 'shifting' or 'translating' a subgroup H by an element from the parent group G. An example is **Problem 9.5** from Handin 9, where $2 + H$ is the set obtained by adding 2 (an element from G) to each element of H.
2. **Relation to the group G:**
   - Cosets partition the group G. This means that each element of G belong to exactly one coset of H in G.

- All cosets of H in G have the same size as H.
- Cosets are either disjoint or identical; they do not partially overlap.

3. **Role of Cosets:**
   - Cosets help in understanding the structure of G relative to H.
   - They play a crucial role in the study of quotient groups and in Lagrange's theorem, which relates to the order of H and G.

9.8.1 Index [G:H]

We need to compute the index [G:H],which is a measure of how many times the subgroup H fits into the group G. In group theory, this is the number of distinct left cosets of H in G.

9.9 Lagrange's Theorem:

The order of a subgroup H of G times the index of H iin G ([G:H]) equals the order of G(|G|).

- Use the Extended Euclidean algorithm to find x and y in Bezout's identity.
- Compute inverses in the group $\mathbb{P}_n$.
- Compute modular exponents: $a^k \mod n$.

9.10 Multiplication modulo n

Integers m and n: Consider two numbers m and n, where m is smaller than n and they have no common factors (except for 1). This is denoted as gcd(m,n)=1, gcd meaning greatest common divisor.
Bezout's identity: This principle says that for such numbers m and n, you can always find two integers x and y, making the equation xm + yn = 1 true. In the context of modulo n arithmetic, this essentially means there exists an integer x such that xm is equivalent to 1(mod$n$)

9.11 Unique Inverses Modulo n

Uniqueness of inverses: If you find another number x' such that x'm = 1 (mod n), and you compare it with the first x, their difference (multiplied by m) will also be 0 modulo n. This proves that the 'inverse' of m under multiplication modulo n is unique.

9.12 The group P_n

Definition: P_n is defined as the set of all positive integers smaller than n and coprime to n.
$P_n = k \in \mathbb{Z} | 1 \leq k < n, gcd(k, n) = 1$
P_n **as a group:** The set P_n forms a group under multiplication modulo n. This means:

- If you multiply any two numbers in P_n their product, when reduced modulo n, is still in P_n.
- Every number in P_n has a multiplicative inverse in P_n. This is guaranteed by Bezout's identity.

Proof of P_n being a group

- **Closed under multiplication:** If a and b are in P_n, then their product ab is also coprime to n.
- **Existence of inverses:** Using Bezout's identity, you can show that every element in P_n has an inverse. Essentially, if you have a in P_n, there is some x such that xa is 1 modulo n, and this x is also in P_n

9.13 Modular exponentiation

$A^B modC = ((AmodC)^B) modC$

9.13.1 Fast modular exponentiation

Example $3^{13} mod 17$

1. **Binary Representation of the Exponent**
   First, convert the exponent 13 into binary. The binary form of 13 is 1101. To find this, start with the highest value less than 13 (8), and place a 1. Then check if the next number is usable (4), place a 1. As adding 2 will make it 14, we cannot add it. Adding 1 however, will make it 13. 1, 2, 4, 8, 16, 32, 64, 128, 256
2. **Initial Setup**
   Begin with two variables **result** initially set to 1, and **base** set to the base of our exponentiation, which is 3 in this case.
3. **Iterate Through Each Binary Digit of the Exponent**
   We examining each bit (binary digit) of the exponent from right to left (least significant to most significant)
4. **Processing Each Binary Digit**
   - **First digit (rightmost '1')**
     - Since the digit is 1, multiply **result** by **base**: $result = 1 \cdot 3 = 3$
     - Then square the **base** and take modulus 17: $base = 3^2 mod 17 = 9$
   - **Second digit ('0')**
     - Since the digit is 0, we do not multiply the **result**.
     - Square the **base** and take modulus 17: $base = 9^2 mod 17 = 13$
   - **Third digit('1')**
     - The digit is 1, so multiply **result** by **base**: $result = 3 \cdot 13 mod 17 = 4$
     - Square the **base** and take modulus 17: $base = 13^2 mod 17 = 16$
   - **Fourth digit ('1')**
     The digit is 1, so multiply **result** by **base**: $result = 4 \cdot 16 mod 17 = 15$
5. **Final result**
   The final value of **result** after this process is 15. Therefore $3^{13} mod 17 = 15$

9.13.2 Sverre's triks

Let a, b and $n > 1$ be integers. By the division algorithm, we can write $a = un + r$ and $b = vn + s$ where $0 \leq u, v < n$. Then

$$ab = (un + r)(vn + s) = uvn^2 + (us + vr)n + rs \equiv rs \ mod(n)$$

In other words, if we want to compute the product $ab \ mod(n)$, we can start by first reducing a and b modulo n, then multiply together the results.

Example:

$$\begin{aligned} 37^3 = 37 \cdot 37 \cdot 37 &\equiv 11 \cdot 11 \cdot 11 \ mod(13) \\ &\equiv 121 \cdot 11 \\ &\equiv 4 \cdot 11 \ mod(13) \\ &\equiv 44 \\ &\equiv 5 \ mod(13) \end{aligned}$$

The rule is that whenever we see a number which is larger or equal to 13, we reduce it modulo 13 before we proceed with multiplication. This way, all of our numbers are kept at a manageable size.

9.14 Question types

9.14.1 Compute the greatest common divisor

In order to find the greatest common divisor between two numbers, the **Euclidean Algorithm** is used.

1. Let a and b be two numbers such that $a > b$
2. Divide the larger number a by the smaller number b.
3. Replace a with b, and b with the remainder from step 1.
4. Repeat step 1 and step 2 until the remainder is zero.
5. Once the remainder is 0, the divisor will be the GCD of a and b.

Example

$$48 = 18 \cdot 2 + 12$$
$$18 = 12 \cdot 1 + 6$$
$$12 = 6 \cdot 2 + 0$$

Hence, the $GCD(48, 18) = 6$

9.14.2 Modular Exponentiation

9.14.3 Linear Diophantine Equations (solved by Extended Euclidean Algorithm)

Find integers x and y such that $42 + 40y = 2$.

9.14.4 Modular Multiplicative Inverse

Find an integer x such that when multiplied by 14 and divided by 29, the remainder is 1.

9.14.5 Compute inverses in the group P_n

Find an integer x such that $x * 14 = 1 mod(29)$

9.15 Quiz

Question 1

Compute the greatest common divisor $\gcd(39, 29)$**.**
Answer: 1.
Reasoning:
Use the Euclidean algorithm:

- $39 = 29 * 1 + 10$
- $29 = 10 * 2 + 9$
- $10 = 9 * 1 + 1$
- $9 = 1 * 9 + 0$

As the final non-zero remainder is 1, this is the GCD.

Question 2

Compute the greatest common divisor $\gcd(51, 44)$**.**
Answer: 1.
Same reasoning as above.

Question 3

Compute the exponent $22^2 \mod 11$**.**
Answer: 0
Here, simply use the calculator.

Question 4

Compute the exponent $52^4 \mod 23$**.**
Answer: 8
Either use the calculator, or use fast modular exponentiation.
Can also use Sverre's Tricks:

$52^4 = 52 \cdot 52 \cdot 52 \cdot 52 \equiv 6 \cdot 6 \cdot 6 \cdot 6 \; mod(23) \quad \equiv 36 \cdot 6 \cdot 6 \equiv 13 \cdot 6 \cdot 6 mod(23) \quad \equiv 78 \cdot 6 \equiv 9 \cdot 6 mod(23) \quad \equiv 54 \equiv 8 \; mod(23)$

Question 5

Compute the exponent $52^5 \mod 23$**. Answer:** 2
Same as 6.

Question 6

Find integers x **and** y **such that** $42x + 40y = 2$**.**
$x =$
$y =$
Answer: $x = 1, y = -1$
This is intuitive enough to solve by hand.

Question 7

Find integers x **and** y **such that** $34x + 51y = 17$**.**
$x =$
$y =$ **Answer:** $x = -1, y = 1$
Same as 6.

Question 8

Find integers x **and** y **such that** $39x + 8y = 1$**.**
$x =$
$y =$
Answer: $x = 5, y = -1$ Use extended Euclidean algorithm

Question 9

Find an integer x **such that** $x \cdot 14 \equiv 1 \mod 29$**.**
$x =$
Answer: $x = -2$ Use extended Euclidean algorithm.

- Convert the equation to $14x - 29y = 1$
- Solve. **SET 29 IN THE MATRIX, NOT -29**

Question 10

Find an integer x **such that** $x \cdot 30 \equiv 1 \mod 19$**.**
$x =$
Answer: 7
Same procedure as above.

Question 11

Find an integer x such that $x \cdot 27 \equiv 1 \mod 23$.
$x =$
Answer: 6
Same procedure as above.

10 Applications of Lagrange's theorem

10.1 Initial information

- **Modulus:** The number by which you're dividing.
- **Congruence Modulo n ($\equiv$):** This symbol means that two numbers give the same remainder when divided by n. For example $8 \equiv 3 \ mod \ 5$, because both 8 and 3 leave a remainder of 3 when divided by 5.
- **Basic operations:** Just like addition, subtraction, and multiplication, but performed with a wrap-around at the modulus.
- **Prime numbers:** Natural numbers greater than 1 that has no positive divisors other than 1 and itself.
- **Coprime:** Two numbers are coprime if their greatest common divisor (GCD) is 1. For example, 8 and 15 are coprime because they share no divisors other than 1.

For addition the identity element i 0, since a number + 0 is the same number.
For multiplication, the identity element is 1, since a number multiplied with 1 is the same number.

10.2 Lagranges theorem

Lagrange's theorem: For a finite group G, the order (number of elements) of any subgroup H of G divides the order of the group G:

$$|G| = |H| \cdot [G : H]$$

Here:

- **g** is an element in the group G
- **|G|** denotes the **order of the group**, which is the **number of elements** in G.
- **|H|** denotes the **order of the subgroup**.
- **[G:H]** denotes the index, $\frac{|G|}{|H|}$
- **e** is the identity element of the group G. This is a special element that, when combined with any element of the group, results in that element itself.

So, if you take any element g in the group, and raise it to the power of the order of the group $|G|$, you will get the identity element e.

The **order** of an **element** g within group G is the smallest positive integer n such that when you multiply g by itself n times (or perform the group operation n times), you get the identity element e.

In simpler terms, no matter which element you pick from a group, if you multiply it by itself as many times as there are elements in the group, you'll end up back at the starting point (the identity element).

10.3 Euler totient function

Counts the number of integers less than n that are coprime to n
ϕ = phi
$\phi(n) = |P_n|$ Two numbers are *coprime* if their **greatest common divisor (GCD) is 1**.
Used in Euler's and Fermat's theorems to determine the exponent in modular exponentiation.

Euler's totient function, often denoted as $\phi(n)$, is a function from the set of positive integers to the set of positive integers. For any positive integer n, $\phi(n)$ is defined as the number of integers from 1 to n that are **coprime** to n.

10.4 Euler's Theorem and Its Implications

10.4.1 Euler's Theorem in Modular Multiplication:

Euler's theorem focuses on modular multiplication, particularly within the multiplicative group of integers modulo n (denoted as P_n). This group, $P_n = \{k \in \mathbb{Z} | 1 \leq k < n, \gcd(k, n) = 1\}$, consists of integers less than n that are coprime to n.

Let a be an element of P_n.
Then $a^{\phi(n)} \equiv 1 \mod n$

- a: This is any integer that is coprime to n, meaning a and n have a greatest common divisor of 1. Such integers a have multiplicative inverses modulo n.
- P_n: This represents the set of integers that are coprime to n. Specifically, it's the set of integers less than n which have no common factors with n other than 1.
- $\phi(n)$: Euler's totient function, which gives the count of the number of elements in P_n, or equivalently, the number of integers less than n that are coprime to n.
- $1 \mod n$: This expression indicates congruence modulo n. Saying that two numbers are **congruent modulo** n means they have the same remainder when divided by n. In the context of Euler's Theorem, it means that $a^{\phi(n)}$ and 1 leave the same remainder when divided by n.

10.4.2 What does it say?

The theorem states that if you take an integer a that is coprime to n, and raise a to the power of $\phi(n)$, then the result is congruent to $1 modulo n$. In simpler terms, when you divide $a^{\phi(n)}$ by n, the remainder is 1.

10.4.3 Corollary to Lagrange's Theorem:

A key corollary arising from Lagrange's theorem states that in any finite group G of order $|G|$, every element g raised to the power of $|G|$ is equal to the neutral element e of G. This means for any element g in G, $g^n = e$, where n is the order of G.

10.4.4 Definition of Euler's Totient Function (ϕ):

Euler's totient function ϕ is defined for any positive integer n as the count of integers less than n that are coprime to n, symbolically, $\phi(n) = |P_n|$.

10.4.5 Lemmas Related to ϕ:

- **Lemma for Prime Numbers:** For a prime number p, $\phi(p) = p - 1$. This is because all positive integers less than a prime number are coprime to it.
- **Multiplicative Property:** For two prime numbers p and q, $\phi(pq) = \phi(p)\phi(q)$. This demonstrates the multiplicative nature of the totient function.

10.4.6 Euler's Theorem Formalized:

Euler's theorem asserts that for any two coprime natural numbers a and n, the expression $a^{\phi(n)}$ is congruent to 1 modulo n, i.e., $a^{\phi(n)} \equiv 1 \mod n$. This is an extension of the corollary to Lagrange's theorem for the group P_n.

10.4.7 Fermat's Little Theorem:

Fermat's Little Theorem is a special case of Euler's theorem. It states that for any prime number p and any integer $1 \leq a < p$, $a^{p-1} \equiv 1 \mod p$. This can be deduced from Euler's theorem and the properties of the totient function for prime numbers.

Note: $\gcd(k, n) = 1$ means the greatest common divisor of k and n is 1, indicating that k and n are coprime.

10.4.8 What does it mean?

The value $\phi(n)$ tells us how many numbers are less than n and have no common factors with n other than 1. It's a measure of the "multiplicative order" of the integers modulo n.

When n is a prime, every number less than n is a coprime to n (since primes have no divisors other than 1 and themselves), so $\phi(n) = n - 1$.

In the case of a product of two distinct primes p and q, the totient function value $\phi(p\dot{q})$ is $(p-1)(q-1)$. This is because each of the p numbers from 1 to p (except the number p itself) will be coprime to q since p is a prime, and vice versa for q. So in the product $p * q$, there will be $p * q$ numbers, but we have to exclude p multiples and q multiples, which gives us $p * q - p - q + 1$.

10.4.9 Example

Compute the value of the Euler totient function: $\phi(3053)$.

1. Prime factorization result = 43*71
2. This means that the number of elements in P_3053 is the number of integers between 1 and 3053 without the factor 43 or 71. This is by definition equal to $\phi(3053)$.
3. In this example, 3053 is the product of two distinct primes, $p = 43$ and $q = 71$. This give $(43-1)(71-1) = (42)(70) = 2940$

10.4.10 Example 2

Compute an example that does not use the same case.

10.5 Fermat's Little Theorem (PRIMALITY TEST)

Use different numbers if necessary, for example 2, 3, 5. Can never confirm unless every number has been tried etc.

If p is a prime, then $a^{p-1} = 1mod(p)$

- p: A prime number, which means it is greater than 1 and has no positive divisors other than 1 and itself.
- a: An integer that is not divisible by p (a is not a multiple of p).
- a^{p-1}:
- $1mod(p)$: When a^{p-1} is divided by p, the remainder is 1.

10.5.1 What does it say?

Fermat's Little Theorem states that if you take any integer a that is not a multiple of a prime number p, and you raise a to the power of $p-1$, the result is congruent to 1 modulo p.

10.5.2 Example:

Use Fermat's little theorem to show that 57, 645, and 10261 are not prime numbers.
To prove that a number is not a prime, we use the formula

$$a^{p-1}mod(p)$$

$$2^{57-1}\ mod\ 57 = 4$$
$$2^5 6 \equiv 4\ mod\ 57)$$

The remainder of 2^56 is 4, and $4 \neq 1$, so 57 is not a prime.
Another way to write it is $2^{56} \equiv 4 \; mod \; 57$, meaning 2^56 and 4 give the same remainder then divided by 57

$$2^{645-1} mod(645) = 1$$
$$3^{644} mod(645) = 36$$

The remainder of 3^644 mod 645 is 36, and $36 \neq 1$, so 645 is not a prime.
Another way to write it is $3^644 \equiv 36 \; mod \; 645$

$$2^{10261-1} \; mod(10261) = 1$$
$$3^{10260} \; mod(10261) = 10200$$

The remainder of 3^10260 $mod(10261)$ is 10200, and $10200 \neq 1$, so 10261 is not a prime.
Another way to write it is $3^10260 \equiv 10200 \; mod \; 10261$.

10.6 Corollary of Fermat's Little Theorem

Used for testing if a number (n) is a prime.
If $a^{n-1}! = 1 mod(n)$ then n is not a prime.
The choice of a can be a random selection from a range of numbers (normally between 2 and n-2 inclusive).
Avoid multiples of n as a.

- a: An integer that you've chosen to test against n.
- n: The number you are testing for **primality**
- a^{n-1}: This represents a raised to the power $n - 1$
- 1 mod (n): This indicates that when a^{n-1} is divided by n, the remainder should be 1 if n is a prime.

10.6.1 What does it say?

The statement says that if you raise any integer a (not a multiple of n) to the power of (n-1), and the result is not congruent to 1 modulo n (meaning the remainder when divided by n is not 1), then n cannot be a prime number.

10.7 RSA encryption

Let p and q be (large) prime numbers.
Let x and y be inverses in the group $P_{phi(n)}$
Let m be the message encoded as an integer mod(n)

- **Public:** x
- **Private:** y
- **Encryption:** $m^x mod(n)$
- **Decryption:** $m^y mod(n)$

Everyone can encrypt, but you need y to decrypt.

10.7.1 RSA works because

1. $(m^x)^y = mmod(n)$
2. It is hard to figure out y without the knowledge of the prime factorization of n. Really what you need is $\phi(n)$, but without p and q this is a lot of work to compute.

10.7.2 Basic Concept of RSA Encyption

1. **Selecting Prime Numbers and Computing 'n'**
   - Start by choosing two distinct prime numbers, p and q.
   - Compute n as the product of p and q: $n = p \cdot q$
2. **Computing Ø(n) (Euler's Totient Function):**
   Calculate $\phi(n)$, which is $(p-1) \cdot (q-1)$. This value represents the count of numbers less than n that are coprime to n.
3. **Choosing Public and Private Keys:**
   - Choose two numbers x and y such that they are less than $\phi(n)$ and satisfy the equation $xy = 1 \ mod \ (\phi(n))$.
     In RSA terminology x is usually denoted as e (encryption component) and y as d (decryption exponent).
   - x and y are chosen in a way that they are inverses of each other under modulo (n). They should also be coprime to $\phi(n)$
   - **What does this mean?**
     - When we say that two numbers are inverses of each other under a certain modulus, we mean that their product, under this modulus is 1.
     - When we say that the numbers satisfy the equation
       $$xy = 1 \ mod \ (\phi(n))$$
       we mean that when we multiply x and y together, and then divide by $\phi(n)$, the remainder is 1.
     - The inverseness is a key property in RSA, because it ensures that the operations of encryption and decryption are reversible.
   - **How to Find x and y**
     - **Choose an Appropriate x:**
       Select a number x that is coprime to n. This means x and n should have no common factors other than 1. Often in RSA, x is choosen as a small prime number like 3,5,17 etc.
     - **Use the Extended Euclidean Algorithm:**
       To find the modular inverse of x under n, apply the Extended Euclidean algorithm.
4. **Encryption Process:**
   To encrypt a message m (where $0 \leq m < n$, compute the encrypted message $e(m)$ as $e(m) = m^x \ mod \ n$
5. **Decryption Process:**
   To decrypt an encrypted message M, compute the original message m using $d(M) = M^y modn$

10.7.3 Why RSA Works

- **Inverse functions**
  e (encryption) and d (decryption) functions are inverses of each other. If $gcd(m,n) = 1$, then $d(e(m)) = (m^x)^y = m^{xy} = m^{1+k\cdot(n)} = m \ mod \ n$, This uses Euler's theorem which states $m^{(n)} = 1 \ mod \ n$ for any m coprime to n.

- **Special Case - When** $gcd(m, n) \neq 1$
  If $gcd(m, n) = p$ (one of the prime factors), the RSA algorithm still works because of congruences modulo p and q. If p divies m, then m is equivalent to 0 $mod\ p$, and so is the decrypted message. The same logic applies to q. This ensures that $d(e(m)) = m\ mod\ (q)$

10.7.4 Example:

Primes: $p = 97, q = 13$
Using p and q, find RSA encryption key x>3, and decryption key y>3, and encrypt the message 102.

10.7.5 Public and Private Keys

- **Public Key:** Consists of the pair (n, x). It is shared publicly and is used for encrypting messages.
- **Private Key:** Consists of the pair (n, y). It's kept secret and is used for decrypting messages.

10.8 Quiz

Question 1

Compute the value of the Euler totient function:
$\phi(3053) =$

1. Prime factorization result = 43*71
2. This means that the number of elements in P_3053 is the number of integers between 1 and 3053 without the factor 43 or 71. This is by definition equal to $\phi(3053)$.
3. In this example, 3053 is the product of two distinct primes, $p = 43$ and $q = 71$. This give $(43 - 1)(71 - 1) = (42)(70) = 2940$

Question 2

In this problem, you are given the primes $p = 97$ and $q = 13$.
Using p and q, find RSA encryption key $x > 3$, and decryption key $y > 3$, and encrypt the message 102.
x:
y:
Encrypted message:
Følg guiden i 10.7.2. **REMEMBER THAT X AND Y SHOULD BE COPRIME TO** $\phi(n)$.
By following this, we get:

- $n = p \cdot q = 97 \cdot 13 = 1261$
- $\phi(n) = (p - 1)(q - 1) = 1152$
- $e = 5$
- $d = 461$ because both are coprime of $\phi(n)$ and $5 \cdot 461 = 2305 = 1\ mod\ 1152$
- $e(m) = e(102) = 102^5\ mod\ 1261 = 215$. Remember to use n as mod in this final part.

11 Coding theory

11.1 Introduction

11.1.1 Binomial Coefficient

The symbol $\binom{n}{m}$, also known as the binomial coefficient, represents the number of ways to choose m elements from a set of n elements without considering the order of selection.

Formula:
$\binom{n}{m} = \frac{n!}{m!(n-m)!}$

1. **Set of n Elements:** Imagine you have a set of n different elements. For example, if $n = 5$, you might have a set like A,B,C,D,E.
2. **Choosing m Elements:** From this set, you want to select m elements. The number of items you choose, m is always less than or equal to the total number of elements in the set.
3. **Without Considering the Order of Selection:** When you select m items from the set, the order in which you pick them doesn't matter. For example, if you choose A, B, and C from the set, it's considered the same combination as choosing B, C, and A. In permutations, the order would matter (ABC would be different from BCA), but in combinations it does not.

11.1.2 Notation

$n, k \in \mathbb{Z}^+$ means that n and k are positive integers, and $n, k \in \mathbb{Z}$ would allow all integers.

- Z_2^n means a set of binary sequences of length n. Each element of the sequences is in the set $\{0, 1\}$.
- **Messages:** the original binary sequences we want to transmit.
- **Code words:** original messages appended with extra bits used for error detection.
- $E_1 : (Z/2) \to (Z/2)^2$ **- Encodes the message consisting of one character, to a code word with two characters.**
  $E1(w1) = w_1w_1$ - The encoding function is a function of w_1, and takes w_1 (one character) as input and encodes it to a **code word** (two characters) where the two characters are both w_1. As w_1 can be both 0 and 1, this leaves the possible codewords:
  $\begin{bmatrix} w & E_1(w) \\ 0 & 00 \\ 1 & 11 \end{bmatrix}$
- **Distances between any two code words:**
  $\begin{bmatrix} E_1(w) & 00 & 11 \\ 00 & 0 & 2 \\ 11 & 2 & 0 \end{bmatrix}$

11.1.3 Additional example

$E_2 : (\mathbb{Z}/2)^{x2} \to (\mathbb{Z}/2)^{x3}$
Encoding function takes a message consisting of two bits, and converts it into a code word of 3 bits.
$E_2(w_1w_2) = w_1w_2x$ where $x = w_1 + w_2 mod 2$
As can be seen, the added bit is a result of addition mod 2.
We create the first matrix from earlier to show the code messages first:
$\begin{bmatrix} w & E_1(w) \\ 00 & 000 \\ 01 & 011 \\ 10 & 101 \\ 11 & 110 \end{bmatrix}$
Notice the mod 2 kicking in in the final row. Now that we have this table, we need to find the distances between any two codewords:

$$\begin{bmatrix} E_1(w) & 000 & 011 & 101 & 110 \\ 000 & 0 & 2 & 2 & 2 \\ 011 & 2 & 0 & 2 & 2 \\ 101 & 2 & 2 & 0 & 2 \\ 110 & 2 & 2 & 2 & 0 \end{bmatrix}$$

REMEMBER: WE ARE LOOKING FOR THE OVERALL DIFFERENCES, NOT DIFFERENCE IN 1s. This means that the difference between 011 and 101 is 2, because the first digits are different and the second digits are different.

11.1.4 Theorem 16.12 (Error Detection)

Context: You have a set of original messages W and their encoded versions C, with C having longer binary sequences than W.
Main Point: To identify all errors that change up to k bits in the encoded message, the smallest number of different bits in any pair of encoded messages (code words) must be at least $k + 1$.

11.1.5 Theorem 16.13 (Error Correction)

Context: Using the same setup as in Theorem 16.12.
Main Point: To correct errors that affect up to k bits in an encoded message, the difference between any pair of encoded messages (in terms of bits) must be at least $2k + 1$.

$E : (Z/2)^2 \to (Z/2)^5$

- E is the encoding function.
- $(Z/2)^2$ denotes the set of all 2-dimensional vectors over the field Z/2 (set of integers mod 2). So $(Z/2)^2$ is just the set of all binary pairs, such as (0,0), (0,1), (1,0), (1,1).
- $(Z/2)^5$ represents the set of all 5-dimensional vectors over the field of Z/2. Binary sequences of length 5.
- So E maps from $(Z/2)^2$ to $(Z/2)^5$

11.1.6 Binary symmetric channel

The channel is binary because an individual string is represented by one of the bits 0 or 1.
When a transmitter sends the signal 0 or 1 in such a channel, associated with either signal is a (constant) probability p for incorrect transmission. When that probability p is the same for both signals, the channel is called *symmetric*.
Here, we have probability p of sending 0 and having 1 received. The probability of sending signal 0 and having it received correctly is then $1 - p$

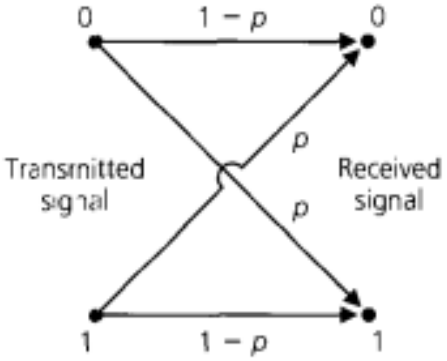

Figure 25: Binary symmetric channel

11.1.7 Example 1

Consider the string c = 10110.
C is an element of the group $\mathbb{Z}_2^5$, formed from the direct product of five copies of $(\mathbb{Z}_2, +)$. When sending each bit (individual signal) of c through the binary symmetric channel, we assume that the probability of incorrect transmission is $p = 0.05$, so that the probability of transmitting c with no errors is $(0.95)^5$. We assume that the transmission of each signal does not depend in any way on the transmission of prior signals. Consequently, the probability of the occurence of all these *independent* events (in their prescribed order) is given by the product of their individual probabilities.

The probability of incorrect transmission for the first bit is 0.05, so witht hte assumption of independent events, $(0.05)(0.95)^4 = 0.041$ is the probability of sending $c = 10110$ and receiving $r = 00110$.

From this, we get $e = 10000$, we can write $c + e = r$ and interpret r as the result of the sum of the original message c and the particular *error pattern* $e = 10000$. Since $c, r, e \in \mathbb{Z}_2^5$ and -1 = 1 in $\mathbb{Z}_2$, we also have $c + r = e$ and $r + e = c$

11.1.8 Probability of differing in two places

If we transmit $c = 10110$, what is the probability that r differs from c in exactly two places?
We sum the probabilities for each error pattern consisting of two 1's and three 0's. Each such pattern has probability 0.002. There are $\binom{5}{2}$ such patterns, so the probability of two errors in transmission is given by

$$\binom{5}{2} \cdot (0.05)^2(0.95)^3 = 0.021$$

11.1.9 Theorem 16.10

Let $c \in \mathbb{Z}_2^n$. For the transmission of c through a binary symmetric channel with probability p of incorrect transmission:

- The probability of receiving $r = c + e$, where e is a *particular* error pattern consisting of k 1's and $(n - k)$ 0's is
$$p^k(1 - p)^{n-k}$$
- The probability that (exactly) k errors are made in the transmission is
$$\binom{n}{k} p^k(1 - p)^{n-k}$$

A binary symmetric channel is considered "good" when $p < 10^-5$.

11.1.10 Improving accuracy of transmission

Adding extra bits to the original message to form a code word.
Encoding process

- Let's say we have a set of messages, W, which we want to transmit.
- Each message, denoted as $w \in W$, is a bitstring of length m.
- In the encoding process, each message w is appended with $n - m$ extra bits, forming a code word c.
- This encoded word, c, is a bitstring of length n and is an element of $\mathbb{Z}_2^n$ (the set of all bitstrings of length n.
- The function E describes this encoding process, turning each message w into a code word c.

Transmission and Noise:

- The encoded word c is then transmitted over the channel.

- Due to noise in the channel, the received word, denoted as $T(c)$ may differ from c. This variation is unpredictable due to the changing nature of the noise.

Decoding Process:

- Upon receiving $T(c)$, a decoding function D is applied.
- The goal of D is to remove the extra bits and, hopefully, recover the original message w.
- Ideally, the composition of D, T, and E should return the original message w, but this is not always possible due to transmission errors

Efficiency and Practicality:

- The efficiency of this coding scheme is measured by the ration m/n, which should be as high as possible to avoid adding too many extra bits.

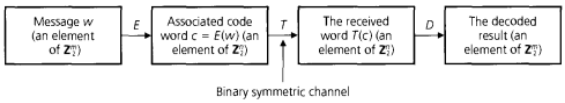

Figure 26: Enter Caption

11.1.11 Probability calculations related to sending messages

$$\underbrace{(1-p)^9}_{\text{All nine bits are correctly transmitted}} + \underbrace{\binom{9}{1}p(1-p)^8}_{\text{One bit is changed in transmission and an error is detected.}}.$$

Figure 27: Enter Caption

11.1.12 Example

Coding Scheme Description

- **Block Code:** This is a $(m+1, m)$ block code with $m = 8$. So, each code word has 9 bits (8 original bits, plus 1 parity bit.
- **Message Set** The set of messages, W is Z_2^8, the set of all 8-bit binary strings.
- **Encoding Function E:**
  - For each message $w = w_1w_2...w_8$ in W, the encoding function $E : Z_2^8 \to Z_2^9$ is defined by $E(w) = w_1w_2...w_8w_9$
  - The 9^{th} bit w_9 is the parity bit, calculated as the sum (modulo 2) of the first 8 bits. This means w_9 is 0 if there's an even number of 1s in the first 8 bits, and 1 if there's an odd number

Examples:

1. For the message 11001101, $E(11001101) = 110011011$ since there are an odd number of 1s in the original message.
2. For the message 00110011, $E(00110011) = 001100110$ since there are an even number of 1s in the original message

Error Detection:

- After the transmission, the received code word $T(c)$ is checked for the number of 1s. If $T(c)$ has an odd number of 1s, we know an error occurred during the transmission
- This method can detect single errors but cannot correct them.

Probability Calculations:

- **Probability of Correct Transmission or Single Error (for $p = 0.001$):**

  - The probability of all nine bits being transmitted correctly is $(0.999)^9$
  - The probability of exactly one bit error occuring is $\binom{9}{1}(0.001)(0.999)^8$
  - Therefore, the total probability is $(0.999)^9 + 9(0.001)(0.999)^8 \approx 0.99996417$

- **Probability of Correct Reception with Retransmission:**
  If an error is detected, the message can be retransmitted. The probability of eventually receiving the correct message is approximately 0.99996393.
- **Probability of Undetected Errors:**
  If an even number of errors occur, they go undetected, and the message is incorrectly accepted as correct.

11.2 Hamming metric

- Consider a code $C \subseteq Z_2^4$, where $c_1 = 0111$, $c_2 = 111 \in C$.
- Both the transmitter and the receiver know the elements of C.
- If the transmitter sends c_1, but the receiver gets T_c1 as 1111, then the receiver will think that c_2 was transmitted, which leads to a decision based on c_2.
- As the two code words are almost the same, this is a challenge. Therefore we need to discuss **closeness**

11.2.1 Closeness

Weight of a Sequence (wt(x)

- The weight of a sequence x, denoted as $wt(x)$, is the count of how many 1s are in the sequence.
- For example if $x = 1100101$, the weight $wt(x)$ is 4 because there are four 1s in the sequence.

Distance Between Two Sequences(d(x,y)):

- If you have two binary sequences, x and y, both of the same length, the distance between them (d(x, y)), is the number of positions where the corresponding components in x and y are different.
- For example,if $x = 10101$ and $y = 10011$, the distance $d(x, y)$ is 2, because the sequences differ in the 3rd and 5th positions.

Example:
For $n = 5$, let $x = 01001$ and $y = 11101$. Then $wt(x) = 2$, and $wt(y) = 4$, and $d(x, y) = 2$. In addition, x+y = 10100, so $wt(x + y) = 2$

When $x, y \in Z_2^n$, we write $d(x, y) = \Sigma_{i=1}^n d(x_1, y_1)$ where

$$\text{for each } 1 \leq i \leq n, \quad d(x_i, y_i) = \begin{cases} 0 & \text{if } x_i = y_i \\ 1 & \text{if } x_i \neq y_i \end{cases}$$

Lemma 16.2 For all $x, y \in Z_2^n, wt(x + y) \leq wt(x) + wt(y)$

Properties of Distance Functions:

- **Non-negativity:** $d(x, y) \geq 0$. This means the distance between any two sequences is always zero or positive.
- **Identity of Indiscernibles:** $d(x, y) = 0$ if and only if $x = y$.
- **Symmetry:** $d(x, y) = d(y, x)$
- **Triangle Inequality:** $d(x, z) \leq d(x, y) + d(y, z)$. The distance from x to z is always less than or equal to the sum of the distances from x to y and from y to z.

When a function satisfies the four properties, it is called a **distance function** or **metric**, and we call ($\mathbf{Z}_2^n$,d) a **metric space**. Therefore, d is often referred to as the **Hamming metric**

Hamming sphere:
For $n, k \in \mathbb{Z}^+$ and $x \in Z_2^n$, the sphere of radius k centered at x is defined as $S(x,k) = \{y \in Z_2^n \mid d(x,y) \leq k\}$.

- n and k are positive integers (Z^+)
- x is a binary sequence of length n. The set Z_2^n contains all possible binary sequences of that length.
- **Sphere of Radius k Centered at x, denoted S(x,k):**
  This is the set of all binary sequences y that are within a Hamming distance of k from the binary sequence x. In other words, it includes every binary sequence y that differs from x in at most k position.
- $S(x,k) = \{y \in Z_2^n \mid d(x,y) \leq k\}$**:** The sphere S with radius k, centered at x includes all sequences y within Z_2^n for which the distance between x and y, $d(x,y)$, is less than or equal to k.

Example:
For $n = 3$ and $x = 110 \in Z_2^3$, $S(x,1) = 110, 010, 100, 111$ and $S(x,2) = 110, 010, 100, 111, 000, 101, 011$. Note that x itself is included, as $d(x,y) \leq k$, **NOT** $d(x,y) = k$

Theorem 16.12 Let $E : W \to C$ be an encoding function with the set of messages $W \subseteq Z_2^m$ and the set of code words $E(W) = C \subseteq Z_2^n$, where $m < n$. If our objective is error detection, then for $k \in Z^+$, we can detect all transmission errors of weight $\leq k$ if and only if the minimum distance between code words is at least $k + 1$.

Simplified: The encoding function maps a set of messages W to a set of code words C. $m < n$ as the code words are longer. To be able to notice when errors happened in up to k bits of our code during transmission, we need to make sure that our code words are designed so that they differ from each other by at least $k + 1$ bits.

Theorem 16.13
Let E, W and C be as in Theorem 16.12. If our objective is error correction, then for $k \in Z^+$, we can construct a decoding function $D : Z_2^n \to W$ that corrects all transmission errors of weight $\leq k$ if and only if the minimum distance between code words is at least $2k + 1$.

11.3 Generator matrix

- **Matrix:** A generator matrix is a matrix used to generate code words in a linear block code.
- **Linear Block Code:** This is a type of error-correcting code where each code word is created by linearly combining a set of basis vectors. These basis vectors are the rows of the generator matrix.

11.3.1 How does it work?

- **Encoding Messages:** To encode a message using a generator matrix, you represent the message as a vector. Then, you multiply this vector by the generator matrix. The result of this multiplication is your code word.
- **Matrix Dimensions:** If you have a generator matrix of size mxn, where m is the number of rows, and n the number of columns, it means that you can encode messages of length m into code words of length n.

11.3.2 Example

Consider a matrix G of size 2 x 4: Let $G = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$

To encode a message $v = \begin{bmatrix} v_1 & v_2 \end{bmatrix}$, you multiply v by G:

code word $= v \times G$

So, if $v = \begin{bmatrix} 1 & 0 \end{bmatrix}$, the code word is $\begin{bmatrix} 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \end{bmatrix}$.

It is common to **partition** the matrix in two, where the first $n/2$ columns create the identity matrix. We can therefore write the matrix as $G = [I_2|A]$ where A is the last part. Now, as we use the matrix to encode, we can further write

$$E(010) = (010)G$$

Brush-Up on Matrix Multiplication:
Multiply v with each of the columns in the matrix, so you get:

- $1 \cdot 1 + 0 \cdot 0 = 1$
- $0 \cdot 1 + 1 \cdot 0 = 0$
- $0 \cdot 1 + 1 \cdot 0 = 0$
- $1 \cdot 1 + 0 \cdot 0 = 1$

Ending up with 1001.

IMPORTANT: REMEMBER THAT 1+1 mod 2 = 0!!!!

11.3.3 Example from book

Let

$$G = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & * & * & * \\ 0 & 1 & 0 & * & * & * \\ 0 & 0 & 1 & * & * & * \end{array}\right]$$

be a 3×6 matrix over Z_2. The first three columns of G form the 3×3 identity matrix I_3. Letting A denote the matrix formed from the last three columns of G, we write $G = [I_3|A]$ to denote its structure. The (partitioned) matrix G is called a generator matrix.

We use G to define an encoding function $E : Z_2^3 \to Z_2^6$ as follows. For $w \in Z_2^3$, $E(w) = wG$ is the element in Z_2^6 obtained by multiplying w, considered as a three-dimensional row vector, by the matrix G on its right. Unlike the results on matrix multiplication in Chapter 7, in the calculations here we have $1 + 1 = 0$, not $1 + 1 = 2$.

We find here, for example, that

$$E(110) = (110)G = [110|***] = [110101]$$

and

$$E(010) = (010)G = [010|***] = [010011]$$

The set of code words obtained by this method is

$$C = \{000000, 100110, 010011, 001101, 110101, 101011, 011110, 111000\} \subseteq Z_2^6$$

and one can recapture the corresponding message by simply dropping the last three components of the code word. **BY MANUAL INSPECTION**, the minimum distance between code words is found to be 3, so we can detect errors of weight ≤ 2 or correct single errors.

11.3.4 Parity-Check Matrix(H)

- The parity-check matrix H is an $(n-m)xn$ matrix and is formed as $[A^T|I_{n-m}]$
- H is used for decoding and error checking. It's designed so that when multiplied by the transpose of valid encoded message $(E(w))^T$, the result is a zero vector.

11.3.5 Full process

- **Using the Generator Matrix G to Encode a Message w:**
  The message w is encoded into a codeword $E(w)$ using the generator matrix G. If w is a k-bit message, then $E(w) = w \cdot G$, resulting in an n-bit codeword.
- **Constructing the Parity-Check Matrix H:**
  To construct H, take the last $n - k$ columns of G (matrix A), and transpose it, and then append an $(n - k)x(n - k)$ identity matrix to it. So $H = [A^T | I_{N-K}]$
- **Decoding Process:**
  - **Calculating the Syndrome:**
    The received codeword r (which may contain errors) is used to calculate the syndrome S. The syndrome is calculated as $S = H \cdot r^T$
  - **Error Detection and Correction:**
    The syndrome S is used to detect and correct errors in r. If S is non-zero, it indicates an error, and the pattern of S helps in locating and correcting the error.
  - **Extracting the Message:**
    Once any errors are corrected, the original message can be extracted from the corrected codeword. For a generator matrix in standard form, the original message is typically the first k bits of the corrected codeword.

Brush-Up on Matrix Transposition Exchange the rows and columns.

11.4 EXAM questions

- Probability calculations related to sending messages over a binary symmetric channel.
- Compute the minimum distance between codewords and its implications on error detection and correction capabilities of the encoding.

11.5 Quiz

Question 1

Given a binary symmetric channel with a probability of transmission failure of $p = 0.01$. What is the probability of making 1 or 2 errors when transmitting the code word 11110? Give the answer as a decimal number, rounded to 4 decimals.

In order to solve this, we use the formula

$$\binom{n}{k} p^k (1-p)^{n-k}$$

Figure 28: n = length, k = errors

The computation therefore becomes:

$$\binom{5}{1} \cdot 0,01^1 \cdot (0.99)^{5-1} + \binom{5}{2} \cdot 0,01^2 \cdot (0.99)^{5-2} \approx 0.0490$$

Question 2

Given a binary symmetric channel with a probability of transmission failure of $p = 0.014$. What is the probability of making 1 or 2 errors when transmitting the code word 11010?

Give the answer as a decimal number, rounded to 4 decimals.
0.0680
Use the same method as in number 1.

Question 3

Let $E : (\mathbb{Z}/2)^3 \to (\mathbb{Z}/2)^6$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 100101, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$.

First, we need to find H.
We transpose A (the latter part of G), and place it first in the matrix.
Then we append the identity matrix at the end.

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Great, now, we need to find the syndrome. To do this we perform the computation $H \cdot r^T$, where r is the codeword received ([1,0,0,1,0,1]). This returns the bitstring [0,1,1].
We check the matrix where the equivalent column to this bitstring is, and find it in number 3. We therefore flip the third bit in the codeword. We therefore go from [1,0,0,1,0,1] to [1,0,1,1,0,1]. Now, the final message can be found by extracting the 3 first bits.

Question 4

Let $E : (\mathbb{Z}/2)^3 \to (\mathbb{Z}/2)^6$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 101111, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$. ANSWER: [1,0,1]
Same procedure as number 3.

Question 5

Let $E : (\mathbb{Z}/2)^3 \to (\mathbb{Z}/2)^6$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 101011, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$. ANSWER: [0,0,1]
Same procedure as last 2.

Question 6

Find the subset containing all elements of $(\mathbb{Z}/2)^4$ with weight equal to 1.
Give the answer as a set of bitstrings. I.e., if you think the answer is the set consisting of 1001, 1010, and 1011, then you should type $\{[1,0,0,1],[1,0,1,0],[1,0,1,1]\}$.
[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]
When the **weight** is equal to 1, this means all the bitstrings that has **exactly** one 1.

Question 7

Find the subset containing all elements of $(\mathbb{Z}/2)^4$ with weight equal to 2.
Give the answer as a set of bitstrings. I.e., if you think the answer is the set consisting of 1001, 1010, and 1011, then you should type $\{[1,0,0,1],[1,0,1,0],[1,0,1,1]\}$.
[1,1,0,0],[1,0,1,0],[1,0,0,1],[0,1,1,0],[0,1,0,1],[0,0,1,1]
When the **weight** is equal to 2, this means all the bitstrings that has **exactly** 2 1s.

Question 8

Let E be the $(5,3)$ block code which encodes $E(w_1w_2w_3) = w_1w_2w_3w_4w_5$ where $w_4 = w_1 + w_2 \mod 2$ and $w_5 = w_1 + w_3 \mod 2$.
What is the maximum number of transmission errors this block code is always able to detect?
1
Reasoning: By creating a table such as has been shown in the introduction, we can demonstrate that the minimum distance between the different codewords is 2. Now, applying Theorem 6.12 which says "to identify all errors that change up to k bits in the encoded message, the smallest number of different bits in any pair of encoded messages must be at least $k+1$. As 2 can be considered $k+1$, k would have to be 1. Therefore the block code is only able to detect maximum 1 transmission errors.

12 Coding theory and group codes

12.1 Definition 16.11

Let $E : Z_2^m \to Z_2^n$, for $n > m$, be an encoding function. The code $C = E(Z_2^m$ is called a **group code** if C is a subgroup of Z_2^n.
Recall the encoding function $E : Z_2^2 \to Z_2^6$ where

- E(00) = 000000
- E(10) = 101010
- E(01) = 010101
- E(11) = 111111

The subset $C = E(Z_2^2) = [000000, 101010, 010101, 111111,$ is a subgroup of Z_2^6, and an example of a group code.

When code words form a group it is easier to compute the minimum distance between code words.

12.1.1 Theorem 16.14

In a group code, the minimum distance between distinct code words is the minimum of the weights of the nonzero elements of the code.

12.1.2 Theorem 16.15

Let $E : Z_2^m \to Z_2^n$ be an encoding function given by a generator matrix G or the associated parity-check matrix H. Then $C = E(Z_2^m)$ is a group code.

12.1.3 Theorem 16.16

Let $C \subseteq Z_2^n$ be a group code for a parity-check matrix H, and let $r_1, r_2 \in Z_2^n$. For the table of cosets of C in Z_2^n, r_1 and r_2 are in the same coset of C if and only if $H \cdot (r_1)^T = H \cdot (r_2)^T$

12.1.4 Theorem 16.17

When we are decoding by coset leaders, if $r \in Z_2^n$ is a received word and r is decoded as the code word $c*$ (which we then decode to recapture the message), then $d(c*, r) \leq d(c, r)$ for all code words c.

12.2 EXAM Questions

- Compute $l_{\min}$ for a given encoding by showing first that it is a group code and then computing the minimum weight of the non-zero codewords.
- Using the syndrome of a message to error correct it using decoding by coset leaders.

12.3 Quiz

Question 1

Let $E : (\mathbb{Z}/2)^{x3} \to (\mathbb{Z}/2)^{x6}$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 100101, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$.

$$[1, 0, 1]$$

Question 2

Let $E : (\mathbb{Z}/2)^{x3} \to (\mathbb{Z}/2)^{x6}$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 101111, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$.

$$[1, 0, 1]$$

Question 3

Let $E : (\mathbb{Z}/2)^{x3} \to (\mathbb{Z}/2)^{x6}$ be the block code given by the generator matrix

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

If a code word is received as 101011, what is the decoded message (using the error correcting scheme associated with the generator matrix G)?
Give the answer as a bitstring. I.e., if you think the answer is 010, then you should type $[0, 1, 0]$.

$$[0, 0, 1]$$

Question 4

Let E be the $(5, 3)$ block code which encodes $E(w_1w_2w_3) = w_1w_2w_3w_4w_5$ where $w_4 = w_1 + w_2 \mod 2$ and $w_5 = w_1 + w_3 \mod 2$.
What is the maximum number of transmission errors this block code is always able to detect?
1
See quiz chapter 11.

12.4 Function over natural numbers

Refers to a mathematical function where the domain (set of inputs) consists of natural numbers.

13 Dihedral

A dihedral group is a mathematical concept used in the field of group theory. It represents a set of symmetries, including rotations and reflection, of a regular polygon. The term "dihedral" comes from the Greek words for "two sides", reflecting that these groups are related to two-dimensional symmetris.

13.1 Understanding Dihedral Groups Through Symmetries

To understand dihedral groups better, think of them as describing all the different ways you can move a shape so that it looks the same as it did before the move. These movements are called symmetries. There are two main types:

1. **Rotations:** TUrning the shape around its center
2. **Reflections:** Flipping the shape over a line (an axis of symmetry).

Each symmetry operation can be seen as an element of the dihedral group. The total number of these symmetry elements (rotations and reflections) forms the complete dihedral group for a particular shape.

13.2 Dihedral Group of a Square (d_4)

Now, let's apply this to a specific example: a square. The dihedral group of a square is denoted as D_4. Why D_4? The "D" standsfor dihedral, and the "4" signifies that the square has four sides. In D_4 there are eight elements (symmetry operations):

1. **Four rotations**:
   - No rotation (keeping the square as is).
   - Rotation by 90 degrees.
   - Rotation by 180 degrees.
   - Rotation by 270 degrees.
2. **Four reflections**:
   - Reflection over the vertical axis.
   - Reflection over the horizontal axis.
   - Reflection over one diagonal.
   - Reflection over the other diagonal.

Each of these operations returns the square to a position where it looks identical to its original position. This is key in defining the symmetry group. This is key in defining the symmetry group. In group theory, these operations are not just random actions but follow specific rules that align with the properties of a group, such as having an associative operation, an identity element (the no rotation in this case), and every element (operation) having an inverse.

13.3 Assignment example:

Find a subgroup $H \subset D_4$ such that:

- $H \subset D_4$. I.e. $|G| > |H| > 1$
- The index $[D_4 : H]$ is maximal among all non-trivial subgroups. I.e. there does not exist a non-trivial subgroup with a strictly greater index.

13.3.1 Initial clearing:

1. **Subgroup** $H \subset D_4$: A subgroup of D_4 is a set of elements from D_4 that also form a group under the same operation (in this case, symmetries of a square).

2. **Trivial Subgroup:** The trivial subgroups are the smallest and largest possible subgroups of any group. For D_4 these would be the subgroup containing only the identity element (no rotation) and the entire group D_4 itself.
3. **Index of a Subgroup:** The index of a subgroup h in D_4, denoted as $[D_4 : H]$, is the number of distinct left cosets of H in D_4. It's calculated as the order (size) of D_4 divided by the order (size) of H.

13.3.2 Let's find a suitable H

1. **Order of D_4:** As mentioned earlier, D_4 has 8 elements. Therefore $|D_4| = 8$.
2. **Finding H:** We need H to be non-trivial, meaning $|H| > 1$, but also not equal to D_4 itself, so $|H| < 8$. Additionally, we want the index $[D_4 : H]$ to be maximal. To maximize the index, we need to minimize the order of H, but keep it greater than 1. The smallest non-trivial size for H is 2, so let's find a subgroup of D_4 with an order of 2.
   A natural choice for H is a subgroup containing the identity (no rotation) and a 180-degree rotation. This subgroup is $H = e, r^2$, where e is the identity element and r^2 is the 180-degree rotation.
3. **Order of H:** $|H| = 2$
4. **Index $[D_4 : H]$:** The index is $[D_4 : H] = \frac{|D_4|}{|H|} = \frac{8}{2} = 4$.
5. **Maximality of the Index:** To prove that this index is maximal among non-trivial subgroups, consider that any other non-trivial subgroup of D_4 must have more than 2 elements, but fewer than 8. The only options are orders 3,4,5,6, and 7. However, since 8 is not divisible by 3,5,6, or 7, no subgroups of these orders exist. The only other possible order is 4, which would have an index of 2 (less than 4). Therefore, our chosen H indeed has the maximal index among all non-trivial subgroups of D_4

In conclusion, the subgroup $H = e, r^2$ of D_4 is a non-trivial subgroup with the maximal index among all non-trivial subgroups of D_4.

14 Binary numbers

1. 0
2. 1
3. 10
4. 11
5. 100
6. 101
7. 110
8. 111
9. 1000
10. 1001
11. 1010
12. 1011
13. 1100
14. 1101
15. 1110
16. 1111
17. 10000
18. 10001
19. 10010
20. 10011
21. 10100

15 Prime numbers

1. 2
2. 3
3. 5
4. 7
5. 11
6. 13
7. 17
8. 19
9. 23
10. 29
11. 31
12. 37
13. 41
14. 43
15. 47

16. 53
17. 59
18. 61
19. 67
20. 71
21. 73
22. 79
23. 83
24. 89
25. 97

15.1 Other numbers

Number Type	Description
Natural Numbers ($\mathbb{N}$)	The set of positive integers (1, 2, 3, ...). Sometimes zero is included.
Whole Numbers	The set of non-negative integers (0, 1, 2, 3, ...).
Integers ($\mathbb{Z}$)	The set of whole numbers and their negatives (... -3, -2, -1, 0, 1, 2, 3 ...).
Rational Numbers ($\mathbb{Q}$)	Numbers that can be expressed as the quotient or fraction $\frac{p}{q}$ of two integers, where p is the numerator and q is the non-zero denominator.
Irrational Numbers	Numbers that cannot be expressed as a simple fraction - their decimal expansions are non-repeating and non-terminating. Examples include $\sqrt{2}$ and π.
Real Numbers ($\mathbb{R}$)	All the numbers on the continuous number line, including all rational and irrational numbers.
Complex Numbers ($\mathbb{C}$)	Numbers that have a real part and an imaginary part, expressed in the form $a + bi$, where a and b are real numbers, and i is the imaginary unit.

16 Exponential Rules

- Product Rule: $a^m \times a^n = a^{m+n}$
- Quotient Rule: $\frac{a^m}{a^n} = a^{m-n}$ (for $a \neq 0$)
- Power Rule: $(a^m)^n = a^{mn}$
- Power of a Product Rule: $(ab)^n = a^n b^n$
- Power of a Quotient Rule: $\left(\frac{a}{b}\right)^n = \frac{a^n}{b^n}$ (for $b \neq 0$)
- Zero Exponent Rule: $a^0 = 1$ (for $a \neq 0$)
- Negative Exponent Rule: $a^{-n} = \frac{1}{a^n}$ (for $a \neq 0$)

17 Logarithm Rules

- Product Rule: $\log_a(xy) = \log_a(x) + \log_a(y)$
- Quotient Rule: $\log_a\left(\frac{x}{y}\right) = \log_a(x) - \log_a(y)$
- Power Rule: $\log_a(x^n) = n \log_a(x)$
- Change of Base Formula: $\log_a(b) = \frac{\log_c(b)}{\log_c(a)}$

- Logarithm of One: $\log_a(1) = 0$ (for $a > 0, a \neq 1$)
- Logarithm of Base: $\log_a(a) = 1$ (for $a > 0$)

"|" can be considered as where in set notation.

Subset vs proper subset. A set can be both a proper subset and a subset. Sammenlign med geq.

18 12 fo sho